



NEXT.JS · APP ROUTER · SDE2 & SSE

Next.js

App Router, Server Components, నాలుగు caching
పొరలు, rendering strategies – Next 15 మార్పులతో సహా.

Next.js లో అతి కష్టమైన భాగం caching – ఏది ఎక్కడ cache అవుతోందో
తెలియకపోతే bugs ఊహించలేం. ఆ నాలుగు పొరలని ఇక్కడ విడదీశాం.

విషయ సూచిక (Table of Contents)

Part 1 – పునాదులు & Routing

1. Next.js అంటే ఏమిటి, ఎందుకు (React vs Next, framework decisions, versions history)
2. Setup, Project Structure & File Conventions
3. Routing Fundamentals (segments, dynamic, catch-all, route groups)
4. Layouts, Templates, Pages, Loading, Error, Not-Found (file conventions deep)
5. Advanced Routing – Parallel Routes, Intercepting Routes, Route Groups patterns
6. Navigation – `<Link>`, `useRouter`, `redirect`, prefetching internals

Part 2 – Rendering Model (Next.js యొక్క గుండె)

7. Rendering Strategies – Static, Dynamic, Streaming, PPR (ఏది ఎప్పుడు)
8. Server Components vs Client Components (boundary rules, composition patterns)
9. Streaming & Suspense (loading.tsx, granular boundaries, SEO ప్రభావం)
10. Hydration, Client Boundaries & `use client` cost

Part 3 – Data Fetching & Caching (అత్యంత ముఖ్యం)

11. Data Fetching in App Router (fetch, ORM, parallel vs sequential, preload pattern)
12. **Caching Deep – 4 Layers** (Request Memoization, Data Cache, Full Route Cache, Router Cache)
13. Revalidation Strategies (time-based, on-demand, tags, `revalidatePath`)
14. Next.js 13 → 14 → 15 caching మార్పులు (`use cache`, `dynamicIO`)

Part 4 – Mutations, Forms & APIs

15. Server Actions Deep (`'use server'`, security, patterns, limitations)
16. Forms – `useActionState`, `useFormStatus`, `useOptimistic`, progressive enhancement
17. Route Handlers (API routes) – REST, streaming, webhooks, CORS
18. Middleware & Edge Runtime (auth gating, rewrites, geo, limits)

Part 5 – Production Concerns

19. Authentication & Authorization (sessions, cookies, DAL pattern, middleware ఎందుకు సరిపోదు)
20. Database & ORM (Prisma/Drizzle, serverless connection pooling, transactions)
21. Images, Fonts, Metadata & SEO (`next/image`, `next/font`, sitemap, robots, OG)
22. Styling (Tailwind, CSS Modules, CSS-in-JS ఎందుకు సమస్య)
23. Error Handling, Redirects, Not-Found & Logging
24. Internationalization (i18n routing, dictionaries, RSC తో)
25. Performance Optimization (bundle analysis, dynamic imports, Web Vitals, PPR)
26. Deployment – Vercel, self-hosting, Docker standalone, env vars, CDN

27. Testing Next.js apps (unit, integration, E2E, RSC testing సవాళ్లు)

Part 6 – Architecture & Interview

28. Scalable Project Architecture (folder structure, layers, monorepo)

29. Pages Router → App Router Migration

30. Anti-Patterns & Common Pitfalls Cheat-Sheet

31. Rapid-Fire Interview Q&A (SDE2 + SSE)

32. Memory Tips – ఒకసారి చదివి గుర్తుపెట్టుకునే framework

Part 1 – పునాదులు & Routing

1. Next.js అంటే ఏమిటి, ఎందుకు

వివరణ

Next.js = React కోసం ఒక full-stack framework. Vercel దీన్ని 2016 లో విడుదల చేసింది. React ఒక **library** – అది UI rendering మాత్రమే చేస్తుంది. మిగతావన్నీ (routing, data fetching, bundling, SSR, API layer, image optimization, caching, deployment) మీరే ఎంచుకోవాలి. **Next.js ఆ నిర్ణయాలన్నీ మీ కోసం తీసుకుంటుంది.**

React ఇచ్చేది:

Components
State & hooks
Reconciliation

Next.js అదనంగా ఇచ్చేది:

File-based routing
SSR / SSG / ISR / Streaming
Server Components (RSC) + Server Actions
API routes (backend!)
Image / Font / Script optimization
Bundling, code splitting (automatic)
Caching (4 layers)
Middleware & Edge runtime
Metadata / SEO APIs

Real-life Scenario

React = ఇంజిన్ కోనడం. అద్భుతమైన ఇంజిన్, కానీ దానితో మీరు ఇంకా ప్రయాణించలేరు – chassis, wheels, steering, brakes, seats అన్నీ మీరే వెతికి, సరిపోయేలా అమర్చుకోవాలి (React Router + Vite + TanStack Query + Express backend + ...).

Next.js = పూర్తి కారు. ఇంజిన్ అదే (React), కానీ మిగతా అన్నీ ఇప్పటికే సరిగ్గా అమర్చి ఉన్నాయి, పరీక్షించబడ్డాయి. **నష్టం:** కారు design మీ ఇష్టం కాదు – framework నియమాలు పాటించాలి.

Next.js ఎందుకు – నిజమైన కారణాలు

కారణం	వివరణ
SEO & first-load వేగం	Server లో HTML render → crawler కి content వెంటనే; LCP మెరుగు
Zero-config performance	Code splitting, prefetching, image/font optimization automatic
Full-stack in one repo	Frontend + API + DB access ఒకే codebase (BFF అవసరం లేదు)
RSC – client bundle తగ్గింపు	Non-interactive UI కి zero JavaScript
Decisions ready	Team వాదించాల్సిన అవసరం లేదు (routing/data/caching అన్నీ ఇచ్చేసింది)
Deployment	Vercel లో git push = deploy; self-host కూడా సాధ్యం

ఎప్పుడు Next.js వద్దు (honest view – interview లో ఇది చెప్తే maturity)

- **Internal dashboard / admin panel** (login వెనుక, SEO అవసరం లేదు) → Vite + React SPA సరళం, చవక, వేగం.
- **ఇప్పటికే బలమైన backend ఉంది** (Java/Go/Django) → Next.js server features duplicate అవుతాయి; SPA + ఆ API మేలు.
- **Static blog/docs మాత్రమే** → Astro/Hugo తేలికైనవి (JS తక్కువ).
- **Team కి React అనుభవం లేదు** → RSC mental model కష్టం.
- **Serverless-unfriendly workloads** (long-running jobs, websockets) → వేరే server అవసరం.

App Router vs Pages Router (ముఖ్యమైన నిర్ణయం)

	Pages Router (legacy)	App Router (2026 default)
Folder	pages/	app/
Data fetching	getServerSideProps , getStaticProps , getInitialProps	async Server Components, fetch
Layouts	_app.js hacks (state పోతుంది)	nested layout.tsx (state నిలుస్తుంది) ✓
RSC	✗	✓
Streaming	పరిమితం	✓
Server Actions	✗	✓
Loading/Error UI	manual	file conventions (loading.tsx , error.tsx) ✓
Nested routing	పరిమితం	✓
స్థితి	maintained (deprecated కాదు)	సిఫార్సు

Versions timeline (interview లో అడుగుతారు)

Version	సంవత్సరం	ముఖ్య మార్పు
9	2019	API routes, <code>getServerSideProps</code> / <code>getStaticProps</code>
9.5	2020	ISR (Incremental Static Regeneration)
12	2021	Middleware, Rust compiler (SWC)
13	2022	App Router (beta) , RSC, streaming, <code>next/image</code> v2, Turbopack alpha
13.4	2023	App Router stable , Server Actions (alpha)
14	2023	Server Actions stable, Partial Prerendering (preview), Turbopack dev
15	2024-25	React 19 , caching defaults మారాయి (fetch/GET routes ఇక cache కావు), <code>async params</code> / <code>searchParams</code> , Turbopack stable (dev), <code>after()</code> , forms improvements
15.x	2025-26	<code>use cache</code> directive, <code>dynamicIO</code> , PPR స్థిరీకరణ

Next 15 యొక్క అతిపెద్ద మార్పు: *"caching by default" → "uncached by default"*. Next 13/14 లో `fetch` automatic గా cache అయ్యేది – ఇది చాలామందిని గందరగోళపరిచింది (stale data bugs). Next 15 లో మీరు **explicit** గా cache అడగాలి.

Next.js ఎలా run అవుతుంది – mental model

Build time (next build)

- └ Static routes → HTML + RSC payload ముందే generate → CDN
- └ Dynamic routes → server function గా bundle
- └ Client components → JS chunks

Request time

- └ Static route → CDN నుండి నేరుగా (server touch కాదు) ✗
- └ ISR route → cached HTML + background revalidate
- └ Dynamic route → server render → stream HTML + RSC payload

↓

Browser: HTML paint → JS download → hydrate client components → interactive

Gotchas (సాధారణ తప్పులు)

- **"Next.js = SSR" అనుకోవడం** – Next.js **default** గా **static**; dynamic అవ్వాలంటే dynamic APIs (`cookies()` , `headers()` , `searchParams`) వాడాలి లేదా `dynamic = "force-dynamic"` .
- **App Router ని Pages Router లాగా వాడటం** – `useEffect` + `useState` తో data fetch చేయడం (server components ఉన్నా).
- **Next 13/14 tutorials ని Next 15 లో follow చేయడం** – caching behaviour పూర్తిగా వేరు.

- అన్నిటికీ **Next.js** – SPA సరిపోయే చోట అనవసర complexity.
- **Vercel మాత్రమే deploy option అనుకోవడం** – self-host (Docker standalone) పూర్తిగా సాధ్యం.

Key Points

- Next.js = React + routing + rendering + data + API + optimization **decisions ready**.
- **App Router** = RSC + nested layouts + streaming + Server Actions (2026 default).
- Next 15 = React 19 + **uncached-by-default** + async `params` .
- Next.js **default static** – dynamic అనేది opt-in (dynamic APIs ద్వారా).
- SPA/internal tools కి Next.js అనవసరం కావచ్చు.

Interview దృష్టి

- "React బదులు Next.js ఎందుకు?" → SEO/LCP, RSC bundle తగ్గింపు, full-stack, zero-config perf, ready decisions – **ఎప్పుడు వద్దే కూడా** చెప్పడం ముఖ్యం.
- "App vs Pages Router?" → పై table + migration సాధ్యం.
- "Next 15 లో ఏం మారింది?" → caching defaults, React 19, async params, Turbopack.

2. Setup, Project Structure & File Conventions

వివరణ

```
npx create-next-app@latest my-app
# ✓ TypeScript? Yes
# ✓ ESLint? Yes
# ✓ Tailwind CSS? Yes
# ✓ src/ directory? Yes      ← సిఫార్సు (config files వేరుగా ఉంటాయి)
# ✓ App Router? Yes
# ✓ Turbopack for dev? Yes    ← వేగం
# ✓ Import alias? @/*

npm run dev      # dev server (Turbopack)
npm run build    # production build
npm start        # production server
npm run lint
```

File conventions – Next.js యొక్క "magic" ఇక్కడే

`app/` folder లో నిర్దిష్ట పేర్లు ఉన్న files కి ప్రత్యేక అర్థాలు ఉన్నాయి:

File	పని
<code>page.tsx</code>	ఆ route యొక్క UI (ఇది ఉంటేనే route publicly accessible)
<code>layout.tsx</code>	Shared UI (navigation మారినా re-render అవుతుంది, state నిలుస్తుంది)
<code>template.tsx</code>	Layout లాంటిది కానీ ప్రతి navigation కి కొత్త instance (state reset)

File	పని
loading.tsx	Automatic <code><Suspense></code> fallback
error.tsx	Automatic Error Boundary (<code>'use client'</code> తప్పనిసరి)
global-error.tsx	Root layout errors (own <code><html></code> / <code><body></code> ఉండాలి)
not-found.tsx	404 UI (<code>notFound()</code> call చేసినప్పుడు)
route.ts	API endpoint (<code>page.tsx</code> తో ఒకే folder లో ఉండకూడదు)
default.tsx	Parallel routes కి fallback
middleware.ts	Request-level interception (root లో , app/ బయట)
instrumentation.ts	Observability hooks (server startup)

Project structure (సిఫార్సు)

```

my-app/
├── src/
│   ├── app/
│   │   ├── layout.tsx          # ⚠️ routing మాత్రమే – logic ఇక్కడ కాదు
│   │   ├── page.tsx           # root layout (తప్పనిసరి – html/body ఇక్కడ)
│   │   ├── globals.css        # "/"
│   │   ├── (marketing)/
│   │   │   ├── layout.tsx
│   │   │   ├── about/page.tsx # "/about"
│   │   │   └── pricing/page.tsx
│   │   ├── (app)/
│   │   │   ├── layout.tsx     # authenticated area
│   │   │   │   └── dashboard/
│   │   │   │       ├── page.tsx
│   │   │   │       ├── loading.tsx
│   │   │   │       └── error.tsx
│   │   ├── products/
│   │   │   ├── page.tsx       # "/products"
│   │   │   └── [id]/
│   │   │       ├── page.tsx   # "/products/42"
│   │   │       └── opengraph-image.tsx
│   │   └── api/
│   │       └── webhooks/stripe/route.ts
│   ├── components/
│   │   ├── ui/                # design system (Button, Input...)
│   │   └── features/          # feature-specific components
│   ├── lib/
│   │   ├── db.ts              # DB client singleton
│   │   ├── auth.ts            # session helpers
│   │   ├── dal.ts             # 🔑 Data Access Layer (auth + queries)
│   │   └── utils.ts
│   ├── server/
│   │   ├── actions/           # Server Actions
│   │   └── services/          # business logic
│   ├── types/
│   └── middleware.ts

```

```
├─ public/                                # static assets (/logo.png గా serve అవుతాయి)
├─ next.config.ts
└─ tsconfig.json
```

బంగారు నియమం: `app/` folder లో **routing files మాత్రమే** ఉండాలి. Components, utils, business logic బయట (`components/` , `lib/` , `server/`) – లేకపోతే route tree అర్థంకాకుండా పెరుగుతుంది. (Colocation సాధ్యమే – `app/` లో ఉన్న non-special files routes కావు – కానీ discipline కోసం వేరుగా ఉంచడం మేలు.)

next.config.ts

```
import type { NextConfig } from "next";

const config: NextConfig = {
  reactStrictMode: true,

  images: {
    remotePatterns: [{ protocol: "https", hostname: "cdn.example.com", pathname:
"/images/**" }],
    formats: ["image/avif", "image/webp"],
  },

  experimental: {
    ppr: "incremental",           // Partial Prerendering
    typedRoutes: true,           // <Link href> ని type-safe చేస్తుంది ✨
    // dynamicIO: true,         // కొత్త caching model
  },

  // Self-hosting కి (Docker) – minimal server bundle
  output: "standalone",

  async redirects() {
    return [{ source: "/old-blog/:slug", destination: "/blog/:slug", permanent: true
}];
  },
  async headers() {
    return [{
      source: "/(.*)",
      headers: [
        { key: "X-Content-Type-Options", value: "nosniff" },
        { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
      ],
    }];
  },
};
export default config;
```

Environment variables (ఇక్కడ security bugs సాధారణం)

```
# .env.local (git లో commit చేయొద్దు)
DATABASE_URL="postgres://..."           # 🗝 server-only
```

```
JWT_SECRET="..." # 🔒 server-only
NEXT_PUBLIC_APP_URL="https://x.com" # 🌐 PUBLIC - bundle లోకి వెళ్తుంది!
```

File priority: .env.local > .env.\${NODE_ENV} > .env

⚠️ **NEXT_PUBLIC_ prefix ఉన్న ప్రతి variable బ్రౌజర్ bundle లో plain text గా ఉంటుంది.**

API keys, DB URLs, secrets ఎప్పుడూ ఈ prefix తో పెట్టొద్దు. Server code లో `process.env.SECRET` నేరుగా వాడొచ్చు (అది client కి వెళ్తుంది).

```
// lib/env.ts - startup validation (Topic 27 in TypeScript_Telugu.md)
import { z } from "zod";
const schema = z.object({
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  NEXT_PUBLIC_APP_URL: z.string().url(),
});
export const env = schema.parse(process.env); // ❌ invalid అయితే build/boot fail ✗
```

server-only / client-only packages (అత్యంత ఉపయోగకరం)

```
// lib/db.ts
import "server-only"; // 🔑 ఈ file ని client component import చేస్తే BUILD ERROR
import { PrismaClient } from "@prisma/client";
export const db = new PrismaClient();
```

```
npm i server-only client-only
```

ఇది **accidental secret leaks ని build time లోనే ఆపుతుంది**. ప్రతి DB/secret module లో `import "server-only"` పెట్టడం production best practice.

Gotchas

- **Root layout.tsx లో <html> / <body> లేకపోవడం** → error (ఇది తప్పనిసరి).
- **page.tsx లేని folder** → ఆ route accessible కాదు (404).
- **page.tsx + route.ts ఒకే folder లో** → conflict error.
- **middleware.ts ని app/ లోపల పెట్టడం** → పని చేయదు (root లేదా `src/` root లో).
- **NEXT_PUBLIC_ లో secrets** → public leak.
- **Env variables build time లో inline అవుతాయి** (client కి) → Docker లో runtime env మార్చినా client bundle మారదు!
- **app/ లో business logic కుప్పబెట్టడం** → అర్థంకాని route tree.

Key Points

- File conventions = Next.js యొక్క API (`page` , `layout` , `loading` , `error` , `route` , `middleware`).
- `app/` = routing మాత్రమే; logic బయట.
- `NEXT_PUBLIC_` = public; **server-only package** = accidental leaks ఆపుతుంది.
- `next.config` – images, redirects, headers, `output: "standalone"` (self-host), `typedRoutes` .

Interview దృష్టి

- "Next.js లో secrets ఎలా handle చేస్తావు?" → server-only env + **server-only** package + env validation + `NEXT_PUBLIC_` ప్రమాదం.
- "App folder structure ఎలా organize చేస్తావు?" → routing/logic separation, route groups, DAL.

3. Routing Fundamentals

వివరణ

File-based routing: folder structure = URL structure.

<code>app/page.tsx</code>	→ <code>/</code>	
<code>app/about/page.tsx</code>	→ <code>/about</code>	
<code>app/blog/page.tsx</code>	→ <code>/blog</code>	
<code>app/blog/[slug]/page.tsx</code>	→ <code>/blog/hello-world</code>	(dynamic)
<code>app/shop/[...slug]/page.tsx</code>	→ <code>/shop/a/b/c</code>	(catch-all)
<code>app/docs/[...slug]/page.tsx</code>	→ <code>/docs</code> మరియు <code>/docs/a/b</code>	(optional catch-all)
<code>app/(marketing)/about/page.tsx</code>	→ <code>/about</code>	(route group – URL లో " (marketing)" లేదు)
<code>app/@modal/login/page.tsx</code>	→ parallel route slot	
<code>app/api/users/route.ts</code>	→ <code>/api/users</code>	(API endpoint)

Real-life Scenario

File-based routing = ఇంటి address system. గల్లీ పేరు (folder), ఇంటి నంబరు (folder), ఆ ఇంట్లో ఉన్న వ్యక్తి (`page.tsx`). Address చెప్తే ఎక్కడికి వెళ్ళాలో స్పష్టం – వేరే "route registry" file maintain చేయనవసరం లేదు. `[id]` = "ఈ గల్లీలో ఏ ఇంటికైనా" అనే generic rule.

Dynamic routes & params (Next 15 లో async!)

```
// app/products/[id]/page.tsx
// ⚠️ Next 15: params మరియు searchParams ఇప్పుడు Promise!
export default async function ProductPage({
  params,
  searchParams,
}): {
```

```

    params: Promise<{ id: string }>;
    searchParams: Promise<{ [key: string]: string | string[] | undefined }>;
  }) {
    const { id } = await params; // 🔑 await తప్పనిసరి (Next 15)
    const { sort } = await searchParams;

    const product = await getProduct(id);
    if (!product) notFound();

    return <ProductView product={product} sort={sort} />;
  }

  // Next 14 లో: function Page({ params }: { params: { id: string } }) – await లేదు

```

ఎందుకు మార్చారు: `searchParams` route ని dynamic చేస్తుంది. వాటిని Promise చేయడం వల్ల Next.js **PPR (Partial Prerendering)** లో static shell ని ముందే render చేసి, dynamic భాగాలను తర్వాత stream చేయగలదు.

Catch-all routes

```

// app/docs/[...slug]/page.tsx → /docs/a/b/c
const { slug } = await params; // slug: ["a", "b", "c"]

// app/docs/[...slug]/page.tsx → /docs కూడా match అవుతుంది
const { slug } = await params; // /docs → slug: undefined

```

వాడకాలు: CMS pages, documentation, file browsers, multi-level categories.

`generateStaticParams` – build time లో pages ముందే generate

```

// app/blog/[slug]/page.tsx
export async function generateStaticParams() {
  const posts = await getPosts();
  return posts.map((post) => ({ slug: post.slug })); // ✅ ప్రతి slug కి static page
}

// 🌀 generateStaticParams లో లేని slug వస్తే ఏమవుతుంది?
export const dynamicParams = true; // (default) → on-demand render + cache (ISR లాంటిది)
export const dynamicParams = false; // → 404

```

ఇది Pages Router లోని `getStaticPaths` కి సమానం. పెద్ద catalogs కి: top 1000 products ని ముందే generate చేసి మిగతావి `dynamicParams: true` తో on-demand – ఇది build time ని పేలిపోకుండా కాపాడుతుంది.

Route Groups (folder) – URL మార్చుకుండా organize చేయడం

```

app/
├── (marketing)/           # URL లో కనిపించదు
│   └── layout.tsx        # marketing pages కి వేరే layout (header/footer)

```

```

├── page.tsx → /
├── pricing/page.tsx → /pricing
├── (app)/
│   ├── layout.tsx # authenticated app shell (sidebar)
│   └── dashboard/page.tsx → /dashboard
├── (auth)/
│   ├── layout.tsx # centered card layout
│   ├── login/page.tsx → /login
│   └── register/page.tsx → /register

```

ఇది ఎందుకు శక్తివంతం: ఒకే URL level లో **వేర్వేరు layouts** ఇవ్వచ్చు. Route groups లేకపోతే `/login` మరియు `/dashboard` రెండూ root layout ని share చేయాల్సి వస్తుంది.

Private folders `_folder`

```

app/
├── _components/ # routing నుండి పూర్తిగా మినహాయంపు (URL కాదు)
├── _lib/
└── dashboard/page.tsx

```

Colocation – ఏ files routes అవుతాయి?

```

app/dashboard/
├── page.tsx ✓ route (/dashboard)
├── Chart.tsx ✗ route కాదు (సాధారణ component – colocated)
├── utils.ts ✗ route కాదు
└── styles.module.css ✗ route కాదు

```

`page.tsx` (లేదా `route.ts`) ఉన్న folder మాత్రమే **publicly accessible**. మిగతా files ఎన్ని ఉన్నా route కావు.

Route priority (conflict resolution)

```

static > dynamic > catch-all > optional catch-all

/blog/new → app/blog/new/page.tsx (static గెలుస్తుంది)
/blog/hello → app/blog/[slug]/page.tsx
/blog/a/b → app/blog/[...slug]/page.tsx

```

Gotchas

- **Next 15 లో `params` await చేయకపోవడం** → error/warning (Next 14 code copy చేస్తే).
- **`searchParams` వాడితే route dynamic అవుతుంది** – static optimization పోతుంది (తెలియకుండా).
- **Route group folder కి `layout.tsx` ఇవ్వకపోవడం** → group యొక్క ప్రయోజనం లేదు.
- **రెండు route groups లో ఒకే path** (`(a)/about` మరియు `(b)/about`) → build error.
- **`generateStaticParams` లేకుండా వేల dynamic pages** → అన్నీ runtime render (నెమ్మది + ఖరీదు).

- **Folder పేరు లో** `[id]` **vs** `[Id]` – case-sensitive; `params.id` key exact match కావాలి.

Key Points

- Folder = URL; `page.tsx` ఉంటేనే route.
- `[id]`, `[...slug]`, `[...slug]` = dynamic, catch-all, optional catch-all.
- **Next 15:** `params / searchParams` = **Promise** (await చేయాలి).
- `(group)` = URL మార్పుకుండా layouts విభజన; `_folder` = private.
- `generateStaticParams` = build-time static generation (+ `dynamicParams`).

Interview దృష్టి

- "Route groups ఎందుకు?" → ఒకే URL level లో వేర్వేరు layouts + organization.
- "1 లక్ష products కి pages ఎలా?" → top N static via `generateStaticParams` + `dynamicParams: true` + ISR.

4. Layouts, Templates, Loading, Error, Not-Found

వివరణ – file conventions deep

```
app/
├── layout.tsx      → అన్ని pages చుట్టూ (persistent)
├── template.tsx    → layout లాంటిది కానీ ప్రతీసారి కొత్తది
├── page.tsx        → route UI
├── loading.tsx     → automatic <Suspense fallback>
├── error.tsx       → automatic <ErrorBoundary>
├── not-found.tsx   → 404
└── global-error.tsx → root layout errors
```

ఇవి కలిసి ఇలా nest అవుతాయి (Next.js automatic గా చేస్తుంది):

```
<Layout>
  <ErrorBoundary fallback={<Error />}>
    <Suspense fallback={<Loading />}>
      <ErrorBoundary fallback={<NotFound />}>
        <Page />
      </ErrorBoundary>
    </Suspense>
  </ErrorBoundary>
</Layout>
```

Root layout (తప్పనిసరి)

```
// app/layout.tsx
import type { Metadata } from "next";
import { Inter } from "next/font/google";
```

```
import "../globals.css";

const inter = Inter({ subsets: ["latin"], display: "swap", variable: "--font-inter"
});

export const metadata: Metadata = {
  title: { default: "MyShop", template: "%s | MyShop" }, // child pages: "Products
| MyShop"
  description: "Best products online",
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="te" className={inter.variable}>
      <body>
        <Providers>{children}</Providers>      {/* client providers ఇక్కడ */}
      </body>
    </html>
  );
}
```

Root layout నియమాలు: `<html>` మరియు `<body>` తప్పనిసరిగా ఉండాలి; ఇది **Server Component** గా ఉండాలి ('use client' పెడితే మొత్తం app client bundle లోకి!); navigation లో ఇది **re-render అవ్వదు**.

Nested layouts – Next.js యొక్క పెద్ద ప్రయోజనం

```
// app/dashboard/layout.tsx – /dashboard/* అన్నిటికీ
export default async function DashboardLayout({ children }: { children:
React.ReactNode }) {
  const user = await getCurrentUser();      // ✅ server లో data fetch

  return (
    <div className="flex">
      <Sidebar user={user} />                {/* ✅ navigation లో re-render అవ్వదు */}
    </div>
  );
}
```

కీలకం: `/dashboard/analytics` → `/dashboard/settings` navigate చేస్తే – **layout అలాగే ఉంటుంది** (sidebar scroll position, open menus, form state అన్నీ నిలుస్తాయి), `children` మాత్రమే మారుతుంది. ఇది SPA feel ఇస్తుంది.

layout vs template

	layout.tsx	template.tsx
Navigation లో	అదే instance (state నిలుస్తుంది)	కొత్త instance (state reset, effects re-run)

	layout.tsx	template.tsx
DOM	reuse	recreate
ఎప్పుడు	95% cases 	enter/exit animations, per-page analytics, per-navigation useEffect

```
// app/template.tsx - ప్రతి navigation కి fade-in animation
"use client";
import { motion } from "framer-motion";
export default function Template({ children }: { children: React.ReactNode }) {
  return <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }}>{children}</motion.div>;
}
```

loading.tsx – automatic streaming

```
// app/dashboard/loading.tsx
export default function Loading() {
  return <DashboardSkeleton />; // ✗ instant – server data వేచి ఉండదు
}
```

ఏం జరుగుతుంది: Next.js `page.tsx` ని automatic గా `<Suspense fallback=<Loading />>` లో wrap చేస్తుంది. Server data fetch చేస్తున్నప్పుడు **shell + skeleton వెంటనే stream** అవుతుంది → user ఖాళీ screen చూడడు.

 **loading.tsx = మొత్తం page skeleton.** మెరుగైనది – **granular <Suspense>** (Topic 9): వేగంగా వచ్చే భాగాలు వెంటనే చూపించి, నెమ్మది భాగాలకే skeleton.

error.tsx – error boundary

```
// app/dashboard/error.tsx
"use client"; // 🔑 తప్పనిసరి (error boundaries client components)

export default function Error({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  useEffect(() => {
    Sentry.captureException(error, { tags: { digest: error.digest } });
  }, [error]);

  return (
    <div role="alert">
      <h2>ఏదో తప్పు జరిగింది</h2>
      <p>{process.env.NODE_ENV === "development" ? error.message : "మేము దీన్ని"}</p>
    </div>
  );
}
```

```

    పరిశీలిస్తున్నాము."}</p>
    <button onClick={reset}>మళ్ళీ ప్రయత్నించండి</button>
  </div>
);
}

```

error.digest – production లో server error messages **client కి పంపబడవు** (security). బదులు ఒక hash (digest) పంపుతుంది; అదే hash server logs లో ఉంటుంది → correlate చేయొచ్చు.

✖ error.tsx ఏం పట్టుకోదు:

- అదే segment యొక్క **layout.tsx** లో వచ్చిన **errors** (error boundary layout లోపల ఉంటుంది) → parent segment యొక్క **error.tsx** పట్టుకుంటుంది.
- **Root layout errors** → **global-error.tsx** అవసరం.
- Event handler errors, async code (React error boundary నియమాలే).

```

// app/global-error.tsx – root layout errors (own html/body ఉండాలి!)
"use client";
export default function GlobalError({ error, reset }: { error: Error; reset: () => void }) {
  return (
    <html><body>
      <h2>Application error</h2>
      <button onClick={reset}>Retry</button>
    </body></html>
  );
}

```

not-found.tsx & notFound()

```

// app/products/[id]/page.tsx
import { notFound } from "next/navigation";

export default async function Page({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const product = await getProduct(id);
  if (!product) notFound(); // 🔑 దగ్గరి not-found.tsx render + HTTP 404
  return <ProductView product={product} />;
}

// app/products/[id]/not-found.tsx
export default function NotFound() {
  return <div><h2>Product దొరకలేదు</h2><Link href="/products">అన్ని products</Link>
</div>;
}

```

notFound() ఒక **special error throw** చేస్తుంది – దాని తర్వాతి code run అవ్వదు (TypeScript దీన్ని **never** గా గుర్తిస్తుంది, కాబట్టి తర్వాత **product** non-null ✅).

redirect() & permanentRedirect()

```
import { redirect, permanentRedirect } from "next/navigation";

export default async function Page() {
  const session = await getSession();
  if (!session) redirect("/login"); // 307 (temporary)
  if (movedPermanently) permanentRedirect("/new-url"); // 308
}

// ⚠️ redirect() కూడా throw చేస్తుంది → try/catch లోపల వాడితే catch పట్టుకుంటుంది!
// ✅ try block బయట, లేదా catch లో re-throw
```

Route segment config (ప్రతి page/layout లో export చేయొచ్చు)

```
export const dynamic = "auto"; // "auto" | "force-dynamic" | "error" | "force-static"
export const revalidate = 3600; // ISR - సెకన్లు (false = ఎప్పటికీ, 0 = ఎప్పుడూ dynamic)
export const fetchCache = "auto";
export const runtime = "nodejs"; // "nodejs" | "edge"
export const preferredRegion = "bom1"; // Mumbai (data కి దగ్గరగా)
export const maxDuration = 30; // serverless timeout (సెకన్లు)
```

Gotchas

- `error.tsx` కి `'use client'` మర్చిపోవడం → build error.
- `error.tsx` అదే level layout errors ని పట్టుకుంటుందని అనుకోవడం – పట్టుకోదు.
- Root layout ని `'use client'` చేయడం → మొత్తం app client bundle లోకి (RSC ప్రయోజనం సున్నా) ✨.
- `loading.tsx` మాత్రమే వాడి granular Suspense మర్చిపోవడం → నెమ్మది భాగం మొత్తం page ని ఆపుతుంది.
- `redirect()` ని try/catch లోపల → redirect పని చేయదు (error గా పట్టుబడుతుంది).
- `notFound()` తర్వాత code రాయడం – అది ఎప్పటికీ run అవ్వదు.
- `template.tsx` ని అలవాటుగా వాడటం → అనవసర remounts, state loss, నెమ్మది.

Key Points

- `layout` = persistent (state నిలుస్తుంది); `template` = per-navigation remount.
- `loading.tsx` = automatic Suspense; `error.tsx` = automatic ErrorBoundary (**client**).
- `error.digest` = production లో server↔client error correlation.
- Root layout: `<html>` / `<body>` తప్పనిసరి, ఎప్పుడూ Server Component.
- `notFound()` / `redirect()` throw చేస్తాయి – try/catch జాగ్రత్త.

Interview దృష్టి

- "layout vs template?" → state persistence + remount behaviour + వాడకాలు.
- "error.tsx ఏం పట్టుకోదు?" → same-segment layout, root layout (→ global-error), events/async.

5. Advanced Routing – Parallel & Intercepting Routes

వివరణ

Next.js యొక్క రెండు శక్తివంతమైన (కానీ తక్కువ అర్థమయ్యే) features.

Parallel Routes `@folder` – ఒకే layout లో అనేక స్వతంత్ర pages

```
app/
├── layout.tsx           # props: { children, team, analytics }
├── page.tsx             # → children slot
├── @team/
│   ├── page.tsx
│   ├── loading.tsx     # ✅ ప్రతి slot కి own loading/error!
│   ├── error.tsx
│   └── default.tsx     # 🗝️ fallback (soft navigation లో అవసరం)
└── @analytics/
    ├── page.tsx
    └── default.tsx
```

```
// app/layout.tsx – slots props గా వస్తాయి
export default function Layout({
  children,
  team,
  analytics,
}: {
  children: React.ReactNode;
  team: React.ReactNode;
  analytics: React.ReactNode;
}) {
  return (
    <div className="grid grid-cols-2 gap-4">
      <div className="col-span-2">{children}</div>
      <div>{team}</div>
      <div>{analytics}</div>
    </div>
  );
}
```

ఎందుకు శక్తివంతం:

- ప్రతి slot కి **స్వతంత్ర loading/error states** → ఒక widget fail అయినా మిగతావి పని చేస్తాయి.
- ప్రతి slot **స్వతంత్రంగా stream** అవుతుంది (dashboard widgets కి perfect).
- Conditional rendering – role ఆధారంగా వేరే slot చూపించడం.

```
// Conditional slots – role-based dashboards
export default async function Layout({ admin, user }: { admin: React.ReactNode; user: React.ReactNode }) {
  const session = await getSession();
  return session.role === "admin" ? admin : user; // ✅ రెండూ విడిగా render
}
```

default.tsx ఎందుకు అవసరం: user /settings కి soft-navigate చేస్తే, @team slot కి ఆ URL కి matching page ఉండకపోవచ్చు. అప్పుడు Next.js default.tsx render చేస్తుంది. లేకపోతే **hard refresh** లో **404**.

Intercepting Routes (.) folder – modal pattern ★

సమస్య: Instagram లో photo మీద click చేస్తే **modal** లో తెరుచుకుంటుంది, కానీ URL మారుతుంది (/photo/123). ఆ URL ని share చేస్తే – **పూర్తి page** గా తెరుచుకోవాలి. Refresh చేసినా full page.

```
app/
├── layout.tsx                # { children, modal }
├── @modal/
│   ├── default.tsx          # null return
│   └── (.)photo/[id]/page.tsx # 🔑 intercepted → modal గా
├── photo/[id]/page.tsx      # direct visit → full page
└── page.tsx                 # feed
```

Convention:

```
(.)folder    → అదే level
(..)folder   → ఒక level పైన
(..)(..)folder → రెండు levels పైన
(...)folder  → root నుండి
```

```
// app/@modal/(.)photo/[id]/page.tsx
import { Modal } from "@components/Modal";
export default async function PhotoModal({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const photo = await getPhoto(id);
  return <Modal><PhotoView photo={photo} /></Modal>;
}

// app/@modal/default.tsx
export default function Default() { return null; }

// app/layout.tsx
export default function Layout({ children, modal }: { children: React.ReactNode; modal: React.ReactNode }) {
  return <>{children}{modal}</>;
}
```

ఫలితం:

దారి	ఏం జరుగుతుంది
Feed లో photo click (soft navigation)	Modal తెరుచుకుంటుంది, URL /photo/123 ✅
/photo/123 నేరుగా type/share/refresh	Full page ✅

దారి	ఏం జరుగుతుంది
Modal లో back button	Feed కి తిరిగి 

```
// components/Modal.tsx – router.back() తో close
"use client";
export function Modal({ children }: { children: React.ReactNode }) {
  const router = useRouter();
  return createPortal(
    <div className="overlay" onClick={() => router.back()}>
      <div className="modal" onClick={(e) => e.stopPropagation()}>
        {children}
        <button onClick={() => router.back()}>x</button>
      </div>
    </div>,
    document.body
  );
}
```

వాస్తవిక ఉపయోగాలు

Feature	ఉపయోగం
Parallel routes	Dashboards (independent widgets), role-based views, split views, tabs with independent loading
Intercepting routes	Photo/product modals, login modal, quick-view, cart drawer – shareable URL తో

Gotchas

- **default.tsx మర్చిపోవడం** → hard navigation లో 404.
- **Intercepting routes soft navigation లో మాత్రమే పని చేస్తాయి** (`<Link>` ద్వారా); direct URL = full page (ఇది feature).
- **Parallel route slots URL ని ప్రభావితం చేయవు** – `@modal` URL లో కనిపించదు.
- **అతిగా వాడటం** – mental model సంక్లిష్టం; team కి అర్థం కాకపోతే maintenance నరకం.
- **Modal state ని client state లో ఉంచడం** (`useState`) – URL share/back button పోతాయి; intercepting routes యొక్క ప్రయోజనమే అది.

Key Points

- **Parallel routes (`@slot`)** = ఒకే layout లో స్వతంత్ర pages + own loading/error + independent streaming.
- **Intercepting routes (`(.)`)** = soft navigation లో modal, direct URL లో full page.
- `default.tsx` = unmatched slots కి fallback (తప్పనిసరి).
- ఇవి **URL-driven UI** ని సాధ్యం చేస్తాయి (shareable, back-button-friendly modals).

Interview దృష్టి

- "Instagram లాంటి photo modal ఎలా build చేస్తావు?" → intercepting + parallel routes – ఇది impressive answer.
- "Dashboard widgets independent గా load అవ్వాలంటే?" → parallel routes + per-slot loading/error.

6. Navigation & Prefetching

వివరణ

```
import Link from "next/link";
import { useRouter, usePathname, useSearchParams, useParams } from "next/navigation";

// 1. <Link> – declarative (default ఎంపిక)
<Link href="/products">Products</Link>
<Link href={`/${products/${id}}`} prefetch={false}>Details</Link>
<Link href="/dashboard" replace>Dashboard</Link>           {/* history replace */}
<Link href="/x" scroll={false}>No scroll reset</Link>

// 2. useRouter – programmatic (⚠️ "next/navigation", "next/router" కాదు!)
"use client";
const router = useRouter();
router.push("/checkout");
router.replace("/login");
router.back();
router.forward();
router.refresh();           // 🔑 server components ని re-fetch (client state నిలుస్తుంది)
router.prefetch("/heavy-page");

// 3. Server-side navigation
import { redirect } from "next/navigation";
redirect("/login");         // server component / server action లో
```

Real-life Scenario

Prefetching = అతిథి వచ్చే ముందే టీ పెట్టడం. Link viewport లోకి రాగానే Next.js ఆ page యొక్క code+data ని **background లో download** చేస్తుంది. User click చేసేసరికి అది ఇప్పటికే సిద్ధం → **instant navigation**. ఇదే Next.js apps "వేగంగా అనిపించడానికి" ప్రధాన కారణం.

Prefetching – ఎలా పని చేస్తుంది (internals)

Production లో:

- <Link> viewport లోకి వస్తుంది (IntersectionObserver)
- ↓
- Next.js ఆ route యొక్క RSC payload ని fetch చేస్తుంది
- ↓
- Router Cache లో నిల్వ (Next 15: static routes 5 min, dynamic 0s by default)

Click → cache నుండి instant render ✗

prefetch={true} → పూర్తి route prefetch (static)
prefetch={null} → (default) static భాగం + loading.tsx వరకు
prefetch={false} → prefetch లేదు (వందల links ఉన్న pages కి)

Dev mode లో prefetching disabled – "నా app production లో వేగంగా ఉంది కానీ dev లో నెమ్మది" అనేది సాధారణం, ఇదే కారణం.

Soft vs Hard Navigation

	Soft (<Link> , router.push)	Hard (<a> , refresh, window.location)
Page reload	✗	✓
Client state	నిలుస్తుంది ✓	పోతుంది
Layout re-render	✗ (shared layouts reuse)	✓ మొత్తం
Router cache	వాడుతుంది	clear అవుతుంది

<a href> ఎప్పుడూ వాడొద్దు internal links కి – full reload, state loss, నెమ్మది.

router.refresh() – తక్కువ అర్థమయ్యే కానీ ముఖ్యమైన API

```
"use client";
const router = useRouter();

const onDelete = async (id: string) => {
  await deleteItem(id);
  router.refresh(); // 🔑 server components re-fetch → కొత్త data
  // ✓ client state (scroll, form inputs, open modals) నిలుస్తుంది
  // ✓ full page reload కాదు
};
```

ఎప్పుడు: Server Action వాడనప్పుడు (external API call తర్వాత), లేదా **revalidatePath** సరిపోనప్పుడు.

URL state – useSearchParams

```
"use client";
import { useSearchParams, usePathname, useRouter } from "next/navigation";
import { useCallback } from "react";

function Filters() {
  const searchParams = useSearchParams();
  const pathname = usePathname();
  const router = useRouter();
```

```
const setParam = useCallback((key: string, value: string | null) => {
  const params = new URLSearchParams(searchParams.toString());
  if (value === null) params.delete(key);
  else params.set(key, value);
  params.delete("page"); // filter మరితే page reset
  router.push(`${pathname}?${params.toString()}`, { scroll: false });
}, [searchParams, pathname, router]);

return <select value={searchParams.get("sort") ?? ""} onChange={(e) =>
  setParam("sort", e.target.value)}>...</select>;
}
```

⚠ **useSearchParams ఒక client component ని వాడే page ని dynamic చేస్తుంది** – ఆ component ని **<Suspense>** లో wrap చేయాలి, లేకపోతే static generation లో error:

```
<Suspense fallback=<FiltersSkeleton />><Filters /></Suspense>
```

Server component లో search params

```
// ✅ Client component అవసరం లేదు – server లోనే చదవొచ్చు
export default async function Page({ searchParams }: { searchParams: Promise<{ q?: string; page?: string }> }) {
  const { q, page = "1" } = await searchParams;
  const results = await search(q, Number(page));
  return <Results data={results} />;
}
```

useLinkStatus (Next 15.3+) – navigation pending indicator

```
"use client";
import { useLinkStatus } from "next/link";
function LoadingIndicator() {
  const { pending } = useLinkStatus();
  return pending ? <Spinner /> : null;
}
<Link href="/slow"><LoadingIndicator />Go</Link>
```

Gotchas

- **next/router import చేయడం** (Pages Router) → App Router లో **next/navigation** ✅.
- **<a> internal links కి** → full reload.
- **useSearchParams ని Suspense లేకుండా** → static build error / మొత్తం page dynamic.
- **Client component లో useRouter కి 'use client' మర్చిపోవడం.**
- **Prefetch dev లో పని చేయదని తెలియకపోవడం** → performance ని తప్పుగా అంచనా.
- **వందల links ఉన్న page లో prefetch on** → network flood (**prefetch={false}** వాడాలి).
- **router.push తర్వాత data stale** → **router.refresh()** లేదా **revalidatePath** అవసరం.

Key Points

- `<Link>` = soft navigation + **automatic prefetch** (viewport-based, production only).
- `useRouter` (`next/navigation`), `redirect()` (server).
- `router.refresh()` = server components re-fetch, client state నిలుపుతూ.
- URL state = shareable/bookmarkable; server లో `searchParams` prop, client లో `useSearchParams` (+Suspense).

Interview దృష్టి

- "Next.js navigation ఎందుకు instant?" → prefetch + RSC payload + router cache + soft navigation.
 - " `router.refresh()` ఎప్పుడు?" → mutation తర్వాత server data refresh, client state కోల్పోకుండా.
-

Part 2 – Rendering Model

7. Rendering Strategies – Static, Dynamic, Streaming, PPR

RENDERING STRATEGIES · STATIC, DYNAMIC, ISR, STREAMING

RENDERING STRATEGIES

Static (SSG)	build time lo render CDN నుంచి – అత్యంత వేగం
Dynamic (SSR)	ప్రతి request కి render తాజా data, నెమ్మది
ISR	static + కాలానుగుణ refresh రెండింటి మేలు కలయిక
Streaming	భాగాలుగా పంపడం నెమ్మది భాగం మిగతాదాన్ని ఆపదు

App Router lo ఇది ఎలా నిర్ణయమవుతుంది

మీరు ఎండుకోరు – మీరు **ఏం వాడారో** దాన్ని బట్టి Next నిర్ణయిస్తుంది.
cookies(), headers(), searchParams వాడితే → dynamic.
ఏదీ వాడకపోతే → static. అందుకే "నా page ఎందుకు static అయింది?" అనే ప్రశ్న వస్తుంది.

వివరణ – Next.js లో ఒక page ఎలా render అవుతుంది

Static (SSG/prerendered)	→ build time లో HTML → CDN → అత్యంత వేగం
ISR	→ static + నిర్దిష్ట కాలానికి regenerate
Dynamic (SSR)	→ ప్రతి request కి server render
Streaming	→ dynamic కానీ chunks గా (Suspense boundaries)
PPR	→ static shell + dynamic holes (ఒకే page లో!) ★
Client (CSR)	→ browser లో render ('use client' + client fetch)

Real-life Scenario

Static = ముందే print చేసిన menu cards – వేలమందికి ఒకే card, తక్షణం ఇవ్వచ్చు (CDN).

Dynamic = ప్రతి customer కి వ్యక్తిగతంగా రాసిన bill – సమయం పడుతుంది, కానీ

personalized. **ISR** = రోజుకోసారి కొత్త menu print – తాజాదనం + వేగం రెండూ. **Streaming** =

starter, main course, dessert వచ్చినకొద్దీ పంపడం. **PPR** = ముందే print చేసిన menu, కానీ

"ఈరోజు special" అనే ఖాళీ గడి ఒక్కటే చేత్తో నింపడం ★ – 95% ముందే సిద్ధం, 5% మాత్రమే live.

Static (default) – ఏమీ చేయకపోతే ఇదే

```
// app/about/page.tsx – ఏ dynamic API లేదు → build time లో HTML generate
export default function About() {
  return <h1>About Us</h1>;
}
```

```
}  
// ✅ CDN నుండి serve → TTFB ~20ms
```

Route ఎప్పుడు dynamic అవుతుంది (అత్యంత ముఖ్యం)

Next.js ఈ కింది వాటిలో ఏదైనా వాడితే route ని **automatic గా dynamic** చేస్తుంది:

```
import { cookies, headers, connection } from "next/headers";  
import { unstable_noStore as noStore } from "next/cache";  
  
await cookies(); // ✗ dynamic  
await headers(); // ✗ dynamic  
await draftMode(); // ✗ dynamic  
searchParams (page prop) // ✗ dynamic  
useSearchParams() (Suspense లేకుండా) // ✗ dynamic  
export const dynamic = "force-dynamic";  
fetch(url, { cache: "no-store" }) // ✗ (Next 15 లో fetch default no-store, కానీ route  
dynamic కాదు!)
```

⚠️ **Next 15 సూక్ష్మత:** uncached `fetch` route ని **dynamic చేయదు**. Route dynamic అవ్వాలంటే పైన ఉన్న **dynamic APIs** వాడాలి. ఇది Next 14 నుండి మార్పు – గందరగోళానికి ప్రధాన కారణం.

ISR – Incremental Static Regeneration

```
// Route-level  
export const revalidate = 3600; // ప్రతి గంటకి  
  
// Fetch-level  
const data = await fetch(url, { next: { revalidate: 60 } });  
  
// On-demand (webhook/CMS update తర్వాత)  
import { revalidatePath, revalidateTag } from "next/cache";  
revalidateTag("products");
```

ఎలా పని చేస్తుంది (stale-while-revalidate):

- మొదటి request → generate → cache → serve
- తర్వాతి requests (revalidate window లోపల) → cached ✅ instant
- Window దాటాక మొదటి request → **stale version serve చేస్తుంది** (వేగం) + background లో regenerate → cache update
- తర్వాతి requests → కొత్త version

అంటే ఏ user కూడా regeneration కోసం వేచి ఉండడు – ఇదే ISR యొక్క అందం.

Streaming SSR

```
export default function Page() {
  return (
    <>
      <Header />                                     {/* వెంటనే */}
      <Suspense fallback=<ProductsSkeleton />>
        <Products />                                  {/* నెమ్మది - ready అయ్యాక
stream */}
      </Suspense>
      <Suspense fallback=<ReviewsSkeleton />>
        <Reviews />                                   {/* స్వతంత్రంగా */}
      </Suspense>
    </>
  );
}
```

TTFB మెరుగుదల: streaming లేకపోతే – server అన్ని data వచ్చేవరకు ఏమీ పంపదు (2s). Streaming తో – shell 100ms లో, మిగతావి ready అయ్యాక.

PPR (Partial Prerendering) – భవిష్యత్తు 🌟

```
// next.config.ts
experimental: { ppr: "incremental" }

// app/products/[id]/page.tsx
export const experimental_ppr = true;

export default async function Page({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  return (
    <>
      <ProductDetails id={id} />                       {/* ✅ STATIC – build time లో prerender
*/}
      <Suspense fallback=<CartSkeleton />>
        <Cart />                                       {/* ⚡ DYNAMIC (cookies వాడుతుంది) – hole
*/}
      </Suspense>
      <Suspense fallback=<RecsSkeleton />>
        <Recommendations userId={...} />             {/* ⚡ DYNAMIC – personalized */}
      </Suspense>
    </>
  );
}
```

ఎలా: build time లో Next.js static shell ని generate చేసి (dynamic భాగాల చోట fallbacks తో), request వచ్చినప్పుడు **CDN నుండి shell వెంటనే** పంపి, dynamic holes ని stream చేస్తుంది.

ఇది "static vs dynamic" అనే ఎంపికను తొలగిస్తుంది – ఒకే page లో రెండూ. E-commerce PDP కి perfect (product info static, cart/recommendations personalized).

ఏది ఎప్పుడు – decision table

Page రకం	Strategy
Marketing, docs, blog	Static (+ ISR CMS content సీ)
Product listing/detail	ISR (+ PPR personalization సీ)
User dashboard	Dynamic (+ streaming)
Search results	Dynamic (searchParams)
Admin panel	Dynamic లేదా client-side
Real-time (chat, live scores)	Client fetch/WebSocket + static shell

next build output ని చదవడం (కీలకమైన నైపుణ్యం)

Route (app)	Size	First Load JS
o /	5.2 kB	89 kB
• /blog/[slug]	2.1 kB	86 kB
f /dashboard	8.4 kB	95 kB
• /products/[id]	3.1 kB	88 kB

o (Static)	prerendered as static content
• (SSG)	prerendered as static HTML (generateStaticParams)
f (Dynamic)	server-rendered on demand
• (PPR)	partially prerendered

ప్రతి deploy కి ఈ output చూడండి. Static అనుకున్న route `f` గా ఉంటే – ఎక్కడో dynamic API వాడారు. ఇది performance regression కి ప్రధాన సంకేతం.

Gotchas

- అనుకోకుండా **dynamic** – ఒక చిన్న `cookies()` call మొత్తం page ని dynamic చేస్తుంది (CDN caching పోతుంది).
- `export const dynamic = "force-dynamic"` ని అలవాటుగా → static optimization మొత్తం నష్టం.
- **ISR** లో `revalidate` ఎక్కువ పెట్టడం → stale content; తక్కువ పెట్టడం → server load.
- **Streaming SEO** ని పాడు చేస్తుందని భయం – Google streaming HTML ని సరిగ్గా index చేస్తుంది (కానీ critical content ని shell లో ఉంచడం మేలు).
- **Build output చూడకపోవడం** – ఏ pages static/dynamic అని తెలియకపోవడం.
- **Static page** లో `Date.now()` / `Math.random()` → build time value freeze అవుతుంది.

Key Points

- Next.js **default static**; dynamic APIs (`cookies` , `headers` , `searchParams`) route ని dynamic చేస్తాయి.
- **ISR = stale-while-revalidate** – ఏ user వేచి ఉండడు.

- **Streaming** = TTFB మెరుగు + progressive rendering.
- **PPR** = static shell + dynamic holes (ఒకే page).
- **next build output** = ప్రతి route యొక్క strategy ని నిర్ధారించే సాధనం.

Interview దృష్టి

- "SSG vs SSR vs ISR ఎప్పుడు?" → freshness vs latency vs cost trade-off + ఉదాహరణలు.
- "PPR ఏం పరిష్కరిస్తుంది?" → per-page static/dynamic binary choice ని తీలగిస్తుంది.
- "Page అనుకోకుండా dynamic అయింది – ఎలా కనిపెడతావు?" → build output symbols + dynamic API audit.

8. Server Components vs Client Components

SERVER VS CLIENT COMPONENTS · ఎక్కడ గీత గీయాలి

SERVER vs CLIENT COMPONENTS

Server Component (default)

- ✓ DB / filesystem కి నేరుగా access
- ✓ Bundle కి 0 bytes – JS పంపము
- ✗ useState, useEffect లేవు
- ✗ onClick లాంటి handlers లేవు

Client Component ("use client")

- ✓ Hooks, state, effects
- ✓ Browser APIs, event handlers
- ✗ Bundle కి బరువు చేరుతుంది
- ✗ Secrets ఇక్కడ ఎప్పుడూ పెట్టకూడదు

బంగారు నియమం – "use client" ని అకుల దగ్గర పెట్టండి

Layout మద "use client" పెడితే – దాని కిందున్న అంతా client అవుతుంది.
బదులుగా: interactive చిన్న భాగాన్ని మాత్రమే client చేసి, మిగతాది server గా ఉంచడం.
Server component ని client component కి **children** గా పంపొచ్చు – import చేయలేం.

వివరణ

React_Telugu.md Topic 49 లో RSC concept ఉంది; ఇక్కడ **Next.js-specific composition patterns**.

App Router లో ప్రతి component default గా Server Component.

```
// app/products/page.tsx – Server Component (default, 'use client' లేదు)
import { db } from "@lib/db";

export default async function ProductsPage() {
  const products = await db.product.findMany(); // ✓ DB నేరుగా! bundle లోకి వెళ్ళదు
  return (
    <ul>
      {products.map((p) => (
        <li key={p.id}>
          {p.name}
          <AddToCartButton productId={p.id} /> /* client component */
        </li>
      ))}
    </ul>
  )
}
```

```
);  
}
```

```
// components/AddToCartButton.tsx  
"use client"; // 🔑 boundary  
import { useState } from "react";  
export function AddToCartButton({ productId }: { productId: string }) {  
  const [adding, setAdding] = useState(false);  
  return <button onClick={() => addToCart(productId)}>Add</button>;  
}
```

Real-life Scenario

Server Component = restaurant kitchen; Client Component = table మీద ఉన్న salt shaker.

వంట మొత్తం వంటగదిలో (server) జరుగుతుంది – recipes, ingredients, gas stove (DB, secrets, heavy libraries) ఏదీ మీ table కి రాదు. మీ table కి వచ్చేది **తయారైన dish** (rendered output).

కానీ **ఉప్పు shaker మీ చేతిలో ఉండాలి** – ప్రతిసారి వంటగదికి కబురు పంపలేరు (interactivity). అదే client component. **వీలైనన్ని తక్కువ వస్తువులు table మీద ఉంచడమే లక్ష్యం** ('use client' ని leaf కి దగ్గరగా).

Server vs Client – పూర్తి పోలిక

	Server Component	Client Component
ఎక్కడ run	server మాత్రమే	server (SSR/prerender) + browser
Client bundle	0 KB ✓	✓ ఉంటుంది
async/await	✓ నేరుగా	✗ (use() + Suspense)
DB/filesystem/secrets	✓	✗
useState , useEffect , useReducer	✗	✓
Event handlers (onClick)	✗	✓
Browser APIs (window , localStorage)	✗	✓
Context provide	✗	✓
Class components	✓ (కానీ hooks లేవు)	✓

🔑 Composition నియమాలు (ఇక్కడే అందరూ తప్పు చేస్తారు)

```
// ✗ నియమం 1: Client component లోపల server component ని IMPORT చేయలేం  
"use client";
```

```
import ServerComponent from "../ServerComponent"; // ❌ ఇది client component ను
మారిపోతుంది!

// ✅ కానీ children/props ను PASS చేయొచ్చు!
// app/page.tsx (server)
<ClientWrapper>
  <ServerComponent /> /* ✅ ఇది server లోనే render అయ్యి, output pass అవుతుంది */
</ClientWrapper>

// components/ClientWrapper.tsx
"use client";
export function ClientWrapper({ children }: { children: React.ReactNode }) {
  const [open, setOpen] = useState(false);
  return <div>{open && children}</div>; // ✅ children ఇప్పటికే rendered
}
```

'use client' = boundary, కాదు label. ఒక file లో 'use client' రాస్తే – ఆ file మరియు అది import చేసే అన్ని modules client bundle లోకి వెళ్తాయి. అందుకే దాన్ని tree లో వీలైనంత కిందికి (leaf) పెట్టాలి.

```
// ❌ చెడ్డది – మొత్తం page client bundle లోకి
"use client";
export default function ProductPage() {
  const [qty, setQty] = useState(1);
  return (
    <div>
      <ProductImages /> /* client లోకి వెళ్ళాల్సిన అవసరం లేదు */
      <ProductDescription /> /* ఇదీ */
      <Reviews /> /* ఇదీ (heavy!) */
      <input value={qty} onChange={(e) => setQty(+e.target.value)} /> /* ఇది మాత్రమే
కావాలి */
    </div>
  );
}

// ✅ మంచిది – interactive భాగం మాత్రమే client
export default async function ProductPage() { // server
  const product = await getProduct(id);
  return (
    <div>
      <ProductImages images={product.images} /> /* server */
      <ProductDescription html={product.description} /> /* server */
      <Reviews productId={product.id} /> /* server */
      <QuantitySelector /> /* 'use client' - leaf
మాత్రమే ✅ */
    </div>
  );
}
```

Props serialization నియమం

Server → Client కి props **serializable** ఉండాలి (RSC payload లో JSON-లాంటి format లో వెళ్తాయి):

```
// ✅ పంపగలిగేవి
string, number, boolean, null, undefined, Date, Map, Set, BigInt,
arrays, plain objects, Promises, React elements (JSX), Server Actions

// ❌ పంపలేనివి
functions (Server Actions తప్ప), class instances, Symbols,
Date methods, DB connections, Prisma model instances (methods ఉంటే)

<ClientComp onSave={() => {}} /> // ❌ Error (Server Action అయితే ✅)
<ClientComp user={prismaUserInstance} /> // ⚠️ methods ఉంటే fail
<ClientComp data={JSON.parse(JSON.stringify(obj))} /> // workaround (ugly)
```

Provider pattern (context ని Server Component tree లో వాడటం)

```
// app/providers.tsx
"use client";
export function Providers({ children }: { children: React.ReactNode }) {
  const [client] = useState(() => new QueryClient());
  return (
    <QueryClientProvider client={client}>
      <ThemeProvider>{children}</ThemeProvider>
    </QueryClientProvider>
  );
}

// app/layout.tsx (server) – ✅ children server components గానే ఉంటాయి
export default function RootLayout({ children }: { children: React.ReactNode }) {
  return <html><body><Providers>{children}</Providers></body></html>;
}
```

ఇది కీలకమైన pattern: Provider client component అయినా, children గా pass అయిన server components **server లోనే render అవుతాయి**. Root layout ని 'use client' చేయాల్సిన అవసరం లేదు.

Data ని client కి పంపడం vs server లో వాడటం

```
// ❌ Anti-pattern – server లో fetch చేసి మొత్తం client కి పంపడం
export default async function Page() {
  const products = await getProducts(); // 500 products
  return <ClientProductList products={products} />; // ⚠️ మొత్తం RSC payload లో
  వెళ్తుంది!
}

// ✅ మేలు – server లో render, interactive భాగం మాత్రమే client
export default async function Page() {
  const products = await getProducts();
  return (
    <ul>
      {products.map((p) => (
        <li key={p.id}>
          <ProductInfo product={p} /> // server – HTML మాత్రమే */
          <AddToCart productId={p.id} /> // client – తక్కువ props */
        )
      )}
    </ul>
  );
}
```

```

    </li>
  )}
</ul>
);
}

```

server-only & client-only guards

```

import "server-only";    // ఈ module ని client import చేస్తే BUILD FAIL ✓
import "client-only";    // ఈ module ని server import చేస్తే BUILD FAIL ✓

```

Gotchas

- **'use client'** ని **root layout/page** లో → మొత్తం app client bundle (RSC ప్రయోజనం సున్నా).
- **Server component** లో **useState / onClick** → error.
- **Client component** లో **async** → support లేదు (React 19 **use()** + **Suspense** వాడాలి).
- **Non-serializable props** → runtime error.
- **Server component** లో **secret** ని **client** కి **prop** గా పంపడం → **RSC payload** లో **plain text** గా **leak!** 🚨
- **Client component** ని **import** చేస్తే అది **"client అవుతుంది"** అని **అనుకోవడం** – server component client component ని **import** చేయొచ్చు (అదే normal pattern).
- **'use client'** ప్రతి **file** లో **పెట్టడం** – ఒక client file import చేసే వాటికి అవసరం లేదు (అవి automatic client).

Key Points

- Default = Server Component; **'use client'** = **boundary**, leaf కి దగ్గరగా.
- Client → server **import** ❌, కానీ **children** గా **pass** ✅.
- Props **serializable** ఉండాలి; secrets ఎప్పుడూ props గా వద్దు.
- Providers = client wrapper + server children (root layout server గానే ఉంటుంది).
- **server-only** package = accidental leaks build time లో అవుతుంది.

Interview దృష్టి

- **"'use client' ఏం చేస్తుంది?"** → module boundary; ఆ subtree client bundle లోకి.
- **"Client component లో server component ఎలా వాడతావు?"** → children/props composition – favourite question.
- **"RSC వల్ల నిజమైన లాభం?"** → bundle size ↓, data fetching co-location, secrets safety, waterfall తగ్గింపు.

9. Streaming & Suspense

STREAMING & SUSPENSE • భాగాలుగా పంపడం

STREAMING – Suspense boundary దగ్గర విరగడం

Streaming లేకుండా

అన్ని data వచ్చేదాకా ఏమీ కనిపించదు
నెమ్మది API = ఖాళీ screen
TTFB చెడ్డది

Streaming తో

Shell వెంటనే వెళ్తుంది
నెమ్మది భాగం స్థానంలో fallback
Data వచ్చాక ఆ భాగం మాత్రం చేరుతుంది

HTML shell → <Suspense fallback> → ...తర్వాత... → అసలు content

ఎక్కడ boundary పెట్టాలి

నెమ్మది data fetch చేసే component చుట్టూ – మొత్తం page చుట్టూ కాదు.
Page చుట్టూ పెడితే streaming యొక్క ప్రయోజనం పోతుంది (మళ్ళీ అంతా ఒకేసారి).
Fallback ని skeleton గా చేయడం – layout shift రాకుండా.

వివరణ

Streaming = HTML ని ముక్కలుగా పంపడం, మొత్తం ready అయ్యేవరకు వేచి ఉండకుండా.

Streaming లేకుండా:

Request → [server: అన్ని data fetch – 2000ms] → HTML మొత్తం → user చూస్తాడు
⌚ TTFB = 2000ms (user ఖాళీ screen చూస్తాడు)

Streaming తో:

Request → shell (100ms) → user చూస్తాడు ✓
→ Products ready (800ms) → stream
→ Reviews ready (2000ms) → stream
⌚ TTFB = 100ms

Granular Suspense – loading.tsx కంటే మేలు

```
// ✗ loading.tsx – నెమ్మది ఒక్క component మొత్తం page ని ఆపుతుంది
// app/dashboard/loading.tsx → మొత్తం page skeleton

// ✓ Granular – వేగంగా వచ్చేవి వెంటనే కనిపిస్తాయి
export default async function Dashboard() {
  const user = await getUser(); // 50ms – వేగం, await చేయొచ్చు

  return (
    <>
      <Header user={user} /> // వెంటనే ✓ */
      <Suspense fallback={<StatsSkeleton />}>
        <Stats /> // 300ms */
      </Suspense>

      <Suspense fallback={<ChartSkeleton />}>
        <RevenueChart /> // 2s – నెమ్మది */
      </Suspense>

      <Suspense fallback={<TableSkeleton />}>
        <RecentOrders /> // 800ms */
      </Suspense>
    </>
  )
}
```

```

    </Suspense>
  </>
);
}

```

కీలకం: `await` ని **page component** లో చేస్తే మొత్తం page ఆగుతుంది. **Child component** లోకి తరలిస్తే అది Suspense boundary లో స్వతంత్రంగా stream అవుతుంది.

```

// ❌ page ఆగుతుంది
export default async function Page() {
  const slow = await getSlowData(); // మొత్తం page 2s ఆగుతుంది
  return <div>{slow}</div>;
}

// ✅ shell వెంటనే
export default function Page() {
  return <Suspense fallback=<Skeleton />><SlowSection /></Suspense>;
}

async function SlowSection() {
  const slow = await getSlowData(); // ఇది మాత్రమే ఆగుతుంది
  return <div>{slow}</div>;
}

```

Skeleton design – UX సూక్ష్మతలు

```

// ❌ generic spinner – layout shift (CLS), user కి ఏమీ తెలియదు
<div className="spinner" />

// ✅ content shape ని అనుకరించే skeleton – CLS లేదు
function TableSkeleton() {
  return (
    <div className="space-y-2">
      {Array.from({ length: 5 }).map((_, i) => (
        <div key={i} className="h-12 animate-pulse rounded bg-gray-200" />
      ))}
    </div>
  );
}

```

Skeleton ఎత్తు నిజమైన content ఎత్తుకి సరిపోవాలి – లేకపోతే content వచ్చినప్పుడు page జరుగుతుంది (CLS penalty).

Streaming + SEO

ప్రశ్న: streaming HTML ని Google index చేస్తుందా? **సమాధానం:** అవును – Googlebot streamed HTML ని పూర్తిగా receive చేసి render చేస్తుంది. కానీ:

- **Critical SEO content** (title, h1, description, main content) ని **shell** లో ఉంచడం మేలు (Suspense బయట).
- Metadata (`generateMetadata`) ఎప్పుడూ shell లోనే.

- Reviews, recommendations వంటివి stream చేయొచ్చు.

after() – response తర్వాత పని (Next 15)

```
import { after } from "next/server";

export default async function Page() {
  const data = await getData();

  after(() => {
    // ✅ response పంపిన తర్వాత run అవుతుంది – user వేచి ఉండడు
    logAnalytics({ page: "/products", timestamp: Date.now() });
    updateLastSeen(userId);
  });

  return <View data={data} />;
}
```

ఇది serverless లో కీలకం – గతంలో fire-and-forget promises serverless function terminate అయ్యాక చనిపోయేవి.

Streaming ఎప్పుడు పని చేయదు

- `export const dynamic = "force-static"` – static routes stream అవ్వవు (అవసరం లేదు).
- కొన్ని **CDN/proxy** buffering చేస్తాయి (nginx `proxy_buffering off` అవసరం కావచ్చు).
- **Middleware లో response ని modify చేయడం** streaming ని break చేయొచ్చు.

Gotchas

- **Page component లో await** → shell ఆగుతుంది (child లోకి తరలించాలి).
- **Suspense boundary చాలా పెద్దది** → granularity ప్రయోజనం లేదు.
- **Skeleton size mismatch** → CLS.
- **అన్నిటినీ Suspense లో wrap చేయడం** → చాలా skeletons flash → "jumpy" UI.
- **Streaming లో error handling లేకపోవడం** → stream మధ్యలో fail అయితే partial page (error boundary తప్పనిసరి).
- `loading.tsx` + **granular Suspense రెండూ ఒకేసారి** → double skeletons.

Key Points

- Streaming = TTFB ↓ + progressive rendering.
- `await` ని **child component లోకి తరలించడం** = streaming ని enable చేసే ట్రిక్.
- Granular `<Suspense>` > page-wide `loading.tsx`.
- Skeleton shape/size = CLS నివారణ.
- `after()` = response తర్వాత side effects (analytics, logging).

Interview దృష్టి

- "Streaming ఎలా enable చేస్తావు?" → Suspense boundaries + async child components;
loading.tsx = automatic.
- "Streaming SEO ని పాడుచేస్తుందా?" → లేదు, కానీ critical content shell లో ఉంచాలి.

10. Hydration & Client Boundaries

వివరణ

```
Server: HTML + RSC payload generate
↓
Browser: HTML paint (user content చూస్తాడు, కానీ clicks పని చేయవు)
↓
JS bundle download + parse
↓
Hydration: React client components కి listeners attach
↓
Interactive ✓
```

RSC payload = server components యొక్క rendered output + client component references – ఇది HTML లో `<script>self.__next_f.push(...)</script>` గా embed అవుతుంది (view-source లో చూడొచ్చు).

Hydration mismatch – అత్యంత సాధారణ Next.js error

```
Error: Hydration failed because the server rendered HTML didn't match the client.
```

కారణాలు:

```
// ✗ 1. Time/random – server & client వేర్వేరు values
<p>{new Date().toLocaleString()}</p>
<p>{Math.random()}</p>

// ✗ 2. Browser-only APIs
<p>{window.innerWidth}</p>
<p>{localStorage.getItem("theme")}</p>

// ✗ 3. Invalid HTML nesting (browser auto-corrects → mismatch)
<p><div>hi</div></p>
<Link href="/x"><a>text</a></Link>    // <a> లోపల <a>

// ✗ 4. Browser extensions (Grammarly, dark-mode extensions) DOM ని మార్చుస్తాయి
// ✗ 5. locale/timezone-dependent formatting
```

పరిష్కారాలు:

```
// ✅ 1. Mount తర్వాత మాత్రమే render
"use client";
function ClientOnly({ children }: { children: React.ReactNode }) {
  const [mounted, setMounted] = useState(false);
  useEffect(() => setMounted(true), []);
  if (!mounted) return null;
  return <{children}></>;
}

// ✅ 2. suppressHydrationWarning (జస్ text node కి మాత్రమే)
<time suppressHydrationWarning={new Date().toISOString()}></time>

// ✅ 3. Dynamic import with ssr: false
const Chart = dynamic(() => import("./Chart"), { ssr: false, loading: () => <Skeleton /> });
// ⚠️ Next 15: server component లో ssr:false పని చేయదు – client component లోనే

// ✅ 4. Theme flash (FOUC) – blocking script
// app/layout.tsx <head> లో
<script dangerouslySetInnerHTML={{ __html: `
  (function(){try{var t=localStorage.getItem('theme');
    var d=t==='dark'||(t&&matchMedia('(prefers-color-scheme:dark)').matches);
    document.documentElement.classList.toggle('dark',d);}catch(e){}})();
`}} />
```

Client bundle ని కొలవడం & తగ్గించడం

```
npm i -D @next/bundle-analyzer
ANALYZE=true npm run build
```

```
// next.config.ts
import bundleAnalyzer from "@next/bundle-analyzer";
const withAnalyzer = bundleAnalyzer({ enabled: process.env.ANALYZE === "true" });
export default withAnalyzer(config);
```

```
// Heavy client libraries ని lazy load
import dynamic from "next/dynamic";
const RichEditor = dynamic(() => import("@/components/RichEditor"), {
  loading: () => <EditorSkeleton />,
  ssr: false, // browser-only library
});
const Map = dynamic(() => import("@/components/Map"), { ssr: false });
```

First Load JS ని అర్థం చేసుకోవడం

Route (app)	Size	First Load JS
r o /	5.2 kB	89 kB
+ First Load JS shared by all		85 kB
└─ chunks/framework	45 kB	← React + Next runtime

└ chunks/main	32 kB
└ other shared chunks	8 kB

లక్ష్యం: First Load JS < **100 kB** (గొప్పది), < 150 kB (ఆమోదయోగ్యం), > 200 kB (సమస్య).
Shared chunk పెరిగితే – root layout లో ఏదో heavy client component ఉంది.

Gotchas

- **Hydration mismatch ని ignore చేయడం** – React మొత్తం subtree ని client-render చేస్తుంది (నెమ్మది + SEO నష్టం).
- `suppressHydrationWarning` ని అతిగా వాడటం – నిజమైన bugs దాచేస్తుంది.
- `ssr: false` ని **server component** లో (Next 15 లో error).
- **Root layout లో heavy client providers** → అన్ని pages కీ bundle పెరుగుదల.
- **Date formatting ని server/client రెండింటిలో** → timezone mismatch (server UTC, client IST).

Key Points

- Hydration = HTML + JS → interactive; **mismatch = server/client output తేడా**.
- కారణాలు: time/random, browser APIs, invalid HTML, extensions, locale.
- పరిష్కారాలు: mount gate, `suppressHydrationWarning`, `dynamic({ssr:false})`, blocking theme script.
- **First Load JS < 100 kB** లక్ష్యం; bundle analyzer + `next/dynamic`.

Interview దృష్టి

- "Hydration mismatch ఎందుకు, ఎలా fix?" → కారణాలు 5 + పరిష్కారాలు 4.
- "Client bundle ఎలా తగ్గిస్తావు?" → RSC boundary కిందికి, dynamic imports, analyzer, provider audit.

Part 3 – Data Fetching & Caching

11. Data Fetching in App Router

వివరణ

App Router లో data fetching **component లోనే** జరుగుతుంది – `getServerSideProps` వంటి ప్రత్యేక functions అవసరం లేదు.

```
// ✅ ఇదే ప్రామాణిక pattern
export default async function Page() {
  const data = await fetch("https://api.example.com/data").then((r) => r.json());
  return <View data={data} />;
}

// ✅ ORM/DB నేరుగా (API layer అవసరం లేదు!)
import { db } from "@lib/db";
export default async function Page() {
  const products = await db.product.findMany({ where: { active: true } });
  return <List products={products} />;
}
```

Real-life Scenario

Pages Router = ఒక్కడే గుమాస్తా. పై అంతస్తులో ఎవరికీ ఏం కావాలో అందరూ ముందుగా అతనికి చెప్పాలి (`getServerSideProps`), అతను అన్నీ ఒకేసారి తెచ్చి పంచుతాడు. ఒకరికి ఆలస్యమైతే అందరూ వేచి ఉండాలి.

App Router = ప్రతి గదిలో ఒక intercom. ఎవరికి కావలసినది వాళ్ళే నేరుగా అడుగుతారు (component-level fetch), ఎవరిది ముందు వస్తే వాళ్ళకి ముందు (streaming). **Request memoization = ఒకే వస్తువు ఇద్దరు అడిగితే ఒకసారే తేవడం.**

Waterfall vs Parallel (అత్యంత ముఖ్యమైన performance concept)

```
// ❌ Sequential waterfall – 300 + 400 + 200 = 900ms
export default async function Page() {
  const user = await getUser(); // 300ms
  const posts = await getPosts(); // 400ms – పైనది అయ్యాకే మొదలు
  const stats = await getStats(); // 200ms
  return <View {...{ user, posts, stats }} />;
}

// ✅ Parallel – max(300, 400, 200) = 400ms ✂
export default async function Page() {
  const [user, posts, stats] = await Promise.all([getUser(), getPosts(), getStats()]);
  return <View {...{ user, posts, stats }} />;
}
```

```

}

// ⚠️ నిజమైన dependency ఉంటే waterfall తప్పదు
const user = await getUser();
const posts = await getPostsByUser(user.id); // user.id అవసరం - తప్పదు

```

Component-level waterfall (దాగిన రకం)

```

// ❌ Nested async components - parent ready అయ్యాకే child మొదలవుతుంది
async function Layout() {
  const settings = await getSettings(); // 200ms
  return <div><Content /></div>; // Content 200ms తర్వాత మొదలు
}
async function Content() {
  const data = await getData(); // +300ms → మొత్తం 500ms
}

// ✅ పరిష్కారం 1: Suspense - parallel streaming
<Suspense fallback=<S1 />><Settings /></Suspense>
<Suspense fallback=<S2 />><Content /></Suspense>

// ✅ పరిష్కారం 2: Preload pattern ⭐

```

Preload pattern (senior-level technique)

```

// lib/data.ts
import { cache } from "react";
import "server-only";

export const getUser = cache(async (id: string) => {
  return db.user.findUnique({ where: { id } });
});

// 🔑 preload - await చేయకుండా fetch మొదలుపెట్టడం
export const preloadUser = (id: string) => {
  void getUser(id); // deduped కాబట్టి తర్వాత await చేస్తే cache hit
};

```

```

// app/users/[id]/page.tsx
export default async function Page({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;

  preloadUser(id); // ✗ ఇక్కడే fetch మొదలు (await లేదు)
  const settings = await getSettings(); // ఇది జరుగుతున్నప్పుడే user fetch అవుతోంది

  return (
    <>
      <Settings data={settings} />
      <Suspense fallback=<Skeleton />>
        <UserProfile id={id} /> /* getUser(id) - ఇప్పటికే cache లో ✅ */
      </Suspense>
    </>
  );
}

```

```
);  
}
```

React.cache() – request-level memoization (ORM calls కి)

```
import { cache } from "react";  
  
export const getUser = cache(async (id: string) => {  
  console.log("DB hit!"); // ఒకే request లో ఒక్కసారి print అవుతుంది ✓  
  return db.user.findUnique({ where: { id } });  
});  
  
// ఒకే render pass లో 5 components getUser("1") call చేసినా → 1 DB query
```

fetch() కి ఇది automatic (Request Memoization – Topic 12). **ORM/DB calls కి**

React.cache() manually wrap చేయాలి. ఇది props drilling ని తొలగిస్తుంది – ప్రతి component తనకీ కావలసిన data ని స్వయంగా అడగొచ్చు.

Client-side fetching ఎప్పుడు అవసరం

```
// Server components సరిపోని సందర్భాలు:  
// – Real-time (polling, WebSocket)  
// – User interaction ఆధారంగా (search-as-you-type, infinite scroll)  
// – Optimistic updates + complex cache invalidation  
// – Browser-only data  
  
"use client";  
import useSWR from "swr";  
const { data, error, isLoading } = useSWR("/api/notifications", fetcher, {  
  refreshInterval: 30_000,  
});  
  
// లేదా TanStack Query – server లో prefetch + client hydrate
```

```
// ✓ Hybrid pattern – server prefetch + client interactivity  
import { dehydrate, HydrationBoundary, QueryClient } from "@tanstack/react-query";  
  
export default async function Page() {  
  const qc = new QueryClient();  
  await qc.prefetchQuery({ queryKey: ["posts"], queryFn: getPosts }); // server లో  
  
  return (  
    <HydrationBoundary state={dehydrate(qc)}>  
      <PostsClient /> /* client – instant data, తర్వాత refetch/pagination */  
    </HydrationBoundary>  
  );  
}
```

Sequential vs Parallel – decision guide

Data B కి Data A అవసరమా?

- └ అవును → sequential తప్పదు (కానీ Suspense తో UI ని unblock చేయొచ్చు)
- └ కాదు → Promise.all() లేదా వేర్వేరు Suspense boundaries ✓

Gotchas

- **Page component లో అన్ని await లు వరుసగా** → waterfall.
- **ORM calls ని React.cache() లేకుండా అనేక components లో** → duplicate DB queries.
- **Client component లో async** → support లేదు.
- **fetch ని client component లో useEffect తో** (server లో చేయగలిగినప్పుడు) → waterfall + bundle + no SEO.
- **Promise.all లో ఒకటి fail అయితే మొత్తం fail** → Promise.allSettled కావాలేమో ఆలోచించాలి.
- **Server component లో fetch error handle చేయకపోవడం** → error.tsx కి వెళ్తుంది (కొన్నిసార్లు partial UI మేలు).

Key Points

- Data fetching **component లోనే** (async server components).
- **Parallel > sequential** – Promise.all లేదా వేర్వేరు Suspense boundaries.
- **React.cache()** = ORM/DB calls కి request-level dedup.
- **Preload pattern** = waterfall ని ముందుగానే విరగొట్టడం.
- Client fetching = real-time/interaction/optimistic కి మాత్రమే.

Interview దృష్టి

- "Waterfall ఎలా కనిపెట్టి fix చేస్తావు?" → sequential awaits, nested async components → Promise.all, Suspense, preload.
- "React.cache vs fetch memoization?" → fetch automatic; ORM కి manual wrap.

12. Caching Deep – 4 Layers ★

NEXT.JS · నాలుగు CACHING పొరలు

నాలుగు CACHE పొరలు – ఇదే Next.js లో అతి కష్టమైన భాగం

1 · Request Memoization

ఒకే render లో అదే fetch → ఒక్కసారి
server · ఒక request వరకు

2 · Data Cache

fetch ఫలితం – deploys దాటి కూడా
server · శాశ్వతం

3 · Full Route Cache

render చేసిన HTML + RSC payload
server · build time

4 · Router Cache

client లో navigation కోసం
browser · session

ఎందుకు ఇది గందరగోళం

"నా data ఎందుకు update కావడం లేదు?" – నాలుగు పొరల్లో ఏదో ఒకటి పట్టుకుంది.
Debug చేసే క్రమం: ముందు router cache (client), తర్వాత full route, తర్వాత data cache.
Next 15 లో defaults మారాయి – fetch ఇప్పుడు default గా cache కాదు (ఇది పెద్ద మార్పు).

వివరణ

Next.js లో అత్యంత గందరగోళమైన విషయం ఇదే. App Router లో 4 caching layers ఉన్నాయి, ఒక్కొక్కటి వేర్వేరు చోట, వేర్వేరు కాలం, వేర్వేరు invalidation.

#	Layer	ఏమి cache	ఎక్కడ	కాలం
1	Request Memoization	ఒకే render లో duplicate fetch	Server	ఒక request
2	Data Cache	fetch results	Server (persistent)	Deploy దాటి కూడా
3	Full Route Cache	Rendered HTML + RSC payload	Server/CDN	Build/revalidation వరకు
4	Router Cache	RSC payload (client-side)	Browser memory	Session (30s/5min)

Real-life Scenario

4 layers = ఇంట్లో నీళ్ళ వ్యవస్థ.

1. **Request memoization** = ఒక వ్యక్తి ఒకే గ్లాసుని పదిసార్లు అడిగితే ఒకసారే ఇవ్వడం (ఆ ఒక్క సందర్భానికీ).
2. **Data Cache** = ఇంట్లో ఉన్న నీళ్ళ ట్యాంక్ – రోజంతా ఉంటుంది, అందరికీ.
3. **Full Route Cache** = ముందే నింపి పెట్టిన బాటిళ్లు – ఎవరైనా వచ్చి తీసుకోవచ్చు, తయారే సమయం లేదు.
4. **Router Cache** = మీ చేతిలో ఉన్న బాటిల్ – తిరిగి ట్యాంక్ దగ్గరికి వెళ్ళనవసరం లేదు.

సమస్య: ట్యాంక్ లో నీళ్ళు మార్చినా (DB update), మీ చేతిలో బాటిల్ పాతదే (router cache) – అందుకే అన్ని layers ని invalidate చేయాలి.

Layer 1 – Request Memoization

```
// ఒకే render pass లో ఒకే URL కి అనేక fetch → ఒకే network call ✓  
async function Header() { const u = await fetch("/api/user").then(r => r.json()); }  
async function Sidebar() { const u = await fetch("/api/user").then(r => r.json()); }  
async function Profile() { const u = await fetch("/api/user").then(r => r.json()); }  
// → 1 network request మాత్రమే (React automatic)
```

- **Scope:** ఒక server request (render pass) మాత్రమే.
- **వర్తిస్తుంది:** `fetch()` (GET) కి automatic. **ORM/DB కి కాదు** → `React.cache()` వాడాలి.
- **Opt-out:** `AbortSignal` పంపడం, లేదా వేరే URL/options.
- **లాభం:** props drilling అవసరం లేదు – ప్రతి component తనకీ కావలసినది స్వయంగా fetch చేయొచ్చు.

Layer 2 – Data Cache (fetch results)

```
// Next 15 defaults ⚠  
await fetch(url); // ✗ cache కాదు (Next 15)  
// ✓ cache అయ్యేది (Next 14)  
  
await fetch(url, { cache: "force-cache" }); // ✓ cache  
await fetch(url, { next: { revalidate: 3600 } }); // ✓ 1 గంట (ISR)  
await fetch(url, { next: { tags: ["products"] } }); // ✓ tag-based invalidation  
await fetch(url, { cache: "no-store" }); // ✗ ఎప్పుడూ fresh
```

ఇది deployments దాటి కూడా నిలుస్తుంది (Vercel లో persistent storage; self-host లో

`.next/cache`).

```
// Route-level default సెట్ చేయడం  
export const fetchCache = "default-cache"; // ఈ route లో అన్ని fetch cache  
export const revalidate = 3600; // ఈ route కి ISR
```

Layer 3 – Full Route Cache

Build time (లేదా మొదటి request) లో render అయిన **HTML + RSC payload** ని cache చేస్తుంది.

Static route → build లో render → cache → CDN → అన్ని users కి అదే
 Dynamic route → cache కాదు (ప్రతి request కి render)

Invalidate అయ్యేది: `revalidatePath` / `revalidateTag` , `revalidate` time దాటినప్పుడు, redeploy.

Layer 4 – Router Cache (client-side)

Browser memory లో RSC payloads – soft navigation ని instant చేస్తుంది.

Next 15 defaults:

Static routes → 5 నిమిషాలు (prefetched)
Dynamic routes → 0 సెకన్లు (default; `staleTimes` తో మార్చొచ్చు)

```
// next.config.ts – router cache ని tune చేయడం
experimental: {
  staleTimes: { dynamic: 30, static: 180 }, // సెకన్లు
}
```

Invalidate: `router.refresh()` , Server Action లో `revalidatePath/Tag` , hard navigation, `cookies.set()` .

⚠ ఇదే "నా data update అయింది కానీ UI లో పాతది కనిపిస్తోంది" bug కి #1 కారణం.

Server cache invalidate చేసినా **client router cache** పాతదే ఉండొచ్చు.

మొత్తం flow – ఒక request ఎలా ప్రయాణిస్తుంది

```
User clicks <Link>
↓
1. Router Cache లో ఉందా? → అవును → instant render < (server touch కాదు)
   ↓ లేదు
2. Full Route Cache లో ఉందా? → అవును → cached RSC payload
   ↓ లేదు (dynamic route)
3. Server render మొదలు
   └─ fetch() calls → Request Memoization (ఒకే render లో dedup)
     │               → Data Cache (persistent)
     └─ miss అయితే → నిజమైన network call
   ↓
4. Render పూర్తి → Full Route Cache లో store (static/ISR అయితే)
   ↓
5. Client కి stream → Router Cache లో store
```

Debugging caching

```
// next.config.ts - ఏ fetch cache అయిందో చూడటానికి
logging: { fetches: { fullUrl: true } }
```

```
✓ Compiled /products
GET /products 200 in 45ms
  | GET https://api.example.com/products 200 in 2ms (cache: HIT) ✓
  | GET https://api.example.com/reviews 200 in 340ms (cache: SKIP) ← ఎందుకు?
  | | Cache missed reason: (cache: no-store)
```

Gotchas (ఇక్కడే అందరూ ఇబ్బంది పడతారు)

- **Next 14 → 15 upgrade తర్వాత API calls పిరగడం** – `fetch` ఇక cache కాదు; explicit `force-cache / revalidate` కావాలి.
- `revalidatePath` చేసినా UI లో పాత data → **client router cache**; Server Action లో revalidate చేస్తే automatic clear అవుతుంది, కానీ route handler నుండి చేస్తే కాదు → `router.refresh()`.
- **ORM calls cache కావు** – Data Cache `fetch` కి మాత్రమే. DB కి: `unstable_cache` లేదా `use cache`.
- **Dynamic route లో revalidate** – పని చేయదు (route ఇప్పటికే dynamic).
- `cookies()` వాడితే మొత్తం route dynamic → Full Route Cache పోతుంది.
- **Development లో caching behaviour వేరు** – dev లో చాలా caching disabled; **production build తో test చేయాలి**.
- **Self-host లో Data Cache** – multiple instances ఉంటే cache share కాదు → shared cache handler (Redis) configure చేయాలి.

`unstable_cache` – ORM/DB calls ని cache చేయడం

```
import { unstable_cache } from "next/cache";

const getCachedProducts = unstable_cache(
  async (categoryId: string) => db.product.findMany({ where: { categoryId } }),
  ["products-by-category"], // cache key parts
  { revalidate: 3600, tags: ["products"] } // ✓ tag తో invalidate చేయొచ్చు
);

const products = await getCachedProducts("electronics");
revalidateTag("products"); // → ఈ cache clear
```

Key Points

- **4 layers:** Request Memoization (per-request) → Data Cache (persistent) → Full Route Cache (rendered) → Router Cache (browser).
- **Next 15:** `fetch` default uncached (Next 14 లో cached).
- `fetch` కి automatic; **ORM కి React.cache (dedup) + unstable_cache (persistent)**.
- Stale UI bug = సాధారణంగా **Router Cache**.

- **Production build తో caching test చేయాలి** (dev వేరు).

Interview దృష్టి

- "Next.js caching layers వివరించు" → 4 layers table + ఏది ఎప్పుడు invalidate – ఇది Next.js interviews లో అత్యంత సాధారణ deep question.
- "Update అయినా పాత data కనిపిస్తోంది – debug ఎలా?" → layer by layer: fetch options → revalidate → router cache → CDN.

13. Revalidation Strategies

వివరణ – cache ని ఎప్పుడు, ఎలా refresh చేయాలి

```
// 1 Time-based (ISR)
export const revalidate = 3600; // route level
await fetch(url, { next: { revalidate: 60 } }); // fetch level

// 2 On-demand – path
import { revalidatePath } from "next/cache";
revalidatePath("/products"); // ఒక page
revalidatePath("/products/[id]", "page"); // dynamic route type
revalidatePath("/dashboard", "layout"); // layout + అన్ని children
revalidatePath("/", "layout"); // ⚠ మొత్తం app

// 3 On-demand – tag (అత్యంత flexible ★)
import { revalidateTag } from "next/cache";
await fetch(url, { next: { tags: ["products", `product-${id}`] } });
revalidateTag("products"); // అన్ని products caches
revalidateTag(`product-${id}`); // ఒక product మాత్రమే
```

Real-life Scenario

Time-based revalidation = పాలు ప్రతి రోజు ఉదయం తేవడం. అవసరం ఉన్నా లేకపోయినా. **On-demand** = "పాలు అయిపోయాయి" అని ఫోన్ వస్తేనే తేవడం. ⚡ సమర్థవంతం, కానీ ఫోన్ చేసే వ్యవస్థ (webhook/action) కావాలి. **Tags** = "పాలు అయిపోయాయి" అంటే పాలు మాత్రమే, "వంటగది మొత్తం" కాదు. Fine-grained invalidation.

Tag-based revalidation – production pattern

```
// lib/api.ts – tag strategy
export async function getProducts(categoryId?: string) {
  return fetch(`${API}/products?category=${categoryId ?? ""}`, {
    next: {
      tags: ["products", categoryId ? `category-${categoryId}` : "all-products"],
      revalidate: 3600, // fallback (webhook fail అయితే)
    },
  }).then((r) => r.json());
}
```

```

}

export async function getProduct(id: string) {
  return fetch(`${API}/products/${id}`, {
    next: { tags: [`product-${id}`, "products"] },
  }).then((r) => r.json());
}

```

```

// app/api/webhooks/cms/route.ts - CMS update వచ్చినప్పుడు
import { revalidateTag } from "next/cache";
import { NextResponse } from "next/server";

export async function POST(request: Request) {
  // 🛡️ 1. Webhook signature verify (తప్పనిసరి!)
  const signature = request.headers.get("x-webhook-signature");
  const body = await request.text();
  if (!verifySignature(body, signature)) {
    return NextResponse.json({ error: "Invalid signature" }, { status: 401 });
  }

  const { type, id } = JSON.parse(body);

  // 2. నిర్దిష్టంగా invalidate
  switch (type) {
    case "product.updated": revalidateTag(`product-${id}`); break;
    case "product.created":
    case "product.deleted": revalidateTag("products"); break;
    case "category.updated": revalidateTag(`category-${id}`); break;
  }

  return NextResponse.json({ revalidated: true, now: Date.now() });
}

```

Server Action లో revalidation

```

"use server";
import { revalidatePath, revalidateTag } from "next/cache";

export async function updateProduct(id: string, data: FormData) {
  await db.product.update({ where: { id }, data: { name: data.get("name") as string } });

  revalidateTag(`product-${id}`); // ✅ server cache
  revalidatePath("/products");    // ✅ list page
  // 🗝️ Server Action లో revalidate చేస్తే **client router cache కూడా** automatic clear
  // ✅
}

```

ఇది కీలకమైన తేడా: Server Action లో `revalidatePath/Tag` → server cache + client router cache రెండూ clear. Route handler లో చేస్తే **server మాత్రమే** → client లో `router.refresh()` అవసరం.

revalidate విలువ ఎలా ఎంచుకోవాలి

Content రకం	revalidate	కారణం
Blog/docs	3600 - 86400 (+ webhook)	అరుదుగా మారుతుంది
Product catalog	300 - 3600 (+ on-demand)	price/stock మార్పిడి
Stock/inventory	10 - 60 లేదా dynamic	తరచూ మారుతుంది
News/feed	30 - 60	తాజాదనం ముఖ్యం
User dashboard	dynamic (cache లేదు)	personalized
Stock prices/live	dynamic + client polling	real-time

ఉత్తమ వ్యూహం: long revalidate + on-demand webhook. Webhook పని చేస్తే instant freshness; fail అయితే time-based safety net.

Draft mode (CMS preview)

```
// app/api/draft/route.ts
import { draftMode } from "next/headers";
import { redirect } from "next/navigation";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  if (searchParams.get("secret") !== process.env.DRAFT_SECRET) {
    return new Response("Invalid token", { status: 401 });
  }
  (await draftMode()).enable(); // 🔑 cookie set → route dynamic
  redirect(`/blog/${searchParams.get("slug")}`);
}

// app/blog/[slug]/page.tsx
export default async function Page({ params }: { params: Promise<{ slug: string }> }) {
  const { isEnabled } = await draftMode();
  const post = isEnabled ? await getDraftPost(slug) : await getPublishedPost(slug);
  return <Article post={post} />;
}
```

Gotchas

- revalidatePath("/", "layout") → మొత్తం app cache flush (traffic spike + origin load).
- Webhook signature verify చేయకపోవడం → ఎవరైనా మీ cache ని flush చేయగలరు (DoS vector).
- Route handler లో revalidate చేసి client refresh మర్చిపోవడం → పాత UI.
- Tags చాలా coarse ("data" ఒక్కటే) → ప్రతి update కీ మొత్తం cache పోతుంది.
- Dev లో revalidation test చేయడం – dev లో caching disabled; next build && next start తో test చేయాలి.

- **Self-host + multiple instances** – ఒక instance లో `revalidateTag` చేస్తే మిగతా instances కీ తెలియదు → shared cache handler అవసరం.

Key Points

- **Time-based** (`revalidate`) + **on-demand** (`revalidatePath` / `revalidateTag`) – రెండూ కలిపి వాడాలి.
- **Tags = fine-grained invalidation** (`product-${id}` vs `products`).
- **Server Action** లో **revalidate** → **client router cache కూడా clear** (route handler లో కాదు).
- Webhooks కి **signature verification** తప్పనిసరి.
- Production build తో మాత్రమే test చేయాలి.

Interview దృష్టి

- "CMS update అయినప్పుడు site ఎలా refresh చేస్తావు?" → webhook → signature verify → `revalidateTag` (fine-grained) + time-based fallback.
- " `revalidatePath` vs `revalidateTag` ?" → route-based vs data-based; tags cross-route data కి మేలు.

14. Next.js 13 → 14 → 15 Caching మార్పులు

వివరణ – ఎందుకు ఇది తెలియాలి

Next 13/14 tutorials, Stack Overflow answers, AI-generated code – అన్నీ **పాత caching semantics** ని అనుసరిస్తాయి. Next 15 లో అవి పని చేయవు (లేదా వేరేలా పని చేస్తాయి). Interview లో "Next 15 లో ఏం మారింది?" అనేది సాధారణ ప్రశ్న.

మార్పుల పట్టిక

Behaviour	Next 13/14	Next 15
<code>fetch()</code> default	<code>force-cache</code> (cached) ✓	<code>no-store</code> (uncached) ✗
GET Route Handlers	cached by default	uncached by default
Client Router Cache (pages)	30s cached	0s (staleTimes తో మార్చబడ్డాయి)
<code>params</code> , <code>searchParams</code>	sync objects	Promise (await చేయాలి)
<code>cookies()</code> , <code>headers()</code> , <code>draftMode()</code>	sync	async (await చేయాలి)
React version	18	19
Dev bundler	webpack	Turbopack (stable for dev)

Migration ఉదాహరణలు

```
// ❌ Next 14 code
export default function Page({ params, searchParams }: { params: { id: string } }) {
  const data = await fetch(`/api/${params.id}`); // auto-cached
  const cookieStore = cookies();
  const theme = cookieStore.get("theme");
}

// ✅ Next 15
export default async function Page({
  params, searchParams,
}: {
  params: Promise<{ id: string }>;
  searchParams: Promise<Record<string, string | string[] | undefined>>;
}) {
  const { id } = await params; // 🔑 await
  const data = await fetch(`/api/${id}`, { cache: "force-cache" }); // 🔑 explicit
  const cookieStore = await cookies(); // 🔑 await
  const theme = cookieStore.get("theme");
}
```

```
# Automatic codemod
npx @next/codemod@canary upgrade latest
npx @next/codemod@latest next-async-request-api .
```

use cache directive (Next 15.x, experimental) – భవిష్యత్తు

```
// next.config.ts
experimental: { dynamicIO: true } // లేదా useCache: true
```

```
// 1. File level – ఈ file లోని అన్ని exports cached
"use cache";

// 2. Component level
export async function ProductList() {
  "use cache";
  cacheLife("hours"); // "seconds" | "minutes" | "hours" | "days"
  | "max"
  cacheTag("products"); // invalidation tag
  const products = await db.product.findMany(); // ✅ ORM కూడా cache అవుతుంది!
  return <List products={products} />;
}

// 3. Function level
async function getProducts() {
  "use cache";
  cacheTag("products");
  return db.product.findMany();
}
```

ఎందుకు ఇది పెద్ద మార్పు:

- **ORM/DB calls కూడా cache అవుతాయి** (ఇప్పటివరకు `fetch` కీ మాత్రమే).
- Caching **explicit** – "ఏది cache అవుతోంది?" అనే గందరగోళం పోతుంది.
- Component స్థాయిలో caching → partial caching సాధ్యం.
- `unstable_cache` కీ ప్రత్యామ్నాయం.

dynamicIO model: "ఏదే cache కాదు, మీరు `use cache` అని చెప్పే వరకు." ఇది Next 15 యొక్క "uncached by default" దిశనే మరింత ముందుకు తీసుకెళ్తుంది.

Turbopack

```
next dev --turbo # ✅ stable (Next 15) – 5-10x వేగవంతమైన HMR
next build --turbo # ⚠️ ఇంకా beta/stabilizing
```

Version ఎంచుకోవడం – ఆచరణాత్మక సలహా

పరిస్థితి	సలహా
కొత్త project	Next 15 (React 19) ✅
ఉన్న Next 14 app, stable	codemod తో upgrade plan; caching audit తప్పనిసరి
Next 13 App Router	14 → 15 దశలవారీగా
Pages Router, పెద్ద app	migration అవసరమా అని ముందు నిర్ణయించండి (Topic 29)

Gotchas

- **Upgrade తర్వాత API bills పెరగడం** – cached fetch calls ఇప్పుడు uncached (ప్రతి request కీ origin hit) ✨.
- **Async params ని await చేయకపోవడం** → warning ఇప్పుడు, error తర్వాత.
- `useCache` **experimental** – production లో వాడేముందు జాగ్రత్త.
- **Turbopack build** ఇంకా అన్ని plugins ని support చేయకపోవచ్చు.
- **React 19 breaking changes** – `propTypes`, `defaultProps` (function components), string refs తొలగింపు.

Key Points

- **Next 15 = uncached by default** (fetch, GET handlers, router cache).
- `params` / `searchParams` / `cookies()` / `headers()` = **async**.
- `use cache` + `dynamicIO` = explicit caching యొక్క భవిష్యత్తు (ORM కూడా).
- Upgrade చేసేటప్పుడు **caching audit + API cost monitoring** తప్పనిసరి.

Interview దృష్టి

- "Next 15 లో caching ఏం మారింది, ఎందుకు?" → default flip + కారణం (developers ని stale data bugs గందరగోళపరిచింది) + migration ప్రభావం.

- " `use cache` ఏం పరిష్కరిస్తుంది?" → explicit, ORM-inclusive, component-level caching.
-

Part 4 – Mutations, Forms & APIs

15. Server Actions Deep

SERVER ACTIONS • FORM నుంచి SERVER దాకా

SERVER ACTIONS – form నుంచి server దాకా, API route లేకుండా



లాభం

API route రాయనవసరం లేదు
Progressive enhancement – JS లేకున్నా పని చేస్తుంది
Type safety client నుంచి server దాకా

జాగ్రత్తలు

ఇది ఒక **public endpoint** – auth check తప్పనిసరి
Input ని ఎప్పుడూ validate చేయాలి (zod)
Closure lo ఉన్న variables client కి encrypt అయి వెళ్ళాయి

అతి సాధారణమైన భద్రతా తప్పు

"ఇది server action కదా, సురక్షితం" – కాదు, ఎవరైనా దాన్ని నేరుగా call చేయొచ్చు.
Component lo auth check చేసినా సరిపోదు – action లోపల మళ్ళీ check చేయాలి.

వివరణ

Server Action = client నుండి నేరుగా call చేయగల server function. API route రాయనవసరం లేదు, fetch రాయనవసరం లేదు, request/response handling లేదు.

```
// app/actions/product.ts
"use server";
Actions

import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
import { z } from "zod";

const schema = z.object({
  name: z.string().min(1, "Name required"),
  price: z.coerce.number().positive("Price must be positive"),
});

export async function createProduct(prevState: State, formData: FormData):
Promise<State> {
  // 🛡️ 1. Authentication – ప్రతి action లో తప్పనిసరి!
  const session = await getSession();
  if (!session) return { error: "Unauthorized" };

  // 🛡️ 2. Authorization
  if (session.role !== "admin") return { error: "Forbidden" };

  // 🛡️ 3. Validation
  const parsed = schema.safeParse({
    name: formData.get("name"),
    price: formData.get("price"),
  });
```

```

    if (!parsed.success) {
      return { fieldErrors: parsed.error.flatten().fieldErrors };
    }

    // 4. Business logic
    try {
      const product = await db.product.create({
        data: { ...parsed.data, createdBy: session.userId },
      });
      revalidatePath("/products"); // 5. Cache invalidation
      redirect(`/products/${product.id}`); // 6. Navigation (throws!)
    } catch (e) {
      return { error: "Failed to create product" };
    }
  }
}

```

Real-life Scenario

Server Action = ఇంట్లో ఇంటర్నెట్ బటన్. గతంలో: security కి ఫోన్ చేయాలంటే – నంబర్ తెలియాలి, డయల్ చేయాలి, పరిచయం చెప్పాలి, request చెప్పాలి, response వినాలి (API route + fetch + JSON + error handling). ఇప్పుడు: **ఒక బటన్ నొక్కితే చాలు** – మిగతాదంతా వ్యవస్థ చూసుకుంటుంది.

⚠️ **కానీ ఆ బటన్ ఇంటి బయట కూడా ఉంది** – ఎవరైనా నొక్కగలరు. అందుకే **ప్రతి action లో "నువ్వు ఎవరు?" అని అడగాలి** (auth check).

⚠️ Security – Server Actions public HTTP endpoints

Server Action ఒక POST endpoint గా expose అవుతుంది (obfuscated ID తో).
Attacker ఆ ID ని కనిపెట్టి, ఏ payload తోనైనా, ఎన్నిసార్లైనా call చేయగలడు.

అందుకే ప్రతి Server Action లో ఈ 4 తప్పనిసరి:

```

"use server";
export async function deleteUser(userId: string) {
  // ❌❌❌ ఇది తీవ్రమైన security hole – ఎవరైనా ఏ user నైనా delete చేయగలరు!
  await db.user.delete({ where: { id: userId } });
}

// ✅ సరైనది
export async function deleteUser(userId: string) {
  const session = await getSession(); // 1. Authentication
  if (!session) throw new Error("Unauthorized");

  if (session.role !== "admin" && session.userId !== userId) // 2. Authorization
    throw new Error("Forbidden");

  const parsed = z.string().uuid().safeParse(userId); // 3. Validation
  if (!parsed.success) throw new Error("Invalid id");

  await rateLimit(session.userId); // 4. Rate limiting
}

```

```
await db.user.delete({ where: { id: parsed.data } });
revalidatePath("/admin/users");
}
```

"ఇది server code కాబట్టి safe" అనుకోవడం అత్యంత ప్రమాదకరమైన అపోహ. Client component లో `if (user.isAdmin)` చూపించడం UI మాత్రమే – action లో check లేకపోతే bypass అవుతుంది.

Server Actions ని ఎలా call చేస్తారు

```
// 1 Form action (progressive enhancement ✅ – JS లేకపోయినా పని చేస్తుంది)
<form action={createProduct}>
  <input name="name" />
  <button>Create</button>
</form>

// 2 useActionState (pending + error state)
"use client";
const [state, formAction, isPending] = useActionState(createProduct, { });
<form action={formAction}>...</form>

// 3 Button/event handler నుండి
"use client";
<button onClick={async () => { await deleteProduct(id); }}>Delete</button>

// 4 bind తో extra arguments
const deleteWithId = deleteProduct.bind(null, product.id);
<form action={deleteWithId}><button>Delete</button></form>

// 5 startTransition లో (pending state కావాలంటే)
const [isPending, startTransition] = useTransition();
startTransition(async () => { await updateItem(data); });
```

Server component లోపల inline action

```
// app/page.tsx (server component)
export default function Page() {
  async function handleSubmit(formData: FormData) {
    "use server"; // 🔑 function-level directive
    const email = formData.get("email");
    await subscribe(email as string);
    revalidatePath("/");
  }
  return <form action={handleSubmit}><input name="email" /><button>Subscribe</button>
</form>;
}
// ⚠️ closure variables encrypted గా client కి వెళ్తాయి – secrets ని closure లో పెట్టొద్దు
```

Progressive enhancement – Server Actions యొక్క దాగిన superpower

```
// JavaScript disabled/loading కాకపోయినా ఈ form పని చేస్తుంది ✔️
<form action={createProduct}>
  <input name="name" required />
  <button type="submit">Create</button>
</form>
// Next.js దీన్ని native HTML form POST గా degrade చేస్తుంది
// (useActionState కూడా progressive enhancement ని support చేస్తుంది)
```

Error handling patterns

```
// ✔️ Expected errors → return చేయాలి (throw కాదు)
export async function updateProfile(prev: State, formData: FormData): Promise<State>
{
  const parsed = schema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) return { fieldErrors: parsed.error.flatten().fieldErrors };

  try {
    await db.user.update({ where: { id }, data: parsed.data });
    return { success: true, message: "Saved" };
  } catch (e) {
    if (e instanceof Prisma.PrismaClientKnownRequestError && e.code === "P2002") {
      return { fieldErrors: { email: ["Email already taken"] } };
    }
    console.error(e);
    return { error: "Something went wrong" }; // ⚠️ internal details leak చేయొద్దు
  }
}

// ⚠️ redirect()/notFound() throw చేస్తాయి – try లోపల పెట్టొద్దు!
export async function create(formData: FormData) {
  let id: string;
  try {
    const product = await db.product.create({ data });
    id = product.id;
  } catch (e) {
    return { error: "Failed" };
  }
  redirect(`/products/${id}`); // ✔️ try బయట
}
```

Server Actions vs Route Handlers – ఎప్పుడు ఏది

	Server Action	Route Handler (route.ts)
వాడకం	Mutations from your own UI	Public API, webhooks, third-party clients
Type safety	✔️ end-to-end	manual
Progressive enhancement	✔️	✖️
HTTP methods	POST మాత్రమే	GET/POST/PUT/DELETE...

	Server Action	Route Handler (route.ts)
External consumers	✗	✓
Streaming responses	✗	✓
Caching (GET)	✗	✓
REST semantics	✗	✓

నియమం: "మీ Next.js UI నుండి data మార్పాలంటే → Server Action. బయటివారు call చేయాలంటే → Route Handler."

పరిమితులు

- **POST మాత్రమే** (GET data fetching కి server components వాడాలి).
- **Response streaming లేదు** (progress bars కి route handler).
- **Body size limit** – default 1 MB (`serverActions.bodySizeLimit` తో మార్చొచ్చు); పెద్ద files కి direct-to-S3 upload.
- **Sequential execution** – ఒకేసారి అనేక actions queue అవుతాయి.
- **Serializable arguments మాత్రమే**.

Gotchas

- **Auth/authorization check లేకపోవడం** → అత్యంత తీవ్రమైన Next.js security bug.
- **"use server"** ని client component file లో పెట్టడం → error.
- **"use server"** ని "ఇది server component" అని అనుకోవడం – కాదు! అది **Server Action marker** (server components కి directive అవసరం లేదు).
- **redirect()** ని try/catch లోపల → redirect పని చేయదు.
- **Internal error messages ని return చేయడం** → stack traces/DB errors leak.
- **revalidatePath** మర్చిపోవడం → mutation తర్వాత UI update కాదు.
- **Rate limiting లేకపోవడం** → abuse/DoS.
- **Closure లో secrets** – encrypted అయినా, key rotation/deployment మధ్య సమస్యలు.

Key Points

- Server Action = client నుండి call చేయగల server function (`"use server"`).
- **అవి public endpoints** → auth + authz + validation + rate limit **ప్రతి action లో**.
- Progressive enhancement ✓; `revalidatePath/Tag` తో cache + router cache clear.
- Expected errors → **return**; `redirect()` try బయట.
- Mutations → Actions; public API/webhooks → Route Handlers.

Interview దృష్టి

- "Server Actions security ఎలా?" → public endpoint mental model + 4 checks – **అత్యంత ముఖ్యమైన Next.js security question.**

- "Server Action vs API route?" → పై table.

16. Forms – `useActionState`, `useFormStatus`, `useOptimistic`

వివరణ – పూర్తి production form

```
// app/actions/contact.ts
"use server";
import { z } from "zod";

const schema = z.object({
  name: z.string().min(2, "కనీసం 2 అక్షరాలు"),
  email: z.string().email("సరైన email ఇవ్వండి"),
  message: z.string().min(10, "కనీసం 10 అక్షరాలు"),
});

export type FormState = {
  success?: boolean;
  message?: string;
  fieldErrors?: Partial<Record<keyof z.infer<typeof schema>, string[]>>;
  values?: Record<string, string>; // 🗝 fail అయితే user input తిరిగి చూపించడానికి
};

export async function submitContact(prev: FormState, formData: FormData):
Promise<FormState> {
  const raw = Object.fromEntries(formData) as Record<string, string>;
  const parsed = schema.safeParse(raw);

  if (!parsed.success) {
    return {
      fieldErrors: parsed.error.flatten().fieldErrors,
      values: raw, // ✅ typed data పోకూడదు (progressive
enhancement)
    };
  }

  try {
    await sendEmail(parsed.data);
    return { success: true, message: "సందేశం పంపబడింది!" };
  } catch {
    return { message: "పంపడం విఫలమైంది. మళ్ళీ ప్రయత్నించండి.", values: raw };
  }
}
```

```
// app/contact/form.tsx
"use client";
import { useActionState } from "react";
import { useFormStatus } from "react-dom";
import { submitContact, type FormState } from "@app/actions/contact";
```

```

const initial: FormState = {};

export function ContactForm() {
  const [state, formAction] = useActionState(submitContact, initial);

  return (
    <form action={formAction} className="space-y-4">
      <div>
        <label htmlFor="name">పేరు</label>
        <input id="name" name="name" defaultValue={state.values?.name}
          aria-invalid={!state.fieldErrors?.name}
          aria-describedby={state.fieldErrors?.name ? "name-error" : undefined}
        />
        {state.fieldErrors?.name && (
          <p id="name-error" role="alert">{state.fieldErrors.name[0]}</p>
        )}
      </div>

      <div>
        <label htmlFor="email">Email</label>
        <input id="email" name="email" type="email" defaultValue=
{state.values?.email} />
        {state.fieldErrors?.email && <p role="alert">{state.fieldErrors.email[0]}
</p>}}
      </div>

      <div>
        <label htmlFor="message">సందేశం</label>
        <textarea id="message" name="message" defaultValue={state.values?.message} />
        {state.fieldErrors?.message && <p role="alert">{state.fieldErrors.message[0]}
</p>}}
      </div>

      <SubmitButton />
      {state.message && (
        <p role="status" className={state.success ? "text-green-600" : "text-red-
600"}>
          {state.message}
        </p>
      )}
    </form>
  );
}

// 🔑 useFormStatus form లోపలి *child* లోనే పని చేస్తుంది
function SubmitButton() {
  const { pending } = useFormStatus();
  return (
    <button type="submit" disabled={pending} aria-busy={pending}>
      {pending ? "పంపుతోంది..." : "పంపండి"}
    </button>
  );
}

```

useOptimistic – instant feedback

```
"use client";
import { useOptimistic } from "react";

export function TodoList({ todos }: { todos: Todo[] }) {
  const [optimisticTodos, addOptimistic] = useOptimistic(
    todos,
    (current: Todo[], newTodo: string) => [
      ...current,
      { id: `temp-${Date.now()}`, text: newTodo, done: false, pending: true },
    ]
  );

  async function action(formData: FormData) {
    const text = formData.get("text") as string;
    addOptimistic(text); // ✗ వెంటనే UI లో కనిపిస్తుంది
    await createTodo(text); // fail అయినా/పూర్తయినా React తనే
    revert/sync చేస్తుంది
  }

  return (
    <>
      <ul>
        {optimisticTodos.map((t) => (
          <li key={t.id} style={{ opacity: t.pending ? 0.5 : 1 }}>{t.text}</li>
        ))}
      </ul>
      <form action={action}><input name="text" /><button>Add</button></form>
    </>
  );
}
```

Form reset & focus (సాధారణ UX వివరాలు)

```
"use client";
const formRef = useRef<HTMLFormElement>(null);
const [state, formAction] = useActionState(submitContact, initial);

useEffect(() => {
  if (state.success) {
    formRef.current?.reset();
    toast.success(state.message);
  }
}, [state]);

<form ref={formRef} action={formAction}>...</form>
```

File uploads

```
// Server Action తో (చిన్న files, < 1MB default limit)
"use server";
export async function uploadAvatar(formData: FormData) {
  const file = formData.get("avatar") as File;
  if (!file || file.size === 0) return { error: "File required" };
}
```

```

if (file.size > 2 * 1024 * 1024) return { error: "Max 2MB" };
if (!["image/jpeg", "image/png", "image/webp"].includes(file.type))
  return { error: "Invalid type" };

const bytes = await file.arrayBuffer();
const url = await uploadToS3(Buffer.from(bytes), file.name);
await db.user.update({ where: { id }, data: { avatarUrl: url } });
revalidatePath("/profile");
return { success: true, url };
}

// ✅ పెద్ద files కి – presigned URL (client → S3 నేరుగా, server bandwidth ఆదా)
export async function getUploadUrl(fileName: string, contentType: string) {
  const session = await getSession();
  if (!session) throw new Error("Unauthorized");
  return createPresignedPost({ key: `uploads/${session.userId}/${fileName}`,
    contentType });
}

```

React Hook Form + Server Actions (complex forms)

```

"use client";
const { register, handleSubmit, formState: { errors } } = useForm({ resolver:
  zodResolver(schema) });
const [isPending, startTransition] = useTransition();

const onSubmit = handleSubmit((data) => {
  startTransition(async () => {
    const result = await submitAction(data); // typed object (FormData కాదు)
    if (result?.error) setError("root", { message: result.error });
  });
});
// ⚠️ Trade-off: progressive enhancement పోతుంది; కానీ complex validation/UX సులభం

```

Gotchas

- **useFormStatus** ని form component లోనే వాడటం → ఎప్పుడూ `pending: false`; **child** లో ఉండాలి.
- **value** వాడటం (`defaultValue` బదులు) → progressive enhancement పోతుంది.
- **Error** అయినప్పుడు user input తిరిగి ఇవ్వకపోవడం → user మళ్ళీ మొత్తం ఫైమ్ చేయాలి (ఫారమైన్ UX).
- **useOptimistic** ని action/transition బయట call చేయడం → revert పని చేయదు.
- **Client validation మాత్రమే** – server లో మళ్ళీ validate చేయాలి ఎప్పుడూ.
- **File size/type validate** చేయకపోవడం → security + cost.
- **accept attribute మాత్రమే నమ్మడం** – client-side hint మాత్రమే.

Key Points

- **useActionState** = action + state + pending; **useFormStatus** = **child** లో pending.
- **useOptimistic** = instant UI + automatic rollback.
- **defaultValue + values return** = progressive enhancement + మంచి UX.

- File uploads: చిన్నవి actions లో, పెద్దవి presigned URLs.
- Server validation ఎప్పుడూ తప్పనిసరి.

Interview దృష్టి

- "Next.js లో form ఎలా build చేస్తావు?" → Server Action + Zod + useActionState + useFormStatus + progressive enhancement + error/values return.
- "Optimistic update ఎలా?" → `useOptimistic` + action; failure లో automatic revert.

17. Route Handlers (API Routes)

వివరణ

```
// app/api/products/route.ts → /api/products
import { NextRequest, NextResponse } from "next/server";

export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url);
  const page = Number(searchParams.get("page")) ?? 1;

  const products = await db.product.findMany({ skip: (page - 1) * 20, take: 20 });

  return NextResponse.json(products, {
    headers: { "Cache-Control": "public, s-maxage=60, stale-while-revalidate=300" },
  });
}

export async function POST(request: NextRequest) {
  const session = await getSession();
  if (!session) return NextResponse.json({ error: "Unauthorized" }, { status: 401 });

  const body = await request.json();
  const parsed = createSchema.safeParse(body);
  if (!parsed.success) {
    return NextResponse.json({ error: "Validation failed", details:
      parsed.error.flatten() }, { status: 400 });
  }

  const product = await db.product.create({ data: parsed.data });
  return NextResponse.json(product, { status: 201 });
}

// PUT, PATCH, DELETE, HEAD, OPTIONS కూడా export చేయొచ్చు
```

```
// app/api/products/[id]/route.ts – dynamic (Next 15: params async!)
export async function GET(request: NextRequest, { params }: { params: Promise<{ id:
  string }> }) {
  const { id } = await params;
  const product = await db.product.findUnique({ where: { id } });
  if (!product) return NextResponse.json({ error: "Not found" }, { status: 404 });
}
```

```
    return NextResponse.json(product);  
  }  
}
```

Real-life Scenario

Route Handler = ఇంటి బయటి postbox. ఎవరైనా (mobile app, partner system, webhook, cron) ఉత్తరం వేయొచ్చు/తీసుకోవచ్చు. **Server Action = ఇంట్లోని ఇంటర్నామ్** – ఇంట్లోవాళ్ళకి మాత్రమే సౌకర్యవంతం.

ఎప్పుడు Route Handler అవసరం

- ✓ Webhooks (Stripe, Clerk, GitHub, CMS)
- ✓ Mobile/third-party clients కి public API
- ✓ File downloads / streaming responses
- ✓ OAuth callbacks
- ✓ Cron jobs (Vercel Cron)
- ✓ Health checks (/api/health)
- ✓ Sitemap/RSS/dynamic files
- ✓ Non-Next.js clients
- ✗ మీ Next.js UI నుండి data fetch (server components వాడండి)
- ✗ మీ UI నుండి mutations (Server Actions వాడండి)

Streaming response

```
// app/api/chat/route.ts – LLM streaming  
export async function POST(request: NextRequest) {  
  const { messages } = await request.json();  
  
  const stream = new ReadableStream({  
    async start(controller) {  
      const encoder = new TextEncoder();  
      const completion = await openai.chat.completions.create({ model: "gpt-4o", messages,  
stream: true });  
      for await (const chunk of completion) {  
        const text = chunk.choices[0]?.delta?.content ?? "";  
        controller.enqueue(encoder.encode(`data: ${JSON.stringify({ text })}\n\n`));  
      }  
      controller.close();  
    },  
  });  
  
  return new Response(stream, {  
    headers: {  
      "Content-Type": "text/event-stream",  
      "Cache-Control": "no-cache, no-transform",  
      Connection: "keep-alive",  
    },  
  });  
}
```

Webhooks (raw body + signature)

```
// app/api/webhooks/stripe/route.ts
import Stripe from "stripe";
import { headers } from "next/headers";

export async function POST(request: NextRequest) {
  const body = await request.text(); // 🗝️ raw text (JSON కాదు -
  // signature కి అవసరం)
  const signature = (await headers()).get("stripe-signature")!;

  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(body, signature,
      process.env.STRIPE_WEBHOOK_SECRET!);
  } catch (err) {
    return NextResponse.json({ error: "Invalid signature" }, { status: 400 }); //
    🚫
  }

  // ⚠️ Idempotency - Stripe అదే event ని retry చేయొచ్చు
  const exists = await db.processedEvent.findUnique({ where: { id: event.id } });
  if (exists) return NextResponse.json({ received: true });

  switch (event.type) {
    case "checkout.session.completed": await fulfillOrder(event.data.object); break;
    case "invoice.payment_failed": await handleFailure(event.data.object); break;
  }

  await db.processedEvent.create({ data: { id: event.id } });
  return NextResponse.json({ received: true }); // ✅ త్వరగా 200 (లేకపోతే retry
  // storm)
}
```

CORS

```
export async function OPTIONS() {
  return new Response(null, {
    status: 204,
    headers: {
      "Access-Control-Allow-Origin": process.env.ALLOWED_ORIGIN!, // ⚠️ "*" వద్దు
      // (credentials తో)
      "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
      "Access-Control-Allow-Headers": "Content-Type, Authorization",
      "Access-Control-Max-Age": "86400",
    },
  });
}
```

Caching (Next 15: GET default uncached)

```
export const dynamic = "force-static"; // route ని static చేయడం
export const revalidate = 3600; // ISR

// లేదా per-response headers (CDN కి)
```

```
return NextResponse.json(data, {
  headers: { "Cache-Control": "public, s-maxage=3600, stale-while-revalidate=86400"
},
});
```

Cron jobs (Vercel)

```
// vercel.json
{ "crons": [{ "path": "/api/cron/cleanup", "schedule": "0 3 * * *" }] }
```

```
// app/api/cron/cleanup/route.ts
export async function GET(request: NextRequest) {
  if (request.headers.get("authorization") !== `Bearer ${process.env.CRON_SECRET}`) {
    return new Response("Unauthorized", { status: 401 }); // 🚫 తప్పనిసరి
  }
  await db.session.deleteMany({ where: { expiresAt: { lt: new Date() } } });
  return NextResponse.json({ ok: true });
}
```

Special file routes

```
// app/sitemap.ts
export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const posts = await getPosts();
  return [
    { url: "https://x.com", lastModified: new Date(), changeFrequency: "daily",
    priority: 1 },
    ...posts.map((p) => ({ url: `https://x.com/blog/${p.slug}`, lastModified:
    p.updatedAt })),
  ];
}

// app/robots.ts
export default function robots(): MetadataRoute.Robots {
  return {
    rules: [{ userAgent: "*", allow: "/", disallow: ["/admin/", "/api/"] }],
    sitemap: "https://x.com/sitemap.xml",
  };
}
```

Gotchas

- `page.tsx` + `route.ts` ఒకే folder → conflict.
- Webhook లో `request.json()` వాడటం (raw body అవసరమైనప్పుడు) → signature verification fail.
- Webhook idempotency లేకపోవడం → duplicate orders/emails 🌟.
- Webhook లో నెమ్మది processing → timeout → provider retry storm (queue కి push చేసి వెంటనే 200 ఇవ్వాలి).
- Next 15 లో GET default uncached – పాత behaviour ఆశించి performance regression.

- **Cron endpoint ని protect చేయకపోవడం** → ఎవరైనా trigger చేయగలరు.
- **CORS * + credentials** → browser block + security risk.
- **Route handler లో params await మర్చిపోవడం** (Next 15).

Key Points

- Route Handlers = **public API/webhooks/streaming/cron**; UI mutations కి Server Actions.
- Webhooks: **raw body + signature + idempotency + fast 200**.
- Next 15: GET **uncached by default** → explicit caching.
- `sitemap.ts`, `robots.ts`, `opengraph-image.tsx` = special routes.

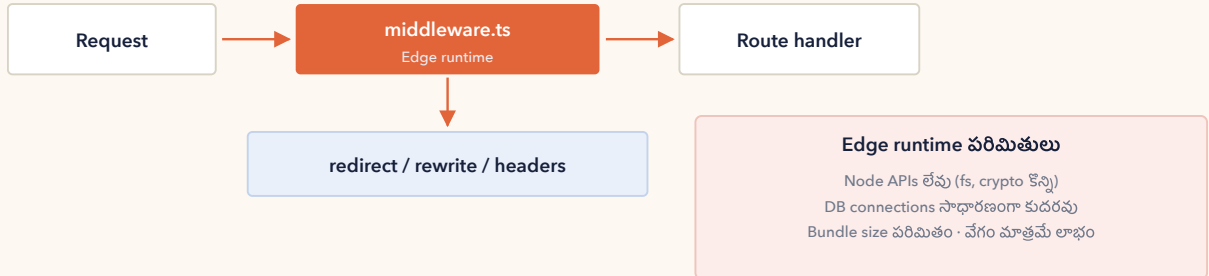
Interview దృష్టి

- "Route handler vs Server Action?" → consumer ఎవరు (external vs your UI) + capabilities.
- "Webhook ని సురక్షితంగా ఎలా handle చేస్తావు?" → signature, idempotency, fast ack + async processing.

18. Middleware & Edge Runtime

MIDDLEWARE & EDGE RUNTIME

MIDDLEWARE – request routing కి ముందు



ఎక్కడ వాడాలి, ఎక్కడ వద్దు

- ✓ Auth cookie ఉందా అని చూసి redirect · A/B split · geo-based rewrite · headers
 - X Database lookup · భారీ లెక్కలు · session validation (DB కావాలంటే)
- నియమం: middleware వేగంగా ఉండాలి – ప్రతి request దాని గుండా వెళ్తుంది.

వివరణ

Middleware = ప్రతి request కి, route render/handler కి ముందు run అయ్యే code.

```

// middleware.ts (project root, src/ ఉంటే src/middleware.ts)
import { NextResponse, type NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // 1. Redirect
  if (pathname === "/old") return NextResponse.redirect(new URL("/new",
    request.url));
  
```

```
// 2. Rewrite (URL మార్చు, content వేరే చోటి నుండి)
if (pathname.startsWith("/blog")) {
  return NextResponse.rewrite(new URL(`/cms${pathname}`, request.url));
}

// 3. Headers జోడించడం
const response = NextResponse.next();
response.headers.set("x-request-id", crypto.randomUUID());

// 4. Cookies
response.cookies.set("locale", "te", { httpOnly: true, sameSite: "lax", maxAge: 31536000 });

return response;
}

export const config = {
  matcher: [
    "/(?!api|_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|webp)$).*",
  ],
};
```

Real-life Scenario

Middleware = apartment gate security. ప్రతి ఒక్కరూ లోపలికి వెళ్ళేముందు gate దాటాలి. Guard త్వరగా చూసి – "నువ్వు వెళ్ళు" (next), "నువ్వు వేరే gate కి" (redirect/rewrite), "visitor pass తీసుకో" (header/cookie) అని నిర్ణయిస్తాడు.

⚠️ కానీ guard దగ్గర ప్రతి ఒక్కరి పూర్తి వివరాలు ఉండవు – అతను DB చూడలేడు, ఎక్కువ సమయం తీసుకోలేడు (అందరూ ఆగిపోతారు). అందుకే నిజమైన **authorization లోపల (page/action) జరగాలి**, gate దగ్గర కాదు.

Edge Runtime – పరిమితులు

Middleware **Edge Runtime** లో run అవుతుంది (V8 isolate, Node.js కాదు):

✅ అందుబాటులో	❌ లేదు
<code>fetch</code> , Web APIs	Node.js APIs (<code>fs</code> , <code>net</code> , <code>child_process</code>)
<code>crypto.subtle</code>	Native modules
<code>Request</code> / <code>Response</code>	Most DB drivers (Prisma standard, <code>pg</code>)
Cookies, headers, URL	<code>Buffer</code> (పరిమితం)
JWT verify (<code>jose</code>)	<code>bcrypt</code> (native), heavy libraries

పరిమితులు: ~1-4 MB bundle size, ~25s execution (సాధారణంగా milliseconds లో ఉండాలి), **DB access దాదాపు అసాధ్యం.**

⚠ Middleware ని authorization కి మాత్రమే వాడొద్దు (CVE-2025-29927 పాఠం)

```
// ❌ ఇది సరిపోదు - ఇదే మాత్రమే ఉంటే ప్రమాదం
export function middleware(request: NextRequest) {
  const token = request.cookies.get("session");
  if (!token) return NextResponse.redirect(new URL("/login", request.url));
  return NextResponse.next();
}
```

ఎందుకు: 2025 లో Next.js లో ఒక తీవ్రమైన vulnerability (CVE-2025-29927) కనుగొన్నారు - ఒక special header (`x-middleware-subrequest`) పంపి **middleware ని పూర్తిగా bypass** చేయొచ్చు. Patch వచ్చింది, కానీ **పాఠం శాశ్వతం:**

Middleware = optimistic UX check (redirect). నిజమైన authorization ఎప్పుడూ data layer లో (DAL/Server Action/page) జరగాలి.

```
// ✅ సరైన వ్యూహం - రెండు layers
// Layer 1: middleware - వేగవంతమైన redirect (UX)
export async function middleware(request: NextRequest) {
  const session = await verifyJwtLightweight(request.cookies.get("session")?.value);
  // DB కాదు
  if (!session && isProtected(request.nextUrl.pathname)) {
    const url = new URL("/login", request.url);
    url.searchParams.set("from", request.nextUrl.pathname);
    return NextResponse.redirect(url);
  }
  return NextResponse.next();
}

// Layer 2: DAL - నిజమైన authorization (ప్రతి data access లో) 🛡
// lib/dal.ts
import "server-only";
import { cache } from "react";

export const verifySession = cache(async () => {
  const cookie = (await cookies()).get("session")?.value;
  const session = await decrypt(cookie);
  if (!session?.userId) redirect("/login");
  return { userId: session.userId, role: session.role };
});

export async function getUserOrders() {
  const session = await verifySession(); // 🛡 ప్రతిసారి
  return db.order.findMany({ where: { userId: session.userId } }); // 🛡 user కి తనవే
}
```

సాధారణ middleware వాడకాలు

```
// 1. A/B testing
const bucket = request.cookies.get("bucket")?.value ?? (Math.random() < 0.5 ? "a" : "b");
const res = NextResponse.rewrite(new URL(`/variant-${bucket}${pathname}`,
```

```

request.url));
res.cookies.set("bucket", bucket);

// 2. Geo-based routing
const country = request.headers.get("x-vercel-ip-country") ?? "IN";
if (pathname === "/" ) return NextResponse.rewrite(new
URL(`/${country.toLowerCase()}`, request.url));

// 3. Bot protection / rate limiting (Upstash Redis – edge-compatible)
const ip = request.headers.get("x-forwarded-for") ?? "unknown";
const { success } = await ratelimit.limit(ip);
if (!success) return new NextResponse("Too many requests", { status: 429 });

// 4. Maintenance mode
if (process.env.MAINTENANCE === "true" && !pathname.startsWith("/maintenance")) {
  return NextResponse.rewrite(new URL("/maintenance", request.url));
}

// 5. Security headers (CSP with nonce)
const nonce = Buffer.from(crypto.randomUUID()).toString("base64");
const csp = `default-src 'self'; script-src 'self' 'nonce-${nonce}' 'strict-dynamic';
object-src 'none';`;
const res = NextResponse.next({ request: { headers: new Headers({ "x-nonce": nonce })
} });
res.headers.set("Content-Security-Policy", csp);

```

Matcher patterns

```

export const config = {
  matcher: [
    "/dashboard/:path*", // prefix
    "/((?!api|_next/static|_next/image|favicon.ico).*)", // negative lookahead
    { source: "/api/:path*", has: [{ type: "header", key: "authorization" }] }, // conditional
  ],
};

```

Gotchas

- **Middleware ని authorization కి మాత్రమే వాడటం** → bypass risk (**CVE-2025-29927 పాఠం**).
- **Middleware లో DB call** → edge runtime లో పని చేయదు/నెమ్మది; ప్రతి request కి latency.
- **Matcher లేకపోవడం** → static assets కి కూడా run → performance నష్టం.
- **Node.js APIs వాడటం** → build error.
- **Middleware లో ఎక్కువ logic** → ప్రతి request నెమ్మది అవుతుంది (TTFB పెరుగుతుంది).
- **NextResponse.next() return చేయకపోవడం** → request hang.
- **Cookies ని NextResponse మీద కాకుండా వేరే చోట set చేయడం** → apply కావు.

Key Points

- Middleware = **edge, ప్రతి request కి, వేగంగా ఉండాలి** (redirect/rewrite/headers/cookies).
- **Authorization ని middleware మీద ఆధారపడొద్దు** – DAL/page/action లో enforce చేయాలి.
- Edge runtime = Web APIs మాత్రమే (Node APIs, most DB drivers ❌).

- `matcher` తో scope పరిమితం చేయాలి.

Interview దృష్టి

- "Middleware లో *auth* చేయొచ్చా?" → optimistic redirect కి అవును; **authorization కి కాదు** – CVE + defense-in-depth చెప్తే బలమైన impression.
 - "Edge vs Node runtime?" → APIs, cold start, DB access, వాడకాలు.
-

Part 5 – Production Concerns

19. Authentication & Authorization

వివరణ – Next.js లో auth యొక్క సరైన architecture

Layer 1: Middleware → optimistic redirect (UX వేగం) – security కాదు
Layer 2: DAL (Data Access Layer) → 🔒 నిజమైన authorization (ప్రతి data access లో)
Layer 3: Server Actions → 🔒 ప్రతి mutation లో auth check
Layer 4: UI → conditional rendering (UX మాత్రమే)

బంగారు నియమం: "Data ని touch చేసే ప్రతి చోట auth check ఉండాలి." UI checks మరియు middleware checks **UX కోసం మాత్రమే.**

Session strategies

	Stateless (JWT in cookie)	Stateful (DB session)
Storage	encrypted cookie	DB/Redis + session id cookie
Scaling	✅ server state లేదు	DB lookup ప్రతి request
Revocation	❌ కష్టం (expiry వరకు valid)	✅ instant logout అన్ని devices లో
Size	cookie limit 4KB	id మాత్రమే
Edge/middleware verify	✅ (jose తో)	❌ (DB అవసరం)
ఎప్పుడు	simple apps, edge-first	enterprise, compliance, instant revoke

DAL pattern (Next.js team అధికారిక సిఫార్సు) ★

```
// lib/dal.ts
import "server-only"; // 🔒 client import చేస్తే build fail
import { cache } from "react";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export const verifySession = cache(async () => {
  const cookie = (await cookies()).get("session")?.value;
  const session = await decrypt(cookie);
  if (!session?.userId) redirect("/login");
  return { isAuth: true, userId: session.userId, role: session.role };
});

// ✅ ప్రతి data function లో session check + ownership filter
export const getUser = cache(async () => {
```

```

const session = await verifySession();
return db.user.findUnique({
  where: { id: session.userId },
  select: { id: true, name: true, email: true, role: true }, // 🛡️
password/tokens కాదు!
});
});

export const getOrder = cache(async (orderId: string) => {
  const session = await verifySession();
  const order = await db.order.findUnique({ where: { id: orderId } });

  // 🛡️ Ownership check – ఇది లేకపోతే IDOR vulnerability!
  if (!order || (order.userId !== session.userId && session.role !== "admin")) {
    notFound(); // 404 (403 కాదు – existence ని leak చేయొద్దు)
  }
  return order;
});

```

IDOR (Insecure Direct Object Reference) – `/orders/123` కి బదులు `/orders/124` అని type చేస్తే ఇంకొకరి order కనిపించడం. **అత్యంత సాధారణ real-world vulnerability.** Ownership check ప్రతి query లో తప్పనిసరి.

DTO pattern – data leaks నివారణ

```

// ❌ మొత్తం user object client కి
return db.user.findUnique({ where: { id } }); // password hash, tokens, internal flags leak!

// ✅ DTO – role ఆధారంగా ఏమి ఇవ్వాలో నిర్ణయించడం
function toUserDTO(user: User, viewerRole: Role) {
  return {
    id: user.id,
    name: user.name,
    ...(viewerRole === "admin" && { email: user.email, lastLoginIp: user.lastLoginIp })
  },
};
}

```

సాంత్ session implementation (learning + control కి)

```

// lib/session.ts
import "server-only";
import { SignJWT, jwtVerify } from "jose";
import { cookies } from "next/headers";

const key = new TextEncoder().encode(process.env.SESSION_SECRET);

export async function encrypt(payload: { userId: string; role: string; expiresAt: Date }) {
  return new SignJWT(payload)
    .setProtectedHeader({ alg: "HS256" })

```

```

        .setIssuedAt()
        .setExpirationTime("7d")
        .sign(key);
    }

export async function decrypt(session?: string) {
    if (!session) return null;
    try {
        const { payload } = await jwtVerify(session, key, { algorithms: ["HS256"] });
        return payload as { userId: string; role: string };
    } catch { return null; }
}

export async function createSession(userId: string, role: string) {
    const expiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000);
    const session = await encrypt({ userId, role, expiresAt });

    (await cookies()).set("session", session, {
        httpOnly: true, // 🛡️ JavaScript చదవలేదు (XSS రక్షణ)
        secure: process.env.NODE_ENV === "production", // 🛡️ HTTPS మాత్రమే
        sameSite: "lax", // 🛡️ CSRF రక్షణ
        expires: expiresAt,
        path: "/",
    });
}

export async function deleteSession() {
    (await cookies()).delete("session");
}

```

```

// app/actions/auth.ts
"use server";
import bcrypt from "bcryptjs";

export async function login(prev: State, formData: FormData): Promise<State> {
    const parsed = loginSchema.safeParse(Object.fromEntries(formData));
    if (!parsed.success) return { fieldErrors: parsed.error.flatten().fieldErrors };

    await rateLimit(`login:${parsed.data.email}`); // 🛡️ brute force రక్షణ

    const user = await db.user.findUnique({ where: { email: parsed.data.email } });

    // 🛡️ timing attack నివారణ – user లేకపోయినా hash compare చేయాలి
    const valid = user
        ? await bcrypt.compare(parsed.data.password, user.passwordHash)
        : await bcrypt.compare(parsed.data.password, DUMMY_HASH);

    // 🛡️ "email తప్పు" vs "password తప్పు" అని చెప్పొద్దు (user enumeration)
    if (!user || !valid) return { error: "Invalid email or password" };

    await createSession(user.id, user.role);
    redirect("/dashboard");
}

export async function logout() {
    await deleteSession();
}

```

```
    redirect("/login");  
  }  
}
```

Libraries – ఏది ఎప్పుడు

Library	ప్రత్యేకత	ఎప్పుడు
Auth.js (NextAuth v5)	OAuth providers చాలా, free, self-hosted	Social login + credentials, control కావాలి
Clerk	Full UI components, orgs, MFA (paid)	వేగంగా ship చేయాలి, B2B/orgs
Lucia (deprecated → guide)	lightweight, DB-agnostic	learning + custom control
Supabase Auth	Supabase వాడితే built-in	Supabase stack
సొంతం	పూర్తి control	simple needs, learning; ⚠️ ఖచ్చితత్వం అవసరం

```
// Auth.js v5 (Next.js App Router)  
// auth.ts  
import NextAuth from "next-auth";  
import Google from "next-auth/providers/google";  
import Credentials from "next-auth/providers/credentials";  
  
export const { handlers, auth, signIn, signOut } = NextAuth({  
  providers: [  
    Google,  
    Credentials({  
      async authorize(credentials) {  
        const user = await verifyUser(credentials);  
        return user ?? null;  
      },  
    )],  
  },  
  session: { strategy: "jwt" },  
  callbacks: {  
    jwt({ token, user }) { if (user) token.role = user.role; return token; },  
    session({ session, token }) { session.user.role = token.role as string; return session; },  
  },  
  pages: { signIn: "/login" },  
});  
  
// app/api/auth/[...nextauth]/route.ts  
export const { GET, POST } = handlers;  
  
// వాడకం (server component/action లో)  
const session = await auth();
```

RBAC (Role-Based Access Control)

```
// lib/permissions.ts
const PERMISSIONS = {
  admin: ["users:read", "users:write", "orders:read", "orders:write",
"settings:write"],
  manager: ["users:read", "orders:read", "orders:write"],
  user: ["orders:read"],
} as const satisfies Record<Role, readonly string[]>;

export async function requirePermission(permission: string) {
  const session = await verifySession();
  const allowed = PERMISSIONS[session.role] as readonly string[];
  if (!allowed.includes(permission)) {
    throw new ForbiddenError(`Missing permission: ${permission}`);
  }
  return session;
}

// Server Action లో
"use server";
export async function deleteUser(id: string) {
  await requirePermission("users:write"); // 🔒
  await db.user.delete({ where: { id } });
  revalidatePath("/admin/users");
}
```

Gotchas

- **Middleware మీద మాత్రమే ఆధారపడటం** → bypass risk (Topic 18).
- **Ownership check లేకపోవడం** → IDOR (అత్యంత సాధారణ real bug).
- **Password hash/tokens ని client కి పంపడం** → DTO వాడాలి.
- **httpOnly లేని cookie లేదా localStorage లో token** → XSS లో దొంగిలించబడుతుంది.
- **"Email not found" vs "Wrong password"** → user enumeration.
- **Rate limiting లేకపోవడం** → brute force.
- **Session ని client component లో validate చేయడం** – client code ని ఎప్పుడూ నమ్మొద్దు.
- **sameSite set చేయకపోవడం** → CSRF.
- **Server Component లో session ని prop గా client కి పంపడం** → RSC payload లో కనిపిస్తుంది (public data మాత్రమే పంపాలి).

Key Points

- **4 layers:** middleware (UX) → **DAL** (🔒 real authz) → actions (🔒) → UI (UX).
- **verifySession + ownership check + DTO** = production auth pattern.
- Cookies: **httpOnly** + **secure** + **sameSite** + expiry.
- Login: rate limit + timing-safe + generic error messages.
- Stateless JWT vs stateful sessions – revocation trade-off.

Interview దృష్టి

- "Next.js లో auth ఎలా implement చేస్తావు?" → layered architecture + DAL + ownership + DTO – ఇది SSE-level answer.
- "IDOR అంటే ఏమిటి?" → ownership check లేకపోవడం + `notFound()` (403 కాదు) ఎందుకు.

20. Database & ORM Integration

వివరణ

```
// lib/db.ts – Prisma singleton (dev లో hot reload వల్ల connections పేలకుండా) ☆
import "server-only";
import { PrismaClient } from "@prisma/client";

const globalForPrisma = globalThis as unknown as { prisma?: PrismaClient };

export const db =
  globalForPrisma.prisma ??
  new PrismaClient({ log: process.env.NODE_ENV === "development" ? ["query", "error"] : ["error"] });

if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = db;
```

ఈ singleton ఎందుకు: dev లో ప్రతి hot reload కీ కొత్త `PrismaClient` → వందల DB connections → "too many connections" error. ఇది Next.js + Prisma యొక్క #1 సాధారణ సమస్య.

Serverless connection pooling – అత్యంత ముఖ్యమైన production సమస్య

సమస్య: ప్రతి serverless function instance కీ own DB connection.
Traffic spike → 500 concurrent functions → 500 connections
→ Postgres default max_connections = 100 → ✨ "too many clients"

పరిష్కారాలు:

పరిష్కారం	వివరణ
PgBouncer / Supabase pooler	Connection pooler ముందు పెట్టడం (transaction mode)
Prisma Accelerate	Prisma యొక్క managed pooler + edge cache
Neon / PlanetScale	Serverless-native DBs (HTTP-based, pooling built-in)
Drizzle + HTTP driver	<code>@neondatabase/serverless</code> – HTTP, connections లేవు
<code>connection_limit=1</code>	ప్రతి function కీ ఒక్క connection

```
# Prisma + PgBouncer
DATABASE_URL="postgresql://user:pass@pooler:6543/db?
pgbouncer=true&connection_limit=1"
DIRECT_URL="postgresql://user:pass@db:5432/db"      # migrations కి (pooler bypass)
```

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  directUrl = env("DIRECT_URL")
}
```

Prisma vs Drizzle (2026 ఎంపిక)

	Prisma	Drizzle
API	Schema-first, generated client	SQL-like, TypeScript-first
Bundle	పెద్దది (engine binary)	చిన్నది (~10KB)
Edge support	Accelerate/driver adapters తో	✅ native
Migrations	✅ అద్భుతం (prisma migrate)	✅ (drizzle-kit)
Type inference	generated types	inferred from schema ✅
Learning curve	తక్కువ	SQL తెలిస్తే సులభం
Complex queries	పరిమితం (raw కి fallback)	✅ SQL కి దగ్గరగా

```
// Drizzle ఉదాహరణ
import { drizzle } from "drizzle-orm/neon-http";
import { eq, and, desc } from "drizzle-orm";

const products = await db
  .select()
  .from(productsTable)
  .where(and(eq(productsTable.active, true), eq(productsTable.categoryId, id)))
  .orderBy(desc(productsTable.createdAt))
  .limit(20);

type Product = typeof productsTable.$inferSelect;      // ✅ inferred
```

Query patterns

```
// 1. N+1 నివారణ - include/join
// ❌ N+1: 1 + 100 queries
const posts = await db.post.findMany();
for (const post of posts) post.author = await db.user.findUnique({ where: { id:
post.authorId } });

// ✅ 1 query
```

```

const posts = await db.post.findMany({ include: { author: { select: { id: true, name: true } } } });

// 2. Select మాత్రమే కావలసినవి (over-fetching నివారణ)
const users = await db.user.findMany({ select: { id: true, name: true } });

// 3. Pagination – cursor-based (offset కంటే scalable)
const posts = await db.post.findMany({
  take: 20,
  ...(cursor && { skip: 1, cursor: { id: cursor } }),
  orderBy: { createdAt: "desc" },
});

// 4. Transactions
await db.$transaction(async (tx) => {
  const order = await tx.order.create({ data: orderData });
  await tx.inventory.update({
    where: { productId },
    data: { stock: { decrement: qty } },
  });
  // ⚠ ఏదైనా throw అయితే మొత్తం rollback ✅
});

// 5. Optimistic concurrency (race condition నివారణ)
const updated = await db.product.updateMany({
  where: { id, version: currentVersion },
  data: { stock: { decrement: 1 }, version: { increment: 1 } },
});
if (updated.count === 0) throw new Error("Concurrent modification – retry");

```

Caching DB queries

```

// 1. React.cache – request-level dedup (Topic 11)
export const getProduct = cache(async (id: string) => db.product.findUnique({ where: { id } }));

// 2. unstable_cache – persistent Data Cache
const getCachedProducts = unstable_cache(
  async () => db.product.findMany({ where: { featured: true } }),
  ["featured-products"],
  { revalidate: 3600, tags: ["products"] }
);

// 3. Next 15.x – "use cache"
async function getProducts() {
  "use cache";
  cacheTag("products");
  cacheLife("hours");
  return db.product.findMany();
}

```

Gotchas

- **Prisma singleton లేకపోవడం** → dev లో connection exhaustion.
- **Serverless లో pooler లేకపోవడం** → production లో traffic spike వస్తే DB down 🚨.

- **Client component లో DB import** → build error (`server-only` దీన్ని ముందే పట్టుకుంటుంది).
- **N+1 queries** → నెమ్మది (Prisma logging తో కనిపెట్టాలి).
- `select` లో **లేకుండా మొత్తం rows** → over-fetching + password hash leak risk.
- **Migrations ని pooler ద్వారా run చేయడం** → fail (`directUrl` కావాలి).
- **Transactions లో నెమ్మది operations** (external API calls) → lock contention.
- **Edge runtime లో standard Prisma** → పని చేయదు (Accelerate/adaptor అవసరం).

Key Points

- **Singleton pattern** (dev hot reload) + **connection pooler** (serverless) – రెండూ తప్పనిసరి.
- Prisma (DX, migrations) vs Drizzle (edge, bundle, SQL control).
- `select / include` తో N+1 & over-fetching నివారణ.
- Cursor pagination, transactions, optimistic concurrency.
- DB caching: `React.cache` (dedup) + `unstable_cache / use cache` (persistent).

Interview దృష్టి

- "Serverless లో DB connections ఎలా handle చేస్తావు?" → pooler/PgBouncer/serverless driver + `connection_limit` – production maturity signal.
- "N+1 ఎలా కనిపెడతావు?" → query logging, APM, `include` వాడకో.

21. Images, Fonts, Metadata & SEO

NEXT/IMAGE · ఆటోమేటిక్ OPTIMISATION

IMAGE OPTIMISATION – next/image ఏం చేస్తుంది

Format	WebP / AVIF ఆటోమేటిక్
Size	device కి సరిపడా – srcset
Lazy	viewport lo వచ్చేదాకా load కాదు
CLS	width/height తప్పనిసరి → layout shift లేదు

తరచుగా వచ్చే తప్పులు

width/height ఇవ్వకపోవడం → layout shift (Core Web Vitals దెబ్బ)
 Above-the-fold image కి priority ఇవ్వకపోవడం → LCP నెమ్మది
 బయటి domain నుంచి images → next.config lo remotePatterns చేర్చాలి

next/image – automatic optimization

```
import Image from "next/image";

// Local image – width/height automatic (import time లో తెలుసు)
import hero from "@/public/hero.jpg";
<Image src={hero} alt="Hero" priority placeholder="blur" />
```

```
// Remote image – dimensions ఇవ్వాలి
<Image
  src="https://cdn.example.com/product.jpg"
  alt="Product"
  width={800}
  height={600}
  priority // 🔑 LCP image కి (lazy loading అవుతుంది + preload)
  quality={85}
  sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 800px"
  placeholder="blur"
  blurDataURL="data:image/jpeg;base64,..."
/>

// Fill (parent size ప్రకారం)
<div className="relative h-64 w-full">
  <Image src={url} alt="" fill className="object-cover" sizes="100vw" />
</div>
```

ఏం చేస్తుంది: WebP/AVIF conversion, responsive srcset, lazy loading, **CLS నివారణ** (dimensions reserve), on-demand resize + CDN cache.

```
// next.config.ts
images: {
  remotePatterns: [{ protocol: "https", hostname: "cdn.example.com" }], // 🔒
  తప్పనిసరి
  formats: ["image/avif", "image/webp"],
  deviceSizes: [640, 750, 828, 1080, 1200, 1920],
  minimumCacheTTL: 60,
}
```

priority నియమం: above-the-fold LCP image కి **మాత్రమే** (సాధారణంగా page కి ఒకటి). అన్నిటికీ పెడితే – అన్నీ ఒకేసారి download → LCP పాడవుతుంది.

next/font – zero layout shift

```
import { Inter, Noto_Sans_Telugu } from "next/font/google";
import localFont from "next/font/local";

const inter = Inter({
  subsets: ["latin"],
  display: "swap",
  variable: "--font-inter",
  preload: true,
});

const telugu = Noto_Sans_Telugu({ subsets: ["telugu"], variable: "--font-telugu" });

const custom = localFont({
  src: [{ path: "../fonts/Custom-Regular.woff2", weight: "400" }],
  variable: "--font-custom",
});
```

```
<html className={` ${inter.variable} ${telugu.variable}`}>
```

లాభాలు: fonts **self-hosted** (Google కి request లేదు → privacy + వేగం), automatic **size-adjust** (**CLS = 0**), subsetting, preload.

Metadata API

```
// Static
export const metadata: Metadata = {
  title: { default: "MyShop", template: "%s | MyShop" },
  description: "...",
  keywords: ["shop", "products"],
  metadataBase: new URL("https://myshop.com"), // 🗝 relative URLs కి
  openGraph: {
    title: "MyShop", description: "...", url: "https://myshop.com",
    siteName: "MyShop", locale: "te-IN", type: "website",
    images: [{ url: "/og.png", width: 1200, height: 630 }],
  },
  twitter: { card: "summary_large_image", images: ["/og.png"] },
  robots: { index: true, follow: true, googleBot: { index: true, "max-image-preview":
    "large" } },
  alternates: { canonical: "/", languages: { "en-US": "/en", "te-IN": "/te" } },
};

// Dynamic
export async function generateMetadata({ params }: Props): Promise<Metadata> {
  const { id } = await params;
  const product = await getProduct(id); // ✅ deduped (page లో మళ్ళీ fetch
    చేసినా)
  if (!product) return { title: "Not Found" };

  return {
    title: product.name,
    description: product.description.slice(0, 160),
    openGraph: { images: [product.imageUrl] },
    alternates: { canonical: `/products/${id}` },
  };
}
```

Dynamic OG images

```
// app/products/[id]/opengraph-image.tsx
import { ImageResponse } from "next/og";

export const size = { width: 1200, height: 630 };
export const contentType = "image/png";

export default async function Image({ params }: { params: { id: string } }) {
  const product = await getProduct(params.id);
  return new ImageResponse(
    (
      <div style={{ display: "flex", flexDirection: "column", width: "100%", height:
        "100%",
        alignItems: "center", justifyContent: "center", background:
```

```

"#111", color: "#fff" }}>
  <h1 style={{ fontSize: 60 }}>{product.name}</h1>
  <p style={{ fontSize: 32 }}>₹{product.price}</p>
</div>
),
{ ...size }
);
}

```

Structured data (JSON-LD) – rich results

```

export default async function ProductPage({ params }: Props) {
  const product = await getProduct((await params).id);
  const jsonLd = {
    "@context": "https://schema.org",
    "@type": "Product",
    name: product.name,
    image: product.imageUrl,
    offers: {
      "@type": "Offer",
      price: product.price,
      priceCurrency: "INR",
      availability: product.stock > 0
        ? "https://schema.org/InStock" : "https://schema.org/OutOfStock",
    },
    aggregateRating: { "@type": "AggregateRating", ratingValue: product.rating,
    reviewCount: product.reviewCount },
  };

  return (
    <>
      <script type="application/ld+json" dangerouslySetInnerHTML={{ __html:
JSON.stringify(jsonLd) }} />
      <ProductView product={product} />
    </>
  );
}

```

Gotchas

- **priority** ని అన్ని **images** కీ → LCP పాడవుతుంది.
- **sizes** ఇవ్వకపోవడం (**fill** తో) → browser అతిపెద్ద image download చేస్తుంది.
- **remotePatterns** లో **hostname: "**"** → ఎవరైనా మీ image optimizer ని వాడొచ్చు (cost abuse) .
- **metadataBase** లేకపోవడం → OG images relative URLs తో break.
- **Client component** లో **metadata export** → పని చేయదు (server only).
- **alt** లేకపోవడం → a11y + SEO నష్టం.
- **next/font** బదులు **<link> Google Fonts** → CLS + privacy + extra request.
- **Self-host** లో **image optimization** → **sharp** install చేయాలి, CPU/memory ఎక్కువ (CDN loader పరిగణించాలి).

Key Points

- `next/image` : formats, responsive, lazy, **CLS-safe**; `priority` LCP కి ఒక్కటి.
- `next/font` : self-hosted, **zero CLS**, subsetting.
- Metadata API: static + `generateMetadata` (deduped fetch); `metadataBase` తప్పనిసరి.
- `opengraph-image.tsx` = dynamic OG; JSON-LD = rich results.

Interview దృష్టి

- "Next.js లో images ఎలా optimize చేస్తారు?" → `next/image` internals + sizes/priority + `remotePatterns` security.
- "SEO ఎలా handle చేస్తారు?" → metadata, canonical, sitemap/robots, JSON-LD, SSR/ISR, Core Web Vitals.

22. Styling in Next.js

వివరణ – RSC యుగంలో styling

విధానం	RSC compatible	Runtime cost	సిఫార్సు
Tailwind CSS	✓	0	✓ default ఎంపిక
CSS Modules	✓	0	✓
Global CSS	✓	0	✓ (base styles కి)
vanilla-extract / Panda	✓	0	✓ type-safe
styled-components / emotion	✗ ('use client' అవసరం)	⚠ ఉంది	✗ కొత్త projects లో

```
// Tailwind + cn helper (ప్రామాణిక pattern)
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
export function cn(...inputs: ClassValue[]) { return twMerge(clsx(inputs)); }

<div className={cn("rounded p-4", isActive && "bg-blue-500", className)} />
// twMerge – conflicting classes ని సరిగ్గా resolve చేస్తుంది ("p-4 p-6" → "p-6")
```

```
// CSS Modules
import styles from "./Card.module.css";
<div className={styles.card} />
```

CSS-in-JS ఎందుకు సమస్య: runtime లో styles generate చేయాలి → React context + runtime అవసరం → **Server Components లో పని చేయదు**. Workaround (registry + `'use client'`) ఉంది కానీ RSC ప్రయోజనం తగ్గుతుంది.

```
// styled-components తప్పనిసరి అయితే - registry అవసరం
// lib/registry.tsx
"use client";
export default function StyledComponentsRegistry({ children }: { children:
React.ReactNode }) {
  const [sheet] = useState(() => new ServerStyleSheet());
  useServerInsertedHTML(() => {
    const styles = sheet.getStyleElement();
    sheet.instance.clearTag();
    return <{styles}</>;
  });
  if (typeof window !== "undefined") return <{children}</>;
  return <StyleSheetManager sheet={sheet.instance}>{children}</StyleSheetManager>;
}
```

Dark mode (flash లేకుండా)

```
// next-themes - ప్రామాణిక పరిష్కారం
"use client";
import { ThemeProvider } from "next-themes";
export function Providers({ children }: { children: React.ReactNode }) {
  return <ThemeProvider attribute="class" defaultTheme="system" enableSystem>
{children}</ThemeProvider>;
}
// app/layout.tsx
<html lang="te" suppressHydrationWarning>      { /* 🗝️ theme class mismatch వల్ల */ }
```

Key Points

- **Tailwind / CSS Modules = RSC-safe, zero runtime** (2026 default).
- Runtime CSS-in-JS = 'use client' + registry (RSC ప్రయోజనం తగ్గుతుంది).
- `cn()` = `clsx` + `tailwind-merge`.
- Dark mode: `next-themes` + `suppressHydrationWarning`.

23. Error Handling, Redirects & Logging

Error handling layers

1. TypeScript + Zod → errors ముందే ఆపడం
2. try/catch (actions, handlers) → expected errors
3. error.tsx (per segment) → render errors
4. global-error.tsx → root layout errors
5. not-found.tsx → 404
6. Sentry/monitoring → production visibility

```
// Typed error hierarchy (Topic 27 in TypeScript_Telugu.md)
export class AppError extends Error {
```

```

    constructor(message: string, public statusCode = 500, public code = "INTERNAL") {
      super(message);
      this.name = this.constructor.name;
    }
  }
export class NotFoundError extends AppError {
  constructor(resource: string) { super(`${resource} not found`, 404, "NOT_FOUND"); }
}
export class ForbiddenError extends AppError {
  constructor(msg = "Forbidden") { super(msg, 403, "FORBIDDEN"); }
}

```

Sentry integration

```

// instrumentation.ts – Next.js server startup hook
export async function register() {
  if (process.env.NEXT_RUNTIME === "nodejs") await import("./sentry.server.config");
  if (process.env.NEXT_RUNTIME === "edge") await import("./sentry.edge.config");
}

// instrumentation-client.ts / sentry.client.config.ts
Sentry.init({
  dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,
  tracesSampleRate: 0.1,
  replaysOnErrorSampleRate: 1.0,
  beforeSend(event) { delete event.request?.cookies; return event; }, // 🛡️ PII
});

// onRequestError hook (Next 15) – server errors capture
export function onRequestError(err: unknown, request: Request, context: unknown) {
  Sentry.captureException(err, { extra: { url: request.url, context } });
}

```

Structured logging

```

// lib/logger.ts – pino (serverless-friendly)
import pino from "pino";
export const logger = pino({
  level: process.env.LOG_LEVEL ?? "info",
  redact: ["req.headers.authorization", "req.headers.cookie", "*.password",
    "*.token"], // 🛡️
  formatters: { level: (label) => ({ level: label }) },
});

logger.info({ userId, orderId, durationMs }, "order created"); // ✅ structured (searchable)
// ❌ console.log(`order created for ${userId}`) – grep చేయలేం, parse చేయలేం

```

Redirects

```

// 1. next.config.ts – static redirects (build time, CDN level ↗)
async redirects() {
  return [
    { source: "/old-blog/:slug", destination: "/blog/:slug", permanent: true },

```

```
// 308
  { source: "/legacy", destination: "/new", permanent: false },
// 307
  { source: "/*:path*", has: [{ type: "host", value: "old.example.com" }],
    destination: "https://new.example.com/*:path*", permanent: true },
  ];
}

// 2. Middleware – dynamic (per-request logic)
// 3. redirect() – server component/action లో
```

Key Points

- Layered error handling; `error.tsx` (client) + `global-error.tsx` + `not-found.tsx`.
- `error.digest` = server↔client correlation (production లో messages hidden).
- `instrumentation.ts` + `onRequestError` = Sentry integration points.
- **Structured logging** (pino) + PII redaction.
- Redirects: config (static, CDN) > middleware (dynamic) > `redirect()` (in-code).

24. Internationalization (i18n)

వివరణ – App Router లో i18n

App Router లో built-in i18n routing **లేదు** (Pages Router లో ఉండేది) – dynamic segment తో manually చేయాలి.

```
app/
├── [locale]/
│   ├── layout.tsx
│   ├── page.tsx
│   └── products/page.tsx
└── middleware.ts          # locale detect + redirect
```

```
// middleware.ts – locale detection
import { match } from "@formatjs/intl-localematcher";
import Negotiator from "negotiator";

const locales = ["te", "en", "hi"];
const defaultLocale = "en";

function getLocale(request: NextRequest): string {
  const cookieLocale = request.cookies.get("NEXT_LOCALE")?.value;
  if (cookieLocale && locales.includes(cookieLocale)) return cookieLocale;

  const headers = { "accept-language": request.headers.get("accept-language") ?? "" };
  const languages = new Negotiator({ headers }).languages();
  return match(languages, locales, defaultLocale);
}
```

```
export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const hasLocale = locales.some((l) => pathname.startsWith(`/${l}/`) || pathname === `/${l}`);
  if (hasLocale) return;

  const locale = getLocale(request);
  return NextResponse.redirect(new URL(`/${locale}${pathname}`, request.url));
}

export const config = { matcher: ["/(?!_next|api|.*\\.\\.\\.).*.") ];
```

```
// lib/dictionaries.ts - server-side dictionaries (client bundle లోకి వెళ్ళవు ✅)
import "server-only";

const dictionaries = {
  en: () => import("@/dictionaries/en.json").then((m) => m.default),
  te: () => import("@/dictionaries/te.json").then((m) => m.default),
  hi: () => import("@/dictionaries/hi.json").then((m) => m.default),
};

export const getDictionary = async (locale: Locale) => dictionaries[locale]();
```

```
// app/[locale]/page.tsx
export async function generateStaticParams() {
  return locales.map((locale) => ({ locale }));    // ✅ ప్రతి locale కి static pages
}

export default async function Page({ params }: { params: Promise<{ locale: Locale }>
}) {
  const { locale } = await params;
  const dict = await getDictionary(locale);
  return <h1>{dict.home.title}</h1>;
}

// SEO - hreflang
export async function generateMetadata({ params }: Promise<Metadata> {
  const { locale } = await params;
  return {
    alternates: {
      canonical: `/${locale}`,
      languages: { "en-US": "/en", "te-IN": "/te", "hi-IN": "/hi" },
    },
  };
}
```

Libraries: `next-intl` (App Router కి ఉత్తమం), `next-i18next` (Pages Router), `paraglide` (compile-time, tree-shakeable).

Formatting (Intl API – library అవసరం లేదు)

```
new Intl.NumberFormat("te-IN", { style: "currency", currency: "INR"
}).format(1234.5); // ₹1,234.50
new Intl.DateTimeFormat("te-IN", { dateStyle: "long" }).format(new Date());
```

```
new Intl.RelativeTimeFormat("te", { numeric: "auto" }).format(-3, "day");
new Intl.PluralRules("te").select(1); // "one"
```

Key Points

- App Router లో **manual** ([locale] segment + middleware).
- Dictionaries **server-side** → client bundle లోకి translations వెళ్ళవు.
- **generateStaticParams** తో ప్రతి locale కి static pages.
- **hreflang** + canonical = SEO.
- **Intl** API = formatting (library అవసరం లేదు).

25. Performance Optimization

వివరణ – Next.js performance checklist (priority క్రమంలో)

1. ☒ సరైన rendering strategy (static/ISR అవకాశం ఉన్న చోట dynamic వాడొద్దు)
2. ☒ Client bundle తగ్గించడం ('use client' boundary కిందికి)
3. ☒ Images (next/image + priority + sizes)
4. ☒ Fonts (next/font)
5. ☒ Data waterfalls తొలగింపు (parallel/preload)
6. ☒ Streaming (Suspense granularity)
7. ☒ Caching (Data Cache + CDN)
8. ☒ Third-party scripts (next/script strategy)
9. ☒ Dynamic imports (heavy client libs)
10. ☒ Database (indexes, N+1, pooling)

Bundle analysis

```
ANALYZE=true npm run build
```

Route (app)	Size	First Load JS	
└─ /	5 kB	92 kB	☒ మంచిది
└─ /dashboard	45 kB	180 kB	☐ పరిశీలించాలి
└─ /editor	320 kB	420 kB	☒ సమస్య!

```
// పరిష్కారం – heavy libraries ని lazy load
const Editor = dynamic(() => import("@/components/Editor"), {
  loading: () => <EditorSkeleton />,
  ssr: false,
});

// Named export అయితే
const Chart = dynamic(() => import("@/components/Chart").then((m) => m.Chart));

// Event handler లో import (button click తర్వాత మాత్రమే)
```

```
const onExport = async () => {
  const { jsPDF } = await import("jspdf"); // ✅ 200KB library, అవసరమైనప్పుడే
  new jsPDF().save("report.pdf");
};
```

Third-party scripts – next/script

```
import Script from "next/script";

<Script src="https://analytics.example.com/s.js" strategy="afterInteractive" />
<Script src="https://chat-widget.com/w.js" strategy="lazyOnload" />
<Script id="critical" strategy="beforeInteractive" src="/polyfill.js" />

// strategy:
// beforeInteractive → hydration కి ముందు (polyfills మాత్రమే – page ని ఆపుతుంది)
// afterInteractive → (default) hydration తర్వాత – analytics
// lazyOnload → idle time లో – chat widgets, ads ✅
// worker → web worker లో (experimental, Partytown)
```

Third-party scripts = INP యొక్క #1 హంతకుడు. ప్రతి script ని audit చేయాలి: "ఇది నిజంగా అవసరమా? `lazyOnload` సరిపోతుందా?"

Core Web Vitals – Next.js-specific పరిష్కారాలు

Metric	లక్ష్యం	Next.js పరిష్కారాలు
LCP < 2.5s	hero image/text	static/ISR, <code>priority</code> image, <code>next/font</code> preload, streaming shell, CDN
INP < 200ms	interaction latency	client bundle ↓, <code>next/script</code> lazy, transitions, heavy work ని server కి
CLS < 0.1	layout shift	<code>next/image</code> dimensions, <code>next/font</code> , skeleton sizes, ad slots reserve
TTFB < 800ms	server response	static/CDN, streaming, DB queries, edge, <code>preferredRegion</code>

```
// Web Vitals monitoring
"use client";
import { useReportWebVitals } from "next/web-vitals";

export function WebVitals() {
  useReportWebVitals((metric) => {
    fetch("/api/vitals", {
      method: "POST",
      body: JSON.stringify({ name: metric.name, value: metric.value, id: metric.id,
rating: metric.rating }),
      keepalive: true, // 🔑 page unload లో కూడా పంపుతుంది
    });
  });
};
```

```
    return null;
  }
```

Database & API performance

```
// 1. Indexes (అత్యధిక ప్రభావం)
// prisma/schema.prisma
model Order {
  id          String   @id @default(cuid())
  userId      String
  status      String
  createdAt   DateTime @default(now())
  @@index([userId, createdAt]) // ✅ common query pattern
  @@index([status])
}

// 2. Parallel fetching (Topic 11)
// 3. Caching (Topic 12)
// 4. Pagination – offset కంటే cursor
// 5. Select మాత్రమే కావలసినవి
```

preferredRegion – latency తగ్గింపు

```
export const preferredRegion = "bom1"; // Mumbai – DB కి దగ్గరగా
// ⚠️ Function ని users కి దగ్గరగా పెట్టడం కంటే **DB కి దగ్గరగా** పెట్టడం మేలు
// (DB round-trips సాధారణంగా అనేకం ఉంటాయి)
```

PPR + streaming కలిపి (ultimate pattern)

```
export const experimental_ppr = true;

export default async function ProductPage({ params }: Props) {
  const { id } = await params;
  return (
    <>
      <ProductDetails id={id} /> // /* static shell – CDN ✂ */
      <Suspense fallback=<PriceSkeleton />>
        <LivePricing id={id} /> // /* dynamic – stream */
      </Suspense>
      <Suspense fallback=<RecsSkeleton />>
        <Recommendations id={id} /> // /* personalized – stream */
      </Suspense>
    </>
  );
}
```

Gotchas

- **Profiling లేకుండా optimize చేయడం** – bundle analyzer + Lighthouse + RUM ముందు.
- **ssr: false ని అలవాటుగా** → SEO + LCP నష్టం.
- **priority అన్ని images కి** → LCP పాడవుతుంది.

- **Third-party scripts** ని `beforeInteractive` → hydration ఆలస్యం.
- **Dev mode లో performance కొలవడం** → prefetch/optimization disabled; **production build తో test చేయాలి.**
- **Lighthouse score మాత్రమే చూడటం** → field data (CrUX/RUM) నిజం చెప్తుంది.

Key Points

- Priority: rendering strategy → bundle → images/fonts → waterfalls → streaming → caching → scripts.
- `next/dynamic` + `next/script` strategies = client cost నియంత్రణ.
- CWV: LCP (static/priority), INP (bundle/scripts), CLS (dimensions/fonts), TTFB (CDN/streaming).
- **Production build తో మాత్రమే measure చేయాలి.**

26. Deployment – Vercel, Self-Host, Docker

Vercel (zero-config)

git push → build → deploy

- ✓ Automatic: CDN, image optimization, ISR, edge middleware, preview deployments, analytics, cron jobs, log drains
- ✗ Trade-offs: vendor lock-in, cost at scale, function timeout limits, cold starts

```
// vercel.json
{
  "buildCommand": "prisma generate && next build",
  "crons": [{ "path": "/api/cron/cleanup", "schedule": "0 3 * * *" }],
  "functions": { "app/api/heavy/route.ts": { "maxDuration": 60, "memory": 3008 } },
  "regions": ["bom1"]
}
```

Self-hosting – Docker standalone

```
// next.config.ts
export default { output: "standalone" }; // 🗝️ minimal server bundle
(node_modules trace)
```

```
# Multi-stage build
FROM node:22-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci

FROM node:22-alpine AS builder
```

```

WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
# ⚠️ NEXT_PUBLIC_* build time లో inline అవుతాయి - ఇక్కడ ఇవ్వాలి
ARG NEXT_PUBLIC_APP_URL
ENV NEXT_PUBLIC_APP_URL=$NEXT_PUBLIC_APP_URL
RUN npx prisma generate && npm run build

FROM node:22-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -g 1001 -S nodejs && adduser -S nextjs -u 1001

COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./next/static

USER nextjs
EXPOSE 3000
ENV PORT=3000 HOSTNAME="0.0.0.0"
CMD ["node", "server.js"]

```

Image size: standalone తో ~150 MB (సాధారణ build ~1 GB).

Self-host లో గమనించాల్సినవి

అంశం	Vercel	Self-host లో మనం చేయాలి
Image optimization	✅ automatic	<code>sharp</code> install; CPU/memory; లేదా CDN loader
ISR/Data cache	✅ distributed	Multiple instances = separate caches → shared cache handler (Redis)
CDN	✅	CloudFront/Cloudflare configure చేయాలి
Middleware	✅ edge	Node.js లో run అవుతుంది (edge కాదు)
Cron	✅	system cron / external scheduler
Log drains	✅	stdout → log aggregator

```

// cache-handler.js - shared ISR cache (multi-instance కి తప్పనిసరి)
// next.config.ts: { cacheHandler: require.resolve("./cache-handler.js"),
cacheMaxMemorySize: 0 }
const { CacheHandler } = require("@neshca/cache-handler");
CacheHandler.onCreation(async () => {
  const client = createClient({ url: process.env.REDIS_URL });
  await client.connect();
  return { handlers: [await createRedisHandler({ client })] };
});
module.exports = CacheHandler;

```

Environment variables – build vs runtime (అత్యంత సాధారణ గందరగోళం)

NEXT_PUBLIC_* → BUILD TIME లో bundle లోకి inline
⚠️ Docker image build తర్వాత మార్పులేం!
→ ప్రతి environment కి వేరే image, లేదా runtime config pattern

Server-only → RUNTIME లో చదవబడతాయి ✅ (container env vars పని చేస్తాయి)

```
// Runtime public config pattern (ఒకే image, అనేక environments)
// app/layout.tsx (server)
const publicConfig = { apiUrl: process.env.API_URL, env: process.env.APP_ENV };
<script dangerouslySetInnerHTML={{ __html:
`window.__CONFIG__=${JSON.stringify(publicConfig)} ` }} />
```

CI/CD pipeline

```
name: CI
on: [push, pull_request]
jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npx prisma generate
      - run: npm run typecheck          # tsc --noEmit
      - run: npm run lint
      - run: npm run test
      - run: npm run build
      - run: npx playwright test        # E2E
```

Production checklist

- ☐ Security headers (CSP, HSTS, X-Content-Type-Options, Referrer-Policy)
- ☐ Env validation at startup (Zod) + secrets audit (NEXT_PUBLIC_ లో లేవని)
- ☐ Error monitoring (Sentry) + source maps upload (public కాదు)
- ☐ Structured logging + PII redaction
- ☐ Rate limiting (auth endpoints, server actions, APIs)
- ☐ DB: connection pooling, indexes, backups, migrations plan
- ☐ Caching: CDN headers, ISR strategy, shared cache handler (self-host)
- ☐ Health check endpoint (/api/health) + uptime monitoring
- ☐ Analytics + Web Vitals RUM
- ☐ Preview deployments + staging environment
- ☐ Rollback plan; feature flags for risky releases
- ☐ robots.txt, sitemap.xml, canonical URLs

Gotchas

- **NEXT_PUBLIC_ ని runtime env అనుకోవడం** → Docker లో మారవు.

- **Self-host multi-instance లో shared cache లేకపోవడం** → inconsistent ISR (ఒక instance లో కొత్తది, ఇంకోదాంట్లో పాతది).
- **sharp లేకపోవడం** (self-host) → image optimization నెమ్మది/fail.
- **Source maps ని public గా serve చేయడం** → source code leak.
- **Serverless లో DB pooling లేకపోవడం** → connection exhaustion (Topic 20).
- **Function timeout** (Vercel hobby 10s, pro 60s) → long jobs కి queue/worker.
- **Cold starts** – edge runtime వేగం కానీ పరిమితం; heavy Node functions నెమ్మది.

Key Points

- Vercel = zero-config; self-host = `output: "standalone"` + Docker.
- Self-host లో: `sharp`, **shared cache handler**, CDN, cron – మనమే చేయాలి.
- `NEXT_PUBLIC_*` = **build-time inline** (Docker లో environment-specific images).
- CI: typecheck + lint + test + build + E2E.

Interview దృష్టి

- "Next.js ని Vercel లేకుండా deploy చేయగలవా?" → standalone + Docker + sharp + cache handler + CDN – వివరాలు తెలిస్తే బలమైన signal.
- "ISR self-host లో ఎలా పని చేస్తుంది?" → filesystem cache + multi-instance సమస్య + Redis handler.

27. Testing Next.js Apps

వివరణ – testing pyramid

E2E (Playwright)	– critical user flows (login, checkout) – నెమ్మది, ఖరీదు
Integration (RTL + MSW)	– components + data + interactions ← 🏆 అత్యధిక ROI
Unit (Vitest)	– pure logic, utils, validation schemas
Static	– TypeScript + ESLint

Unit & integration (Vitest + RTL)

```
// vitest.config.ts
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";
import tsconfigPaths from "vite-tsconfig-paths";

export default defineConfig({
  plugins: [react(), tsconfigPaths()],
  test: { environment: "jsdom", globals: true, setupFiles: "./test/setup.ts" },
});
```

```
// Client component test
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";

it("adds item to cart", async () => {
  const user = userEvent.setup();
  const addToCart = vi.fn();
  render(<AddToCartButton productId="1" onAdd={addToCart} />);

  await user.click(screen.getByRole("button", { name: /add to cart/i }));
  expect(addToCart).toHaveBeenCalled("1");
});
```

⚠ Server Components testing – ఇంకా పరిష్కారం కాలేదు

```
// ❌ async Server Components ని RTL సరిగ్గా render చేయలేదు (experimental support మాత్రమే)
// ✅ ఆచరణాత్మక వ్యూహం:

// 1. Data layer ని విడిగా test చేయడం (pure functions)
it("getProducts filters inactive", async () => {
  const products = await getProducts({ active: true });
  expect(products.every((p) => p.active)).toBe(true);
});

// 2. Presentational భాగాన్ని విడగొట్టి test చేయడం
// ProductPage (server, data fetch) → ProductView (pure, testable) ✅
export function ProductView({ product }: { product: Product }) { /* testable */ }

// 3. Server Components ని E2E తో test చేయడం (అత్యంత నమ్మదగినది)
```

Server Actions testing

```
// Actions = plain async functions → నేరుగా test చేయొచ్చు ✅
import { createProduct } from "@app/actions/product";

vi.mock("@/lib/dal", () => ({ verifySession: vi.fn().mockResolvedValue({ userId: "1", role: "admin" }) }));

it("validates required fields", async () => {
  const formData = new FormData();
  formData.set("name", "");
  const result = await createProduct({}, formData);
  expect(result.fieldErrors?.name).toBeDefined();
});

it("rejects unauthorized users", async () => {
  vi.mocked(verifySession).mockResolvedValueOnce({ userId: "1", role: "user" });
  const result = await createProduct({}, validFormData);
  expect(result.error).toBe("Forbidden");
});
```

Route handlers testing

```
import { GET } from "@app/api/products/route";
import { NextRequest } from "next/server";

it("returns products", async () => {
  const request = new NextRequest("http://localhost:3000/api/products?page=1");
  const response = await GET(request);
  expect(response.status).toBe(200);
  const data = await response.json();
  expect(Array.isArray(data)).toBe(true);
});
```

E2E (Playwright) – Next.js కి అత్యంత విలువైనది

```
// playwright.config.ts
export default defineConfig({
  webServer: {
    command: "npm run build && npm run start", // 🔑 production build తో test
    url: "http://localhost:3000",
    reuseExistingServer: !process.env.CI,
  },
  use: { baseURL: "http://localhost:3000", trace: "on-first-retry" },
  projects: [{ name: "chromium", use: devices["Desktop Chrome"] }],
});
```

```
// e2e/checkout.spec.ts
test("user can complete checkout", async ({ page }) => {
  await page.goto("/products");
  await page.getByRole("button", { name: "Add to cart" }).first().click();
  await page.getByRole("link", { name: "Cart" }).click();
  await expect(page.getByText("1 item")).toBeVisible();

  await page.getByRole("button", { name: "Checkout" }).click();
  await page.getByLabel("Card number").fill("4242424242424242");
  await page.getByRole("button", { name: "Pay" }).click();

  await expect(page.getByRole("heading", { name: /order confirmed/i })).toBeVisible();
});

// Auth state reuse (login ప్రతి test లో వద్దు)
test.use({ storageState: "playwright/.auth/user.json" });
```

Key Points

- **Server Components testing ఇంకా పరిష్కారం కాలేదు** → data layer + presentational split + E2E.
- **Server Actions = plain functions** → నేరుగా testable (auth mock తో).
- E2E ని **production build** తో run చేయాలి (dev behaviour వేరు).
- Integration tests (RTL + MSW) = అత్యధిక ROI.

Interview దృష్టి

- "Server components ఎలా test చేస్తావు?" → పరిమితులను గుర్తించడం + practical strategy – నిజాయితీ ఇక్కడ విలువైనది.
-

Part 6 – Architecture & Interview

28. Scalable Project Architecture

వివరణ – layered architecture

```
src/
├── app/                                # 🚦 ROUTING LAYER (thin!)
│   ├── (marketing)/
│   ├── (app)/
│   └── api/
├── components/                         # 🍌 PRESENTATION LAYER
│   ├── ui/                            # design system (Button, Input, Dialog)
│   └── features/                       # feature components
│       ├── products/
│       └── cart/
├── server/                             # 🗝️ SERVER-ONLY LAYER
│   ├── actions/                       # Server Actions (thin – validate + call service)
│   ├── services/                      # 📦 BUSINESS LOGIC (framework-agnostic ✅)
│   │   ├── product.service.ts
│   │   └── order.service.ts
│   └── repositories/                 # 🗄️ DATA ACCESS
│       └── product.repository.ts
├── lib/                                # 🛠️ SHARED UTILITIES
│   ├── db.ts
│   ├── dal.ts                        # auth + data access
│   ├── env.ts
│   └── utils.ts
└── types/
```

Request flow

```
Browser
  ↓
Middleware (optimistic auth, redirects)
  ↓
app/page.tsx (routing – thin)
  ↓
server/services/*.ts (business logic – testable, framework-free)
  ↓
server/repositories/*.ts (data access)
  ↓
lib/db.ts (Prisma)
  ↓
Database
```

```
// ✅ Server Action = thin (validate → authorize → delegate)
"use server";
export async function createOrder(prev: State, formData: FormData): Promise<State> {
  const session = await verifySession(); // 🔒 auth
  const parsed = orderSchema.safeParse(Object.fromEntries(formData)); // ✅
  validate
  if (!parsed.success) return { fieldErrors: parsed.error.flatten().fieldErrors };

  try {
    const order = await orderService.create(session.userId, parsed.data); // 📦
  } catch (e) {
    if (e instanceof OutOfStockError) return { error: "Item out of stock" };
    logger.error({ e }, "order creation failed");
    return { error: "Something went wrong" };
  }
}

// 📦 Service – Next.js గురించి తెలియదు → పూర్తిగా testable
export class OrderService {
  constructor(private orders: OrderRepository, private inventory:
InventoryRepository) {}

  async create(userId: string, input: CreateOrderInput): Promise<Order> {
    const stock = await this.inventory.check(input.items);
    if (!stock.available) throw new OutOfStockError(stock.missing);
    return this.orders.createWithInventoryUpdate(userId, input);
  }
}
```

ఎందుకు ఈ layering: business logic Next.js మీద ఆధారపడకపోతే – unit test చేయొచ్చు, reuse చేయొచ్చు (cron/worker/CLI లో), framework మారినా నిలుస్తుంది.

Feature-based organization (పెద్ద teams కి)

```
src/features/
├── products/
│   ├── components/
│   ├── actions/
│   ├── services/
│   ├── schemas/
│   ├── types.ts
│   └── index.ts # 🔑 public API – బయటివారు ఇది మాత్రమే import చేయాలి
└── cart/
```

Dependency rule: `app/` → `features/*` → `lib/` ; feature-to-feature imports `index.ts` ద్వారానే. ESLint (`import/no-restricted-paths`) తో enforce చేయాలి.

Monorepo (multiple apps)

```
apps/  
├─ web/           (Next.js – customer)  
├─ admin/         (Next.js – internal)  
└─ docs/  
packages/  
├─ ui/            (shared design system)  
├─ database/      (Prisma schema + client)  
├─ config/        (eslint, tsconfig, tailwind presets)  
└─ shared/        (types, validation schemas, utils)
```

Tools: pnpm workspaces + Turborepo (caching, task graph).

Key Points

- `app/` = **thin routing layer**; logic services లో.
- Services **framework-agnostic** → testable + reusable.
- Server Action = validate → authorize → delegate → revalidate.
- Feature-based + public API + dependency rule (ESLint enforced).

29. Pages Router → App Router Migration

వివరణ – incremental migration

రెండు routers ఒకే app లో పని చేస్తాయి – big-bang rewrite అవసరం లేదు.

```
app/dashboard/page.tsx    ← ఇది గెలుస్తుంది  
pages/dashboard.tsx       ← ignore అవుతుంది (conflict అయితే build error)
```

Migration క్రమం

1. Next.js ని latest కి upgrade (Pages Router లోనే)
2. `app/layout.tsx` create (root layout)
3. కొత్త features ని `app/` లో మాత్రమే రాయడం
4. Leaf routes ముందు migrate (dependencies తక్కువ)
5. Shared components ని క్రమంగా server components కి
6. API routes (`pages/api` → `app/api`) చివర్లో
7. `pages/` ఖాళీ అయ్యేక తొలగింపు

API mapping

Pages Router	App Router
<code>getServerSideProps</code>	async Server Component (+ dynamic APIs)

Pages Router	App Router
<code>getStaticProps</code>	async Server Component + <code>revalidate</code>
<code>getStaticPaths</code>	<code>generateStaticParams</code>
<code>getInitialProps</code>	(తెలగించాలి)
<code>_app.tsx</code>	<code>app/layout.tsx</code> + providers
<code>_document.tsx</code>	<code>app/layout.tsx</code> (<code><html></code> / <code><body></code>)
<code>next/router</code> (<code>useRouter</code>)	<code>next/navigation</code> (<code>useRouter</code> , <code>usePathname</code> , <code>useSearchParams</code>)
<code>router.query</code>	<code>params</code> prop + <code>useSearchParams</code>
<code>pages/api/*</code>	<code>app/api/*/route.ts</code>
<code>next/head</code>	Metadata API
<code>_error.tsx</code> / <code>404.tsx</code>	<code>error.tsx</code> / <code>not-found.tsx</code>
<code>next/image</code> (legacy props)	కొత్త <code>next/image</code> API

```
// ❌ Pages Router
export const getServerSideProps: GetServerSideProps = async ({ params, req }) => {
  const product = await getProduct(params!.id as string);
  if (!product) return { notFound: true };
  return { props: { product } };
};
export default function ProductPage({ product }: Props) { return <View product=
{product} />; }

// ✅ App Router
export default async function ProductPage({ params }: { params: Promise<{ id: string
}> }) {
  const { id } = await params;
  const product = await getProduct(id);
  if (!product) notFound();
  return <View product={product} />;
}
```

సాధారణ migration సవాళ్లు

సవాలు	పరిష్కారం
CSS-in-JS (styled-components)	registry + <code>'use client'</code> , లేదా Tailwind/CSS Modules కి migrate
Global state (Redux/Context)	<code>'use client'</code> provider in layout; server data ని props గా
<code>useRouter().query</code>	<code>params</code> prop (server) / <code>useSearchParams</code> (client)
Auth HOCs	middleware + DAL pattern

సవాలు	పరిష్కారం
Heavy client libraries	<code>dynamic({ ssr: false })</code>
<code>getInitialProps</code>	పూర్తిగా rewrite
Third-party libraries RSC-incompatible	<code>'use client'</code> wrapper

Migration ఎప్పుడు వద్దు

- App ఇప్పటికే బాగా పని చేస్తోంది, కొత్త features లేవు.
- Team కి RSC నేర్చుకునే bandwidth లేదు.
- భారీగా CSS-in-JS మీద ఆధారపడి ఉంది.
- Deadline దగ్గరలో ఉంది.

Pages Router deprecated కాదు – ఇంకా maintained. Migration ఒక **business decision**, technical fashion కాదు.

Key Points

- **Incremental** – రెండు routers సహజీవనం చేస్తాయి.
- Leaf routes ముందు; API routes చివర్లో; కొత్త code app/ లోనే.
- `next/router` → `next/navigation` (అత్యంత సాధారణ మార్పు).
- CSS-in-JS + global state = అతిపెద్ద సవాళ్లు.

30. Anti-Patterns & Common Pitfalls

Rendering & Components

❌ Anti-pattern	✅ సరైనది
Root layout/page లో <code>'use client'</code>	Boundary ని leaf కి దగ్గరగా
Server component లో <code>useEffect</code> fetch	<code>await</code> నేరుగా server లో
Client component లోకి server component import	<code>children</code> /props గా pass
Server → client కి పెద్ద data objects	Server లో render, తక్కువ props
<code>dynamic({ssr:false})</code> అలవాటుగా	SEO/LCP అవసరమైతే server render
Page component లో అన్ని <code>await</code>	Child components + Suspense

Data & Caching

✗	✓
Sequential <code>await</code> (waterfall)	<code>Promise.all</code> / preload / Suspense
ORM calls ని <code>React.cache</code> లేకుండా	<code>cache()</code> wrap (request dedup)
Mutation తర్వాత <code>revalidate</code> మర్చిపోవడం	<code>revalidatePath</code> / <code>revalidateTag</code>
<code>revalidatePath("/", "layout")</code> అలవాటుగా	Fine-grained tags
Dev లో caching test చేయడం	<code>next build && next start</code>
Next 14 caching అంచనాలతో Next 15 code	Explicit <code>cache</code> / <code>revalidate</code>

Security

✗	✓
Server Action లో auth check లేకపోవడం	ప్రతి action లో auth + authz + validate
Middleware ని authorization కి	DAL లో enforce (defense in depth)
Ownership check లేకపోవడం	<code>where: { id, userId }</code> (IDOR నివారణ)
<code>NEXT_PUBLIC_</code> లో secrets	Server-only env + <code>server-only</code> package
Client కి మొత్తం DB object	DTO / <code>select</code>
Webhook signature verify చేయకపోవడం	Signature + idempotency
Internal error messages return	Generic messages + server logs

Performance

✗	✓
<code></code> tag	<code>next/image</code> (+ <code>sizes</code> , <code>priority</code> ఒకటి)
<code><link></code> Google Fonts	<code>next/font</code>
Heavy libs static import	<code>next/dynamic</code>
Third-party scripts <code>beforeInteractive</code>	<code>lazyOnload</code> / <code>afterInteractive</code>
అనుకోని <code>force-dynamic</code>	Build output audit
Bundle analyzer వాడకపోవడం	ప్రతి release కి check

Key Points

5 అత్యంత ప్రమాదకరమైన Next.js తప్పులు:

1. **Server Action లో auth check లేకపోవడం** (security hole)
2. **Root లో 'use client'** (RSC ప్రయోజనం సున్నా)
3. **Middleware = authorization అనుకోవడం** (bypass risk)
4. **Data waterfalls** (నెమ్మది)
5. **Caching semantics అర్థం కాకపోవడం** (stale data / అధిక API cost)

31. Rapid-Fire Interview Q&A

A. Fundamentals

1. **Next.js ఎందుకు React కంటే?** Routing/rendering/data/API/optimization decisions ready; SEO + LCP; RSC bundle తగ్గింపు; full-stack ఒకే repo. (SPA/internal tools కి అనవసరం కావచ్చు.)
2. **App Router vs Pages Router?** RSC, nested layouts (state persistence), streaming, Server Actions, file conventions vs `getServerSideProps` family.
3. **Next.js default static ఆ dynamic ఆ? Static** – dynamic APIs (`cookies` , `headers` , `searchParams` , `force-dynamic`) వాడితేనే dynamic.
4. **SSG/SSR/ISR/PPR తేడా?** Build-time / per-request / stale-while-revalidate / static shell + dynamic holes.
5. **ISR ఎలా పని చేస్తుంది?** Cached version serve → background regenerate → తర్వాతి requests కి కొత్తది. ఏ user వేచి ఉండడు.

B. RSC & Rendering

6. **Server vs Client Components?** Server: zero JS, async, DB/secrets; Client: state/effects/events/browser APIs.
7. **'use client' ఏం చేస్తుంది? Module boundary** – ఆ file + దాని imports client bundle లోకి. Leaf కి దగ్గరగా ఉంచాలి.
8. **Client లో server component వాడొచ్చా?** Import ❌; **children/props గా pass** ✅.
9. **RSC payload అంటే?** Server components యొక్క serialized rendered output + client component references; HTML లో embed అవుతుంది; navigation లో stream అవుతుంది.
10. **Hydration mismatch కారణాలు?** Time/random, browser APIs, invalid HTML nesting, extensions, locale.
11. **Streaming ఎలా enable చేస్తావు?** `loading.tsx` (automatic) లేదా `<Suspense>` + `await` ని child component లోకి తరలించడం.
12. **PPR ఏం పరిష్కరిస్తుంది?** Per-page static/dynamic binary choice – ఒకే page లో static shell + dynamic holes.

C. Caching ★

13. Next.js caching layers ఏమిటి? Request Memoization (per-request `fetch dedup`) → Data Cache (persistent fetch results) → Full Route Cache (rendered HTML/RSC) → Router Cache (client RSC payloads).

14. Next 15 లో caching ఏం మారింది? `fetch` మరియు GET route handlers **default uncached**; client router cache 0s; `params / cookies` async.

15. ORM calls cache అవుతాయా? Data Cache `fetch` కి మాత్రమే → `React.cache` (dedup) + `unstable_cache / use_cache` (persistent).

16. `revalidatePath` vs `revalidateTag` ? Route-based vs data-tag-based; tags cross-route fine-grained invalidation.

17. Revalidate చేసినా UI లో పాత data – ఎందుకు? Client Router Cache; Server Action లో `revalidate` చేస్తే automatic clear, route handler నుండి కాదు → `router.refresh()` .

18. Dev లో caching ఎందుకు వేరు? Dev లో చాలా caching disabled – `next build && next start` లో test చేయాలి.

D. Mutations & APIs

19. Server Action అంటే? Client నుండి call చేయగల server function (`"use server"`); progressive enhancement; POST endpoint.

20. Server Actions security? అవి **public endpoints** → ప్రతి action లో `auth + authz + validation + rate limit`. "Server code కాబట్టి safe" ❌.

21. Server Action vs Route Handler? Your UI mutations vs external consumers/webhooks/streaming/REST.

22. Webhook ని ఎలా handle చేస్తావు? Raw body + signature verify + idempotency key + fast 200 + async processing.

23. `useActionState / useFormStatus / useOptimistic` ? Action state+pending / child-level pending / optimistic UI + auto rollback.

E. Security & Auth

24. Auth ఎలా architect చేస్తావు? Middleware (optimistic redirect) + **DAL (real authz)** + action checks + UI (UX only).

25. Middleware లో auth చాలదా? చాలదు – bypass risk (CVE-2025-29927); data layer లో enforce చేయాలి.

26. IDOR ఎలా అవుతావు? Ownership check ప్రతి query లో (`where: { id, userId }`) + `notFound()` (403 కాదు).

27. Secrets ఎలా protect చేస్తావు? Server-only env, `server-only` package, DTO, RSC payload audit, `NEXT_PUBLIC_` నివారణ.

F. Performance & Production

28. App నెమ్మది – debug ఎలా? Build output (static/dynamic) → bundle analyzer → waterfalls → caching → images/scripts → DB queries → RUM.

29. Client bundle ఎలా తగ్గిస్తావు? 'use client' boundary కిందకి, next/dynamic, provider audit, heavy libs server లో.

30. Serverless DB connections? Pooler (PgBouncer/Accelerate) లేదా serverless driver + connection_limit=1.

31. Self-host లో ISR ఎలా? Filesystem cache; multi-instance అయితే **shared cache handler (Redis)** తప్పనిసరి.

32. NEXT_PUBLIC_ Docker లో ఎందుకు మారవు? అవి **build time లో inline** అవుతాయి → per-environment images లేదా runtime config injection.

33. Edge vs Node runtime? Web APIs + వేగవంతమైన cold start + పరిమిత size vs పూర్తి Node APIs + DB drivers.

34. Server components ఎలా test చేస్తావు? Data layer + presentational split + E2E (RTL support పరిమితం).

32. Memory Tips

1. ఒక్క వాక్యంలో Next.js

"Server లో వీలైనంత ఎక్కువ, client కి వీలైనంత తక్కువ – దాన్ని సాధ్యం చేసే framework."

2. ప్రతి Next.js నిర్ణయానికి 3 ప్రశ్నలు

- | | |
|---------------------------|--|
| 1. ఇది server లో చేయగలనా? | → అవును అయితే server లోనే (bundle ↓, secrets safe) |
| 2. ఇది cache చేయగలనా? | → అవును అయితే static/ISR (వేగం ↑, cost ↓) |
| 3. ఇది ఎవరు call చేయగలరు? | → public endpoint అయితే auth + validate 🔒 |

3. Rendering decision tree

```
ప్రతి user కి ఒకే content?
├─ అవును → మారుతుందా?
│   └─ లేదు → Static
│   └─ అవును → ISR (+ on-demand revalidation)
└─ కాదు (personalized) → Dynamic + streaming
    └─ page లో కొంత భాగం మాత్రమే personalized? → PPR ★
```

4. Caching mental model – 4 layers

Request Memoization → ఒక request లోపల ఒకే fetch (React)
Data Cache → fetch results (persistent, deploy దాటి)
Full Route Cache → rendered HTML/RSC (build/revalidate వరకు)
Router Cache → browser లో RSC payload (session)

"పాత data కనిపిస్తోంది" → ఫైనలుండి కిందికి ఒక్కొక్కటి check చేయి

5. Analogy map

Concept	Analogy
React vs Next.js	ఇంజిన్ vs పూర్తి కారు
File-based routing	ఇంటి address system
Server vs Client Component	Kitchen vs table మీద salt shaker
RSC	Dish మాత్రమే వస్తుంది, వంటగది కాదు
Streaming	Starter → main course → dessert
PPR	ముందే print చేసిన menu + "today's special" ఖాళీ గడి
ISR	రేజిస్ట్రీ కొత్త menu print
4 caching layers	ఇంట్లో నీళ్ళ వ్యవస్థ (ట్యాంక్ → బాటిల్)
Middleware	Gate security (త్వరగా చూస్తాడు, DB చూడలేడు)
Server Actions	ఇంటర్కామ్ బటన్ (కానీ బయట కూడా ఉంది!)
Prefetching	అతిథి వచ్చే ముందే టీ పెట్టడం
DAL	ఇంట్లో ప్రతి గదికి తాళం (gate ఒక్కటే కాదు)

6. Version cheat-sheet

13 = App Router + RSC · 13.4 = stable + Server Actions · 14 = Actions stable + PPR preview
15 = React 19 + uncached-by-default + async params + Turbopack

7. Interview కి ముందు 30 నిమిషాల revision

1. Static/Dynamic/ISR/Streaming/PPR + ఏది ఎప్పుడు (5 min)
2. Server vs Client Components + composition నియమాలు (5 min)
3. 4 caching layers + Next 15 మార్పులు (6 min) ★
4. Server Actions + security (4 checks) (4 min) ★
5. Auth architecture (middleware → DAL → action → UI) (4 min)
6. Waterfalls, preload, React.cache (3 min)
7. Deployment (standalone, cache handler, NEXT_PUBLIC_) (3 min)

8. చివరి మాట

Next.js లో **API లు గుర్తుపెట్టుకోవడం కాదు – ఏ code ఎక్కడ run అవుతోంది అనే స్పష్టత** ముఖ్యం.

ప్రతి line రాసేటప్పుడు మిమ్మల్ని మీరు అడగండి:

- "ఇది server లో run అవుతుందా, client లో అవుతుందా?"
- "ఇది cache అవుతుందా? ఎప్పుడు invalidate అవుతుంది?"
- "దీన్ని బయటివారు call చేయగలరా?"

ఈ మూడు ప్రశ్నలు అలవాటైతే – Next.js లోని 90% bugs (stale data, security holes, hydration errors, bundle bloat) మీ code లోకి రావు.

SDE2 అంటే – App Router లో feature ని సరిగ్గా, secure గా build చేయగలగడం. **SSE అంటే** – **rendering strategy, caching strategy, security boundaries** ని architect చేయగలగడం; trade-offs (cost, latency, freshness, complexity) ని numbers తో justify చేయగలగడం; **ఎప్పుడు Next.js వద్దే కూడా** చెప్పగలగడం.

గుర్తుంచుకోండి: Next.js యొక్క ప్రతి feature ఒక trade-off. Static వేగం కానీ stale; dynamic తాజా కానీ ఖరీదు; RSC bundle తగ్గిస్తుంది కానీ mental model సంక్లిష్టం; Server Actions సులభం కానీ **అవి public endpoints**. ఈ trade-offs తెలిసినవాడే framework ని నిజంగా తెలిసినవాడు.

అనుబంధం – వేగవంతమైన Reference

File conventions

```
page.tsx · layout.tsx · template.tsx · loading.tsx · error.tsx · global-error.tsx
not-found.tsx · route.ts · default.tsx · middleware.ts · instrumentation.ts
sitemap.ts · robots.ts · opengraph-image.tsx · manifest.ts · icon.tsx
```

Route segment config

```
export const dynamic = "auto" | "force-dynamic" | "error" | "force-static";
export const revalidate = false | 0 | number;
export const runtime = "nodejs" | "edge";
export const preferredRegion = "auto" | "global" | "home" | string[];
export const maxDuration = 30;
export const fetchCache = "auto" | "default-cache" | "force-cache" | "force-no-store";
export const experimental_ppr = true;
export const dynamicParams = true;
```

Caching APIs

```
fetch(url, { cache: "force-cache" | "no-store" })
fetch(url, { next: { revalidate: 60, tags: ["products"] } })
revalidatePath("/products") · revalidatePath("/p/[id]", "page") ·
revalidateTag("products")
unstable_cache(fn, keys, { revalidate, tags })
import { cache } from "react" // request-level dedup
"use cache" + cacheTag() + cacheLife() // Next 15.x
```

ముఖ్య imports

```
import { cookies, headers, draftMode } from "next/headers"; // async (Next 15)
import { redirect, permanentRedirect, notFound, useRouter,
  usePathname, useSearchParams, useParams } from "next/navigation";
import { revalidatePath, revalidateTag, unstable_cache } from "next/cache";
import { NextRequest, NextResponse, after } from "next/server";
import Link from "next/link"; import Image from "next/image";
import Script from "next/script"; import dynamic from "next/dynamic";
```

Production checklist

- ☐ strict TypeScript + tsc --noEmit in CI
- ☐ Env validation (Zod) at startup; secrets audit (NEXT_PUBLIC_ లో లేవు)
- ☐ ప్రతి Server Action లో: auth + authz + validation + rate limit 🔒
- ☐ DAL pattern + ownership checks + DTOs 🔒
- ☐ Security headers (CSP, HSTS) + webhook signature verification
- ☐ Caching strategy documented (ఏ route ఏ strategy, revalidate ఎంత)
- ☐ Build output audit (static/dynamic symbols సరిగ్గా ఉన్నాయా)
- ☐ Bundle analyzer + First Load JS < 150 kB
- ☐ next/image + next/font + next/script strategies
- ☐ DB: pooling, indexes, N+1 audit, migrations plan
- ☐ Sentry + structured logging + PII redaction
- ☐ Web Vitals RUM + uptime monitoring
- ☐ E2E tests on production build
- ☐ Self-host: standalone + sharp + shared cache handler + CDN

ఈ guide పూర్తి చేసినందుకు అభినందనలు! Next.js ని నిజంగా నేర్చుకోవడం అంటే – ఒక real app build చేయడం: auth + DB + forms + caching strategy + deployment. అప్పుడు ఈ document ని మళ్ళీ చదవండి; ప్రతి section కీ "అవును, ఇది నేను ఎదుర్కొన్నాను" అనిపిస్తుంది.

Companion docs: [React_Telugu.md](#) · [TypeScript_Telugu.md](#) ·

[JavaScript_Telugu.md](#) · [DBMS_Telugu.md](#) · [Security_Telugu.md](#) · [HLD_Telugu.md](#) ·
[SystemDesign_Go_Telugu.md](#) · [SoftwareEngineering_Telugu.md](#)